

基于混合车队路径优化与滚动重调度的 城市绿色物流配送研究

摘要

本文主要研究城市绿色物流配送中的混合车队调度优化问题。根据客户需求、车辆容量、道路距离、软时间窗、时变速度和绿色配送区限行要求，构建兼顾运营成本、服务时效与碳排放水平的多场景车辆路径优化模型，结合分层启发式搜索、路线—车型匹配、局部路径改进和滚动重调度方法，对静态配送、限行政策约束和突发事件扰动下的车辆使用结构、配送路径、发车时刻及成本构成进行优化求解，并系统分析绿色限行与动态事件对配送成本和低碳效果的影响。

针对问题一，以总配送成本最低为目标，在同时考虑车辆类型差异、载重与容积双容量约束、时变随机速度、软时间窗惩罚、能源消耗和碳排放成本的基础上，建立混合车队路径优化模型，并采用“构造初解—车型分配—迭代改进”的分层启发式策略求解。静态最优方案启用车辆 121 辆，其中新能源车 11 辆、燃油车 110 辆；总行驶距离 15363.84 km，总成本 88982.82 元，碳排放总量 10617.98 kg。无政策限制下车辆启用成本主导总成本，燃油车承担主要配送任务。

针对问题二，在问题一模型基础上引入绿色配送区限行政策，并建立限行时间窗约束和绿色区车型可行性约束。求解时采用“分区建路—车型重分配—局部优化”策略，优先以新能源车服务绿色区任务，再对外围任务进行全局匹配。环保政策影响下的车辆调度策略启用车辆 124 辆，其中新能源车 25 辆、燃油车 99 辆；绿色区 15 条路线中新能源车承担 11 条。政策约束下总成本为 91121.92 元，较问题一增加 2139.10 元；碳排放总量降至 10387.48 kg，减少 230.50 kg。限行政策在小幅增加成本的同时推动了新能源替代和碳排放下降。

针对问题三，进一步考虑订单取消、新增订单、配送地址变更、时间窗调整和车辆故障五类突发事件，建立事件驱动的滚动重调度模型。模型以问题二方案为基线，固定触发时刻前已完成或正在执行的路径，仅对尚未发车任务、备用车辆和可释放车辆进行局部重构，并采用“限行合规优先、服务水平优先、成本与碳排放次优”的词典序评价准则。五类场景下，动态路线数介于 51–84 条，动态总成本介于 33812.35–58985.78 元，动态总迟到介于 6.6931–12.8025 h；平均全天总成本为 90114.00 元，较问题二降低 1007.92 元。除新增订单场景外，其余四类场景均实现成本改善，其中配送地址变更场景改善最大，总成本降低 2693.17 元、碳成本降低 312.48 元。

最后，对关键成本参数和典型突发事件进行敏感性与鲁棒性分析。以问题二方案为基准，碳排放成本系数和迟到惩罚系数在 $\pm 20\%$ 范围内扰动时，总成本变化率均控制在 1.5% 以内；五类动态事件下均保持限行可行，多数场景可降低全天总成本和碳成本。结果表明，模型具有较好的成本稳定性和动态适应能力，可为绿色物流车辆配置、限行政策适应和应急调度提供量化依据。

关键词：车辆路径优化；绿色配送；软时间窗；滚动重调度；低碳物流

一、问题重述

1.1 问题背景

随着城市消费需求增长和即时配送模式普及，物流配送任务呈现出规模扩大、时效要求提高和运行环境复杂化的特征。传统以路径最短或车辆数最少为核心的调度方式，难以适应多目标、多约束并存的运营需求。在“双碳”目标和城市绿色治理背景下，物流配送还需兼顾服务质量、能源消耗和碳排放控制。

混合车队配送调度具有较强综合性。车辆路径问题最早由 Dantzig 和 Ramser 提出 [1]。其后，Solomon 对带时间窗车辆路径问题开展了研究，该问题成为城市配送优化的重要方向 [2]。客户需求受到重量、体积和到达时间约束，车辆速度又随时段变化，路径规划与到达时刻耦合；同时，绿色配送区限行政策改变了不同动力车型的可达范围与使用方式，进一步增加了调度难度。配送过程中还可能出现新增订单、地址调整等动态扰动，要求方案具备实时响应与快速重构能力。

基于上述背景，本文研究时变交通、混合车队、软时间窗、环保约束与动态事件共同作用下的城市绿色配送调度问题。相关研究表明，交通时变性、碳排放成本和新能源车车辆约束会显著影响路径选择与车队配置 [3]，因此有必要构建兼顾经济性、服务性与低碳性的综合优化模型。

1.2 问题概述

题目提供了配送中心与客户点坐标、客户订单、节点间实际道路距离以及客户时间窗等基础数据，为构建城市配送调度模型提供了完整的数据支撑。围绕企业在不同运营情景下面临的决策需求，本文将问题划分为以下三个递进层次：

问题一：在无额外政策限制的静态环境下，综合考虑车辆类型、双容量约束、软时间窗约束以及及时变速度特征，构建以总配送成本最小为目标的车辆调度模型，并给出车辆使用方案、配送路径、到达时刻及成本构成。

问题二：在问题一的基础上，进一步引入绿色配送区内燃油车分时禁行政策，重新规划配送方案，分析环保政策对车队结构、运营成本及碳排放水平的影响。

问题三：针对配送过程中出现的订单新增、订单取消、地址变更、时间窗调整和车辆故障等动态事件，设计可实时响应的滚动重调度策略，并通过典型情景检验模型的适应性与鲁棒性。

二、模型假设

1. 研究周期内车辆从配送中心出发并最终返回配送中心，每辆车至多执行一条完整配送路线，返回后不再安排二次配送。
2. 同一客户的订单需求按客户编号进行汇总；若汇总需求超过可用车型容量上限，则允许拆分为若干配送子任务。
3. 车辆行驶速度按时段划分并考虑随机波动，其中 17:00–19:00 视为一般时段，19:00 之后视为顺畅时段，同时设置最低速度下限。
4. 车辆能耗同时受行驶速度与装载率影响，装载率影响近似为线性关系；燃油车和新能源车的装载附加系数分别取 40% 和 35%。
5. 除完成配送并返回配送中心外，不额外设置车辆必须在当日某一固定时刻前返场的硬约束。
6. 动态事件触发后，仅对尚未完成的配送任务和仍具备调整条件的车辆进行滚动重调度。

三、符号说明

表 1 主要符号说明

符号	含义	单位/说明
$\mathcal{O}$	原始订单集合	—
$G, \tilde{G}$	配送任务集合、含配送中心的扩展节点集合	—
K, A	车辆集合、车型集合	—
d_{ij}	节点 i 到节点 j 的道路距离	km
$q_i^{(w)}, q_i^{(v)}$	客户或任务 i 的重量需求、体积需求	kg, m ³
$Q_k^{(w)}, Q_k^{(v)}$	车辆 k 的载重上限、容积上限	kg, m ³
$[e_i, l_i]$	客户 i 的软时间窗	h
t_i^ω, u_k	情景 ω 下到达客户 i 的时刻、车辆 k 的发车时刻	h
w_i^ω, l_i^ω	情景 ω 下客户 i 的等待时长、迟到时长	h
c_k^{start}	车辆 k 的启动成本	元/辆
$c^{\text{wait}}, c^{\text{late}}$	单位等待成本、单位迟到惩罚成本	元/h
c^{car}	单位碳排放成本	元/kg
$C^{\text{ene}}, C^{\text{car}}$	能源成本、碳排放成本	元
x_{ijk}, y_k	车辆 k 是否从 i 行驶至 j 、车辆 k 是否启用	0-1 变量
g_i, η_k	客户 i 是否位于绿色配送区、车辆 k 是否为燃油车	0-1 变量
$\tau, \mathcal{U}(\tau)$	动态事件触发时刻、触发后待重调度任务集合	h, —

四、问题一模型的建立与求解

4.1 问题分析

问题一属于静态环境下的混合车队车辆路径优化问题，主要难点在于：客户需求同时受到重量和体积双重容量约束，部分客户需拆分后配送；车辆速度具有时段性和随机性，到达时刻会影响等待、迟到及能耗成本；目标为总成本最小，启动成本、能源成本、碳排放成本与时间窗成本相互耦合。该类问题通常属于 NP-hard 组合优化问题，大规模算例中常采用启发式或元启发式方法求解 [5]。

针对上述特征，本文采用“构造初解—车型分配—迭代改进”的求解思路。首先对订单进行补全、聚合与拆分，形成可执行任务集合；其次在车辆数量与容量约束下构造初始路径并完成车型匹配；最后通过路线内重排、跨路线交换、坏路线重组和尾货重平衡等操作降低总成本，得到总配送成本最低的静态调度方案。

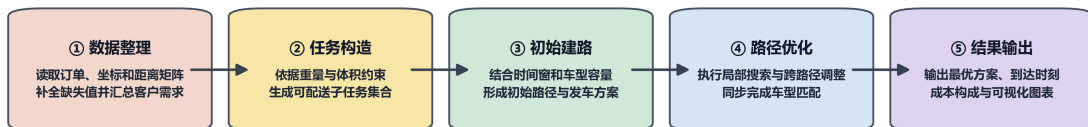


图 1 问题一求解流程图

4.2 模型建立

4.2.1 数据预处理与客户需求聚合

原始订单存在重量或体积字段缺失、同一客户对应多笔订单以及部分客户需求超过单车容量等情况，需先转化为客户层面的可执行配送任务，并同步完成绿色配送区属性识别。

设原始订单集合为 $\mathcal{O}$ ，订单 r 的重量、体积和目标客户分别为 a_r 、 b_r 和 h_r 。记有效订单集合为 $\mathcal{O}^+$ ，平均货物密度为

$$\bar{\rho} = \frac{1}{|\mathcal{O}^+|} \sum_{r \in \mathcal{O}^+} \frac{a_r}{b_r}. \quad (1)$$

对存在单侧缺失的订单，按密度一致性原则补全重量或体积：

$$(\hat{a}_r, \hat{b}_r) = \begin{cases} (a_r, b_r), & a_r > 0, b_r > 0, \\ (\bar{\rho} b_r, b_r), & a_r \text{ 缺失}, b_r > 0, \\ (a_r, a_r / \bar{\rho}), & a_r > 0, b_r \text{ 缺失}. \end{cases} \quad (2)$$

补全后按客户编号聚合需求，并根据最小车型容量进行任务拆分：

$$q_i^{(w)} = \sum_{r \in \mathcal{O}: h_r = i} \hat{a}_r, \quad q_i^{(v)} = \sum_{r \in \mathcal{O}: h_r = i} \hat{b}_r, \quad m_i = \left\lceil \max \left\{ \frac{q_i^{(w)}}{Q_{\min}^{(w)}}, \frac{q_i^{(v)}}{Q_{\min}^{(v)}} \right\} \right\rceil. \quad (3)$$

客户 i 被转化为 m_i 个配送子任务，单个子任务需求为

$$q_{i,r}^{(w)} = \frac{q_i^{(w)}}{m_i}, \quad q_{i,r}^{(v)} = \frac{q_i^{(v)}}{m_i}, \quad r = 1, 2, \dots, m_i. \quad (4)$$

同时，以市中心为圆心识别绿色配送区客户：

$$g_i = \begin{cases} 1, & \sqrt{X_i^2 + Y_i^2} \leq 10, \\ 0, & \sqrt{X_i^2 + Y_i^2} > 10. \end{cases} \quad (5)$$

其中 $g_i = 1$ 表示客户位于绿色配送区。预处理结果见表 2，客户空间分布如图 2 所示，相关任务与空间属性将在后续问题中沿用。

表 2 问题一数据预处理结果

指标	数值	指标	数值
原始订单数	2169	客户数	88
平均货物密度 (kg/m ³)	351.6850	填补后总重量 (kg)	286100.2115
填补后总体积 (m ³)	781.9127	最大客户总重量 (kg)	12197.6473
最大客户总体积 (m ³)	36.0270	拆分后任务数	285
需拆分客户数	61	绿色配送区客户数	15
非绿色配送区客户数	83		

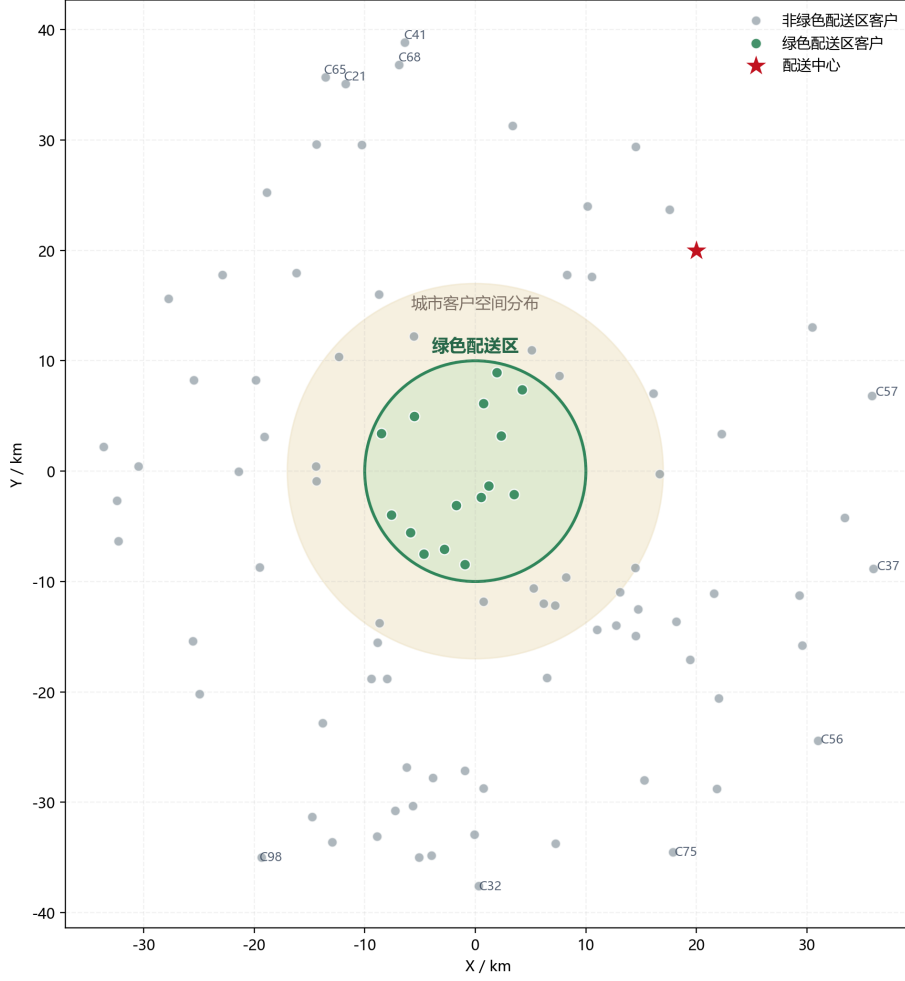


图 2 客户坐标分布图

4.2.2 车辆路径优化模型

经预处理后，将配送子任务记为 $G = \{1, 2, \dots, n\}$ ，配送中心记为节点 0，扩展节点集合为 $\tilde{G} = G \cup \{0\}$ 。记车辆集合为 K ，车辆类型集合为 A ，类型 a 的可用车辆数为 N_a 。速度受时段影响且具有随机性，参考交通流条件下多模糊时间窗车辆路径优化研究中的速度参数设置方法 [4]，用有限情景集合 Ω 表征不同速度实现。对任意情景 ω ，车辆在时刻 t 从节点 i 驶向节点 j 的旅行时间为

$$\tau_{ij}^{\omega}(t) = \inf \left\{ \Delta \geq 0 \mid \int_t^{t+\Delta} v^{\omega}(s) ds \geq d_{ij} \right\}, \quad (6)$$

其中 $v^{\omega}(s)$ 为情景 ω 下时刻 s 的运行速度， d_{ij} 为节点间距离。

定义路径决策变量和车辆启用变量为

$$x_{ijk} = \begin{cases} 1, & \text{若车辆 } k \text{ 从节点 } i \text{ 行驶至节点 } j, \\ 0, & \text{否则,} \end{cases} \quad y_k = \begin{cases} 1, & \text{若车辆 } k \text{ 被启用,} \\ 0, & \text{否则.} \end{cases}$$

同时记 u_k 为车辆 k 的发车时刻， t_i^{ω} 为情景 ω 下到达任务 i 的时刻， w_i^{ω} 和 l_i^{ω} 分别表示等待时长与迟到时长。若车辆 k 在服务任务 i 前的剩余载重为 L_{ik} ，则装载率为 $\lambda_{ik} = L_{ik}/Q_k^{(w)}$ 。

问题一以期望总成本最小为目标，其目标函数写为

$$\min Z_1 = C^{\text{start}} + \frac{1}{|\Omega|} \sum_{\omega \in \Omega} (C_{\omega}^{\text{ene}} + C_{\omega}^{\text{car}} + C_{\omega}^{\text{wait}} + C_{\omega}^{\text{late}}). \quad (7)$$

其中，车辆启动成本为

$$C^{\text{start}} = \sum_{k \in K} c_k^{\text{start}} y_k. \quad (8)$$

在情景 ω 下，等待成本和迟到成本分别表示为

$$C_{\omega}^{\text{wait}} = c^{\text{wait}} \sum_{i \in G} w_i^{\omega}, \quad C_{\omega}^{\text{late}} = c^{\text{late}} \sum_{i \in G} l_i^{\omega}. \quad (9)$$

设车辆 k 的动力类型记为 $p(k)$ ，单位能耗函数记为 $\phi_{p(k)}(v)$ ，则有

$$\phi_{p(k)}(v) = \begin{cases} 0.0025v^2 - 0.2554v + 31.75, & p(k) = \text{燃油车}, \\ 0.0014v^2 - 0.12v + 36.19, & p(k) = \text{新能源车}. \end{cases} \quad (10)$$

进一步记 $\bar{v}_{ij}^{\omega}$ 为情景 ω 下车辆在弧 (i, j) 上的等效运行速度， λ_{ik} 为车辆 k 服务任务 i 前的装载率， $\alpha_{p(k)}$ 为装载附加系数，则情景 ω 下的能源成本可表示为

$$C_{\omega}^{\text{ene}} = \sum_{k \in K} \sum_{i \in \tilde{G}} \sum_{j \in \tilde{G}, j \neq i} x_{ijk} \cdot \frac{d_{ij}}{100} \cdot \pi_{p(k)} \cdot \phi_{p(k)}(\bar{v}_{ij}^{\omega}) (1 + \alpha_{p(k)} \lambda_{ik}), \quad (11)$$

其中 $\pi_{p(k)}$ 表示对应动力类型的单位能源价格。类似地，碳排放成本可写为

$$C_{\omega}^{\text{car}} = \sum_{k \in K} \sum_{i \in \tilde{G}} \sum_{j \in \tilde{G}, j \neq i} x_{ijk} \cdot \frac{d_{ij}}{100} \cdot \mu_{p(k)} \cdot \phi_{p(k)}(\bar{v}_{ij}^{\omega}) (1 + \alpha_{p(k)} \lambda_{ik}) c^{\text{car}}, \quad (12)$$

其中 $\mu_{p(k)}$ 为能耗向碳排放量的换算系数， c^{car} 为单位碳排放成本。

模型主要约束如下。每个配送子任务必须被且仅被一辆车服务：

$$\sum_{k \in K} \sum_{j \in \tilde{G}, j \neq i} x_{ijk} = 1, \quad i \in G, \quad (13)$$

启用车辆必须从配送中心出发并最终返回配送中心：

$$\sum_{j \in G} x_{0jk} = y_k, \quad \sum_{i \in G} x_{i0k} = y_k, \quad k \in K, \quad (14)$$

车辆在客户节点处满足流守恒：

$$\sum_{i \in \tilde{G}, i \neq h} x_{ihk} = \sum_{j \in \tilde{G}, j \neq h} x_{hjk}, \quad h \in G, k \in K, \quad (15)$$

为避免形成与配送中心无关的子回路，引入顺序变量 r_{ik} ，并施加 MTZ 约束：

$$r_{ik} - r_{jk} + nx_{ijk} \leq n - 1, \quad i, j \in G, i \neq j, k \in K. \quad (16)$$

车辆类型可用数量不能超过给定上限。设 K_a 为类型 a 对应的车辆集合，则

$$\sum_{k \in K_a} y_k \leq N_a, \quad a \in A. \quad (17)$$

每条车辆路径同时满足载重容量和容积容量约束：

$$\sum_{i \in G} q_i^{(w)} \sum_{j \in \tilde{G}, j \neq i} x_{ijk} \leq Q_k^{(w)} y_k, \quad k \in K, \quad (18)$$

$$\sum_{i \in G} q_i^{(v)} \sum_{j \in \tilde{G}, j \neq i} x_{ijk} \leq Q_k^{(v)} y_k, \quad k \in K, \quad (19)$$

发车时间不早于配送开始时刻，且车辆到达首个客户的时刻由发车时间和旅行时间共同决定：

$$u_k \geq 8, \quad k \in K. \quad (20)$$

$$t_j^\omega \geq u_k + \tau_{0j}^\omega(u_k) - M(1 - x_{0jk}), \quad j \in G, k \in K, \omega \in \Omega. \quad (21)$$

对任意被车辆 k 连续访问的客户节点 i 和 j ，时序约束为

$$t_j^\omega \geq t_i^\omega + s + \tau_{ij}^\omega(t_i^\omega + s) - M(1 - x_{ijk}), \quad i, j \in G, i \neq j, k \in K, \omega \in \Omega, \quad (22)$$

其中 s 为固定服务时间， M 为足够大的常数。

客户时间窗采用软约束处理，早到可等待，迟到需支付惩罚成本：

$$w_i^\omega \geq e_i - t_i^\omega, \quad l_i^\omega \geq t_i^\omega - l_i, \quad w_i^\omega \geq 0, l_i^\omega \geq 0, \quad i \in G, \omega \in \Omega. \quad (23)$$

最后，决策变量满足

$$x_{ijk} \in \{0, 1\}, \quad y_k \in \{0, 1\}, \quad w_i^\omega \geq 0, \quad l_i^\omega \geq 0. \quad (24)$$

上述模型将车辆启用、路径连通、双容量限制、时变旅行时间、软时间窗服务水平和低碳成本统一到同一优化框架中，其核心目标始终是最小化静态配送总成本。

4.2.3 求解策略

由于该模型同时包含离散路径决策、连续发车时刻选择、车型匹配以及时变随机速度情景，直接求取精确解的计算代价较高。本文采用分层启发式策略，在保持可行性的基础上持续降低期望总成本。

首先生成速度情景，并以情景平均总成本评价路径方案。对给定路径 r 和车型 a ，其最优发车时刻表示为

$$u_{ra}^* = \arg \min_{u \in \mathcal{U}_r} \frac{1}{|\Omega|} \sum_{\omega \in \Omega} C_{ra}^\omega(u), \quad (25)$$

其中 $\mathcal{U}_r$ 为候选发车时刻集合， $C_{ra}^\omega(u)$ 为情景 ω 下车型 a 按发车时刻 u 执行路径 r 的总成本。

其次构造容量导向的初始路径。在车辆剩余载重和容积范围内选择可装载客户，并综合考虑装载填充度、当前位置距离和时间窗紧迫程度，使初始方案具备较好的容量可行性与服务可行性。初始路径形成后，逐条确定可执行车型集合，并在车辆数量上限内选择期望总成本最低的车型；随后对发车时刻进行离散搜索，以降低等待成本和迟到成本。

最后执行迭代改进。对单条路径采用客户顺序重排、邻近插入和 2-opt 反转等操作，对不同路径尝试客户交换和客户迁移；仅当调整后满足双容量约束且情景平均总成本下降时接受新方案。对迟到成本较高、行驶距离较长或碎片化任务较多的路径，进一步进行局部重组和尾货重平衡，从而得到满足车辆数量、容量和时间窗约束的低成本静态配送方案。

4.3 模型求解

4.3.1 车辆使用方案

静态最优方案共启用车辆 121 辆，其中新能源车 11 辆、燃油车 110 辆。各车型使用情况见表 3。新能源 1、燃油车 1 和燃油车 2 均达到满负荷使用，说明在当前双容量约束下，中大容量车型更适合承担拆分后的配送任务。

表 3 问题一车辆使用统计结果

车辆类型	动力类型	可用数量	使用数量	剩余数量	使用率/%
新能源 1	新能源	10	10	0	100.00
新能源 2	新能源	15	1	14	6.67
燃油车 1	燃油	60	60	0	100.00
燃油车 2	燃油	50	50	0	100.00
燃油车 3	燃油	50	0	50	0.00

88 个客户经需求拆分后形成 203 个访问任务点，车辆投放主要集中于新能源 1、燃油车 1 和燃油车 2，如图 3 所示。

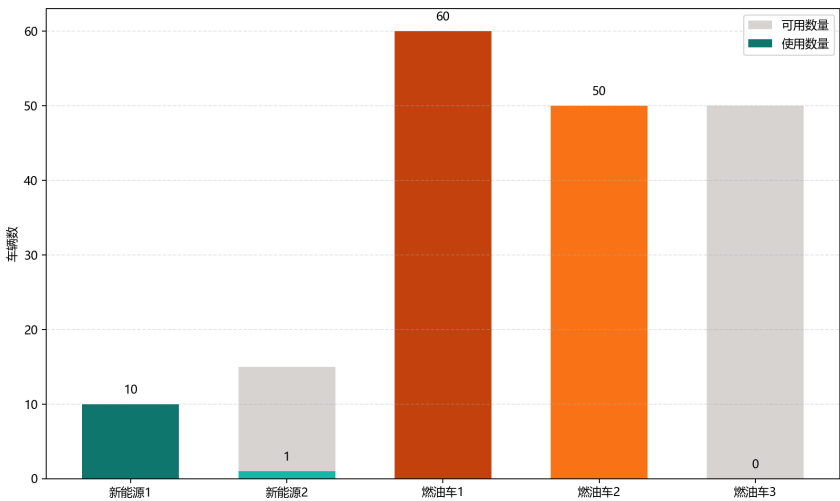


图 3 问题一车辆使用方案图

4.3.2 路径与到达时间结果

最优方案总行驶距离为 15363.84 km，总等待时长为 10.5453 h，总迟到时长为 32.3617 h。路径结果以“单客户高装载率直达配送”为主要特征，部分代表性路线见表 4。

表 4 问题一路径与到达时间代表性结果

路线编号	车辆类型	发车时间	路径	平均到达	等待/h	迟到/h	总成本/元
R001	燃油车 1	10:00	0 → 60 → 0	10:32	0	0	505.88
R010	燃油车 1	10:30	0 → 6 → 0	12:11	0.0183	0	636.75
R012	燃油车 1	11:20	0 → 61 → 0	13:40	0	0.0041	728.42
R014	新能源 1	09:30	0 → 59 → 0	11:51	0	0	528.44

由表 4可见，多数路线能在时间窗内完成服务，仅少量节点出现轻微等待或迟到，说明静态方案具有较好的服务可行性。

全局路径如图 4 所示，路线整体呈由配送中心向外辐射的空间结构，远距离客户多采用直达服务。

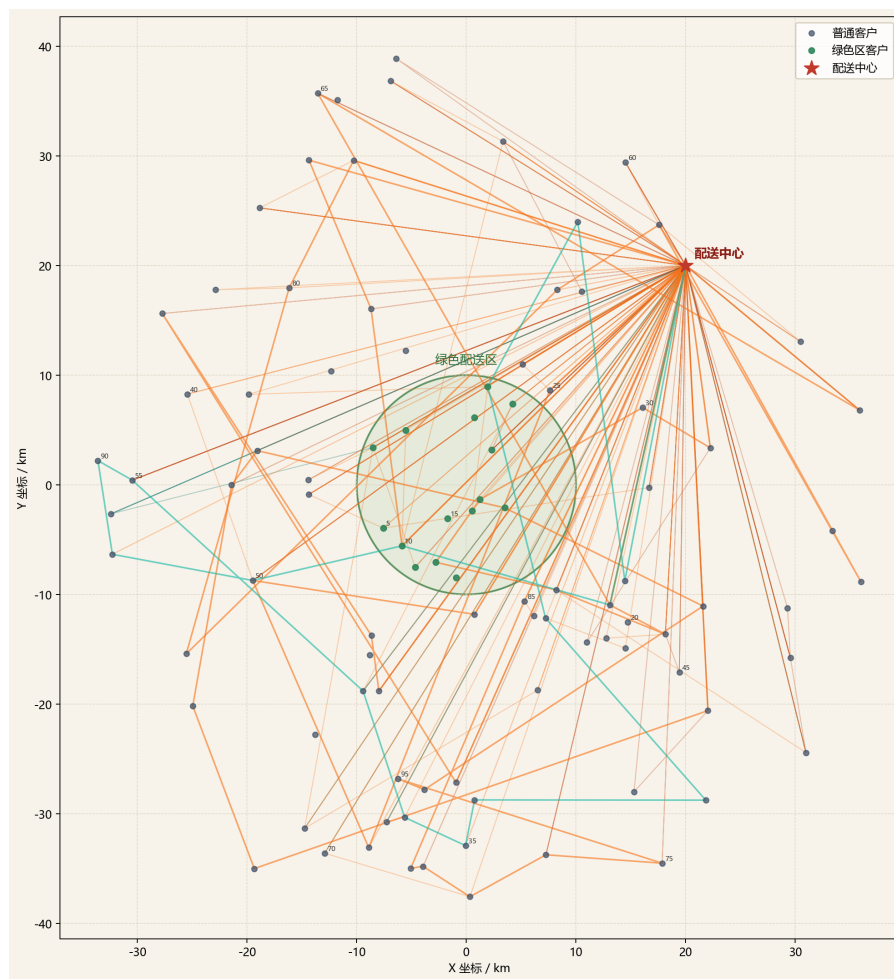


图 4 问题一路径规划结果图

4.3.3 成本构成分析

问题一最优方案总成本为 88982.82 元，其中启动成本、能源成本、碳排放成本和时间窗成本分别为 48400.00 元、31852.15 元、6901.69 元和 1828.99 元，超时返场成本为 0。成本构成见表 5。

表 5 问题一成本构成结果

成本项目	成本值/元	占总成本比例/%
启动成本	48400.00	54.39
能源成本	31852.15	35.79
碳排放成本	6901.69	7.76
时间窗成本	1828.99	2.06

由表 5 可知，启动成本占比最高，为总成本主导项；能源成本次之；时间窗成本占比较低，说明方案整体服务偏差较小。问题一的主要成本压力来自需求拆分后的车辆启用数量增加。

五、问题二模型的建立与求解

5.1 问题分析

问题二在问题一静态调度模型基础上引入绿色配送区限行政策，即燃油车在 8:00–16:00 时段内不得进入以市中心为圆心、半径为 10 km 的绿色配送区。绿色车辆路径问题强调运输成本、能源消耗和环境影响的协同优化 [6]，因此调度方案不仅要满足容量约束和软时间窗约束，还需同时保证政策合规、车型适配、发车时刻合理与碳排放可控。

绿色配送区客户数量有限，但其空间位置更接近核心城区，服务时段也更易与禁行时段重叠。若沿用问题一方案，部分燃油车路线将在进入绿色区时失去可行性，并进一步引起发车时刻后移、迟到惩罚增加和车型结构重构。因此，问题二的核心是在限行零违规前提下，重新平衡总成本、时间窗损失与低碳收益。

基于此，本文采用“分区建路 + 车型重分配 + 局部优化”的求解思路。先区分绿色区任务与外围任务，对绿色区任务优先配置新能源车辆；再结合剩余车池重组外围路径；最后通过全局车型匹配和路径顺序调整，降低政策实施后的附加成本。

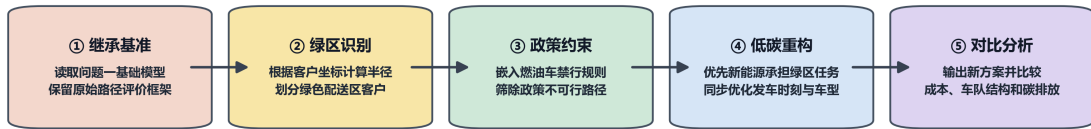


图 5 问题二求解流程图

5.2 模型建立

5.2.1 绿色配送区识别

设客户 i 的坐标为 (x_i, y_i) ，则其到市中心的径向距离为

$$r_i = \sqrt{x_i^2 + y_i^2}. \quad (26)$$

定义绿色配送区指示变量

$$g_i = \begin{cases} 1, & r_i \leq 10, \\ 0, & r_i > 10. \end{cases} \quad (27)$$

当 $g_i = 1$ 时, 客户 i 属于绿色配送区; 当 $g_i = 0$ 时, 客户 i 属于非绿色配送区。由预处理结果可知, 本题共有 15 个客户位于绿色配送区。

5.2.2 限行约束下的路径优化模型

在问题一模型基础上, 引入车辆动力类型指示变量

$$\eta_k = \begin{cases} 1, & \text{车辆 } k \text{ 为燃油车,} \\ 0, & \text{车辆 } k \text{ 为新能源车.} \end{cases} \quad (28)$$

若车辆 k 服务绿色配送区客户 i , 则其到达时刻必须满足政策可行性条件

$$\eta_k g_i = 1 \implies t_i^\omega \notin [8, 16), \quad i \in G, k \in K, \omega \in \Omega. \quad (29)$$

即燃油车仅允许在 8:00 之前或 16:00 之后进入绿色配送区, 而新能源车不受该限制。这样, 问题二的可行域可表示为

$$\mathcal{F}_2 = \{(x, y, t) \in \mathcal{F}_1 \mid \eta_k g_i = 1 \Rightarrow t_i^\omega \notin [8, 16), \forall i, k, \omega\}, \quad (30)$$

其中 $\mathcal{F}_1$ 为问题一在容量约束、流平衡约束、时序约束与软时间窗约束下的可行域。对于给定路线 r 和车型 a , 进一步定义政策可行车型集合

$$\mathcal{A}_r = \{a \in A \mid q_r^{(w)} \leq Q_a^{(w)}, q_r^{(v)} \leq Q_a^{(v)}, \eta_a g_i = 1 \Rightarrow t_i^\omega \notin [8, 16), \forall i \in r, \omega \in \Omega\}. \quad (31)$$

当 $\mathcal{A}_r$ 为空时, 说明路线 r 在当前访问顺序下无法由任一车型合规执行, 需要重新调整发车时刻、访问顺序或任务分组。

考虑到限行政策实施后还需兼顾低碳目标, 本文在问题一总成本目标基础上, 对碳排放与绿色区新能源优先服务进行联合评价。设路线 r 由车型 a 执行时的综合评价值为

$$c_{ra} = C_{ra}^{\text{total}} + \lambda C_{ra}^{\text{car}} - \beta \delta_r (1 - \eta_a), \quad (32)$$

其中 C_{ra}^{total} 为路线总成本, C_{ra}^{car} 为碳排放成本, δ_r 为绿色区路线指示变量, η_a 为车型 a 的动力类型指示变量; λ 为碳排放偏好系数, β 为绿色区新能源服务奖励系数。当路线 r 涉及绿色区客户时, 若由新能源车型执行, 则评价价值适度降低, 以体现政策导向。这样, 问题二的车型分配模型可写为

$$\min \sum_{r \in R} \sum_{a \in A} c_{ra} u_{ra}, \quad (33)$$

满足

$$\sum_{a \in A} u_{ra} = 1, \quad r \in R, \quad (34)$$

$$\sum_{r \in R} u_{ra} \leq N_a, \quad a \in A, \quad (35)$$

$$u_{ra} = 0, \quad a \notin \mathcal{A}_r, \quad (36)$$

其中 u_{ra} 为 0-1 决策变量, N_a 表示车型 a 的可用数量。该模型本质上是在政策可行前提下, 对所有候选路线进行全局车型重分配; 通过调整 λ , 可在总成本和碳排放表现之间进行权衡, 并优先选择满足碳排放控制要求的低成本方案。

5.2.3 求解思路

针对限行约束导致的车型可行性变化, 本文采用分阶段求解策略: 先按绿色配送区属性划分任务, 对绿色区任务优先使用新能源车建路; 再对外围任务和剩余车池进行路径重组; 最后构建“路线—车型”二部图最小费用流模型完成全局车型重分配, 并结合碳排放偏好系数搜索确定最终方案。

5.3 模型求解

5.3.1 车辆使用方案

问题二共启用车辆 124 辆, 其中新能源车 25 辆、燃油车 99 辆。相较问题一, 新能源车增加 14 辆, 燃油车减少 11 辆, 车辆结构明显向新能源车型转移, 见表 6。

表 6 问题二车辆使用统计结果

车辆类型	动力类型	可用数量	使用数量	剩余数量	使用率/%
新能源 1	新能源	10	10	0	100.00
新能源 2	新能源	15	15	0	100.00
燃油车 1	燃油	60	60	0	100.00
燃油车 2	燃油	50	39	11	78.00
燃油车 3	燃油	50	0	50	0.00

涉及绿色配送区的路线共 15 条, 其中新能源车承担 11 条, 燃油车承担 4 条, 限行违规次数为 0。车辆结构如图所示, 新能源 2 由低使用率车型转变为绿色区任务的重要补充运力。

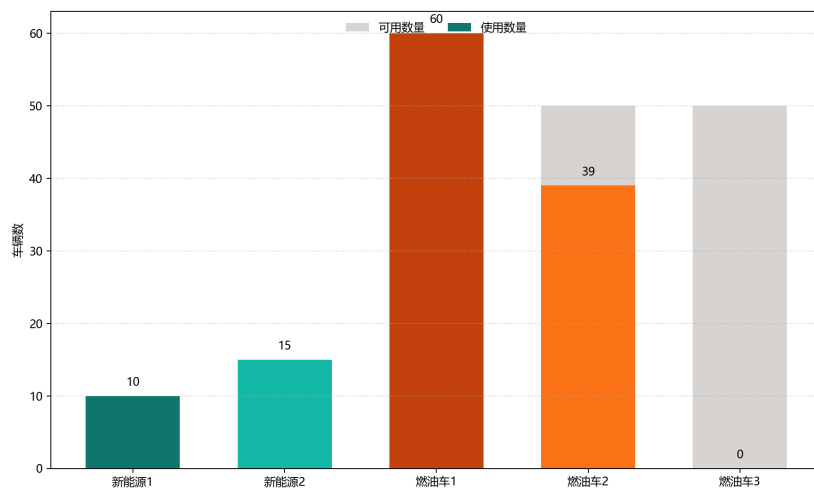


图 6 问题二车辆结构图

5.3.2 路径与到达时间结果

问题二方案总行驶距离为 15557.38 km。受禁行时段影响，部分路线需调整发车时刻和访问顺序，代表性结果见表 7。

表 7 问题二代表性路径与到达时间结果

路线编号	车辆类型	发车时间	访问客户	平均到达	等待/h	迟到/h	总成本/元
R001	燃油车 1	16:00	8	16:56	0	5.1437	847.23
R004	新能源车 1	10:30	6	12:11	0.0183	0	464.11
R010	燃油车 1	15:10	12,4	16:08, 16:54	2.5842	0.3058	707.61
R015	新能源车 2	10:50	6,7,3,2,5,4	13:01-20:05	3.4498	0.0807	601.52

由表 7可见，新能源车可在白天直接服务绿色区客户，燃油车则通过延后发车避开禁行区间。全局路径见图，整体呈现空间分区与时间错峰相结合的分时配送特征。

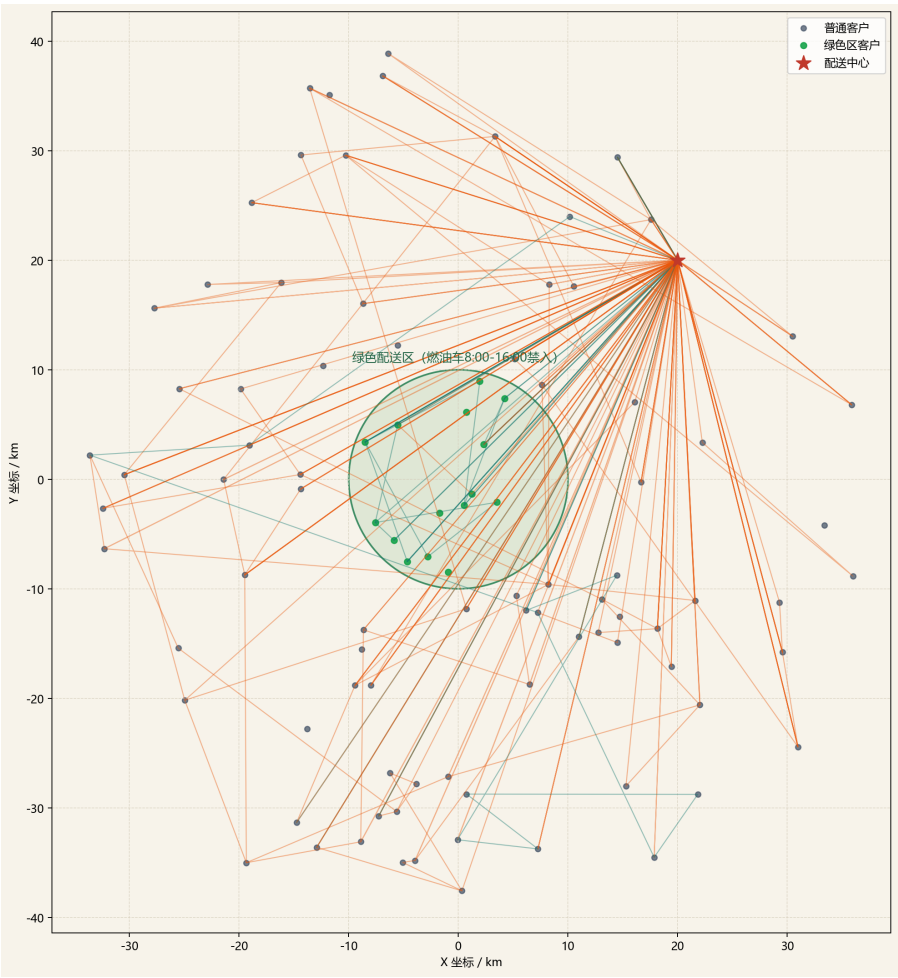


图 7 问题二车辆路径图

5.3.3 政策影响分析

问题二总成本为 91121.92 元，其中启动成本、能源成本、碳排放成本和时间窗成本分别为 49600.00 元、31196.10 元、6751.86 元和 3573.96 元。与问题一相比，总成本增加 2139.10 元，但碳排放成本降低 149.83 元，碳排放总量减少 230.50 kg。关键指标对比见表 8。

表 8 问题一与问题二关键指标对比

指标	问题一	问题二	变化
使用车辆数	121	124	+3
总行驶距离/km	15363.84	15557.38	+193.54
启动成本/元	48400.00	49600.00	+1200.00
能源成本/元	31852.15	31196.10	-656.05
碳排放成本/元	6901.69	6751.86	-149.83
碳排放总量/kg	10617.98	10387.48	-230.50
时间窗成本/元	1828.99	3573.96	+1744.97
总成本/元	88982.82	91121.92	+2139.10
新能源车使用数	11	25	+14
燃油车使用数	110	99	-11

由表 8 可知，限行政策对调度结果的影响主要体现在总成本、车辆使用结构和碳排放三个方面。首先，总成本由 88982.82 元增至 91121.92 元，增加 2139.10 元，主要来源于燃油车避开禁行时段后带来的时间窗成本上升。其次，车辆使用结构明显低碳化，新能源车使用数由 11 辆增至 25 辆，燃油车使用数由 110 辆降至 99 辆，说明绿色配送区任务显著提高了新能源车辆的边际价值。最后，虽然总行驶距离增加 193.54 km，但碳排放总量由 10617.98 kg 降至 10387.48 kg，减少 230.50 kg，碳排放成本同步降低 149.83 元，表明新能源替代在一定程度上抵消了限行政策带来的额外运行代价。

图 8 展示了关键变化幅度和车辆结构差异，进一步说明政策约束促使调度方案由成本单优先转向成本、时效与低碳协同权衡。

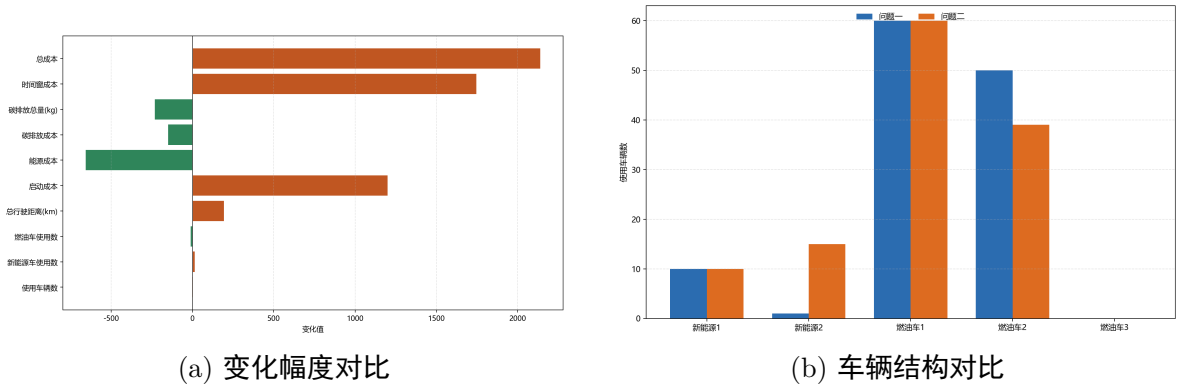


图 8 问题一与问题二变化幅度及车辆结构对比

问题二内部成本构成见表 9。

表 9 问题二成本构成结果

成本项目	成本值/元	占总成本比例/%
启动成本	49600.00	54.43
能源成本	31196.10	34.23
碳排放成本	6751.86	7.41
时间窗成本	3573.96	3.92

由表可知，启动成本仍占主导地位，能源成本次之；时间窗成本占比虽低，但相较问题一增幅最明显。图 9 给出了成本构成和关键指标对比。总体来看，问题二在总成本可控增加的条件下，实现了限行零违规、新能源替代增强和碳排放总量下降。

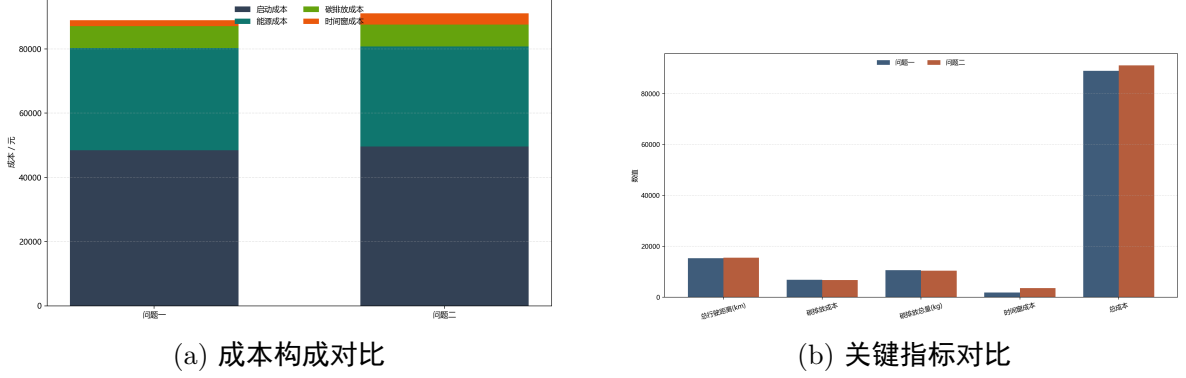


图 9 问题一与问题二成本构成及关键指标对比

六、问题三模型的建立与求解

6.1 问题分析

问题三在问题二限行政策调度方案基础上进一步考虑突发事件影响。与前两问一次性生成全天方案不同，动态场景中车辆状态、未完成任务、客户位置与时间窗会随事件触发而变化，原有静态路径可能局部失效。动态车辆路径问题通常通过事件触发、信息更新和滚动优化修复方案 [7]，因此需建立事件驱动的滚动重调度模型。

该问题的核心特征是“已执行部分固定、未执行部分可调”。当事件在时刻 τ 发生时，已完成或正在执行的车辆路径不再修改，以保持执行过程和调度稳定性。因此，动态优化以问题二方案为基线，仅对触发时刻后尚未发车的剩余任务、备用车辆和可释放车辆进行局部重构。

突发事件包括订单取消、新增订单、配送地址变更、时间窗调整 and 车辆故障五类，分别改变需求规模、客户位置、服务时效和车辆可用性。问题三的关键是在实时更新任务集合与可用车池的基础上，保持限行合规，优先控制迟到风险，并兼顾总成本与碳排放表现。本文据此采用“状态识别—事件更新—滚动重调度—结果回填”的闭环建模思路。

6.2 模型建立

6.2.1 动态事件类型刻画

结合城市配送运行中常见的不确定因素，本文构造五类基础动态事件：

1. 新增订单：触发后产生临时配送需求，使剩余任务集合增加；
2. 订单取消：未执行客户需求被撤销，使剩余任务集合缩减；
3. 配送地址变更：客户坐标发生变化，使相关距离和到达时刻重新计算；
4. 客户时间窗调整：服务允许区间收缩或平移，使时间可行域发生变化；
5. 车辆故障：部分车辆临时停运，使可用车池缩减并推迟车辆释放时刻。

设事件集合为 $\mathcal{E} = \{e_1, e_2, \dots, e_m\}$ ，事件 e 的触发时刻为 τ_e 。其对配送系统的影响可归纳为

对任务、时间窗、位置和车辆状态四类要素的扰动：

$$(\mathcal{U}, \mathcal{W}, \mathcal{L}, \mathcal{K}), \quad (37)$$

其中 $\mathcal{U}$ 表示未完成任务集合， $\mathcal{W}$ 表示客户时间窗参数集合， $\mathcal{L}$ 表示客户空间位置集合， $\mathcal{K}$ 表示车辆可用状态集合。事件触发后，系统状态由

$$(\mathcal{U}, \mathcal{W}, \mathcal{L}, \mathcal{K}) \longrightarrow (\mathcal{U}', \mathcal{W}', \mathcal{L}', \mathcal{K}'), \quad (38)$$

并据此确定新的重调度任务集和可行车辆集合。

为进一步刻画不同事件对状态变量的作用，设新增订单集合为 $\mathcal{U}^+$ ，取消订单集合为 $\mathcal{U}^-$ ，地址变更客户集合为 $\mathcal{A}^{\text{loc}}$ ，时间窗调整客户集合为 $\mathcal{A}^{\text{tw}}$ ，故障车辆集合为 $\mathcal{K}^{\text{fault}}$ 。则事件触发后的任务集合更新可写为

$$\mathcal{U}' = (\mathcal{U} \setminus \mathcal{U}^-) \cup \mathcal{U}^+. \quad (39)$$

对地址变更客户 $i \in \mathcal{A}^{\text{loc}}$ ，其坐标和距离矩阵同步更新为

$$(x'_i, y'_i) = (x_i + \Delta x_i, y_i + \Delta y_i), \quad d'_{ij} = \sqrt{(x'_i - x'_j)^2 + (y'_i - y'_j)^2}. \quad (40)$$

对时间窗调整客户 $i \in \mathcal{A}^{\text{tw}}$ ，服务时间窗更新为

$$[e'_i, l'_i] = [e_i + \Delta e_i, l_i + \Delta l_i]. \quad (41)$$

对故障车辆 $k \in \mathcal{K}^{\text{fault}}$ ，其可用状态更新为不可用直至新的释放时刻 $t_k^{\text{avail}}(\tau)$ 。

6.2.2 滚动重调度框架

设事件在时刻 τ 触发。以问题二调度方案为基线，车辆状态可划分为已完成、执行中、未发车和备用四类。记触发时刻前已确定且不再调整的路径集合为 $\mathcal{F}(\tau)$ ，触发后可重调度任务集合为 $\mathcal{U}(\tau)$ ，可参与后续服务的车辆集合为 $\mathcal{K}(\tau)$ 。动态优化仅作用于 $\mathcal{U}(\tau)$ 与 $\mathcal{K}(\tau)$ ，而 $\mathcal{F}(\tau)$ 保持固定。

定义动态路径决策变量

$$z_{ijk}^\tau = \begin{cases} 1, & \text{触发时刻后车辆 } k \text{ 从节点 } i \text{ 行驶至节点 } j, \\ 0, & \text{否则,} \end{cases} \quad (42)$$

以及车辆参与变量 y_k^τ 。触发后的每个剩余任务必须被且仅被服务一次：

$$\sum_{k \in \mathcal{K}(\tau)} \sum_{j \in \mathcal{U}(\tau) \cup \{0\}, j \neq i} z_{ijk}^\tau = 1, \quad i \in \mathcal{U}(\tau). \quad (43)$$

车辆在重调度子问题中仍需满足载重和容积约束：

$$\sum_{i \in \mathcal{U}(\tau)} q_i^{(w)} \sum_j z_{ijk}^\tau \leq Q_k^{(w)} y_k^\tau, \quad \sum_{i \in \mathcal{U}(\tau)} q_i^{(v)} \sum_j z_{ijk}^\tau \leq Q_k^{(v)} y_k^\tau. \quad (44)$$

若车辆 k 在触发时刻后的可释放时刻为 $a_k(\tau)$ ，则其动态发车时刻 u_k^τ 满足

$$u_k^\tau \geq a_k(\tau), \quad k \in \mathcal{K}(\tau). \quad (45)$$

为体现“合规优先、服务优先、成本次优”的动态决策原则，本文采用词典序评价向量而非单一加权和目标。对任一候选动态方案 s ，定义

$$\Phi(s) = (V(s), I(s), L(s), C(s), E(s)), \quad (46)$$

其中 $V(s)$ 为限行违规次数， $I(s)$ 为是否发生迟到的指示变量， $L(s)$ 为总迟到时长， $C(s)$ 为总成本， $E(s)$ 为碳排放成本。动态重调度目标为

$$\min_{\text{lex}} \Phi(s), \quad s \in \mathcal{S}(\tau), \quad (47)$$

其中 $\mathcal{S}(\tau)$ 为触发时刻下所有可行动态方案的集合， $\min_{\text{lex}}$ 表示按词典序最小化。该目标首先保证限行零违规，其次尽量避免迟到，在服务可达前提下再压缩总成本和碳排放成本。

动态方案的总成本由固定部分和重调度部分组成：

$$C^{\text{dyn}}(\tau) = C^{\text{fix}}(\tau) + C^{\text{res}}(\tau), \quad (48)$$

其中 $C^{\text{fix}}(\tau)$ 为触发前已执行或正在执行路径的既定成本， $C^{\text{res}}(\tau)$ 为触发后重调度部分的增量成本。总迟到时长和碳排放成本也采用同样的固定项与重调度项分解，以保证已发生运输行为不被重复修正。

$$L^{\text{dyn}}(\tau) = L^{\text{fix}}(\tau) + L^{\text{res}}(\tau), \quad E^{\text{dyn}}(\tau) = E^{\text{fix}}(\tau) + E^{\text{res}}(\tau). \quad (49)$$

重调度部分仍需满足载重、容积、时序、软时间窗和绿色配送区限行约束。若车辆 k 发生故障，还需满足车辆释放时刻约束

$$t_k^{\text{avail}}(\tau) \geq \tau + \Delta_k, \quad (50)$$

其中 Δ_k 为故障停运时长。仅当车辆达到可释放时刻后，才允许进入重调度车池参与后续任务分配。此外，对绿色配送区客户仍施加限行约束：

$$\eta_k g_i = 1 \implies t_i^{\omega, \tau} \notin [8, 16], \quad i \in \mathcal{U}(\tau), k \in \mathcal{K}(\tau), \omega \in \Omega. \quad (51)$$

6.2.3 实时响应机制

为实现事件触发后的快速调整，本文构建如下实时响应机制。

首先，以问题二基线方案为初始执行计划，并按触发时刻 τ 判定车辆状态。已完成车辆不再参与当前任务；执行中车辆保留既有路径并记录当前位置、剩余任务和可释放时刻；未发车车辆与备用车辆共同构成候选运力池。

其次，按事件类型更新系统状态。新增订单对应任务集合扩张，订单取消对应任务集合缩减；地址变更更新客户坐标并修正距离关系，时间窗调整更新服务允许区间；车辆故障则推迟车辆可释放时刻，并重新评估可用运力。

随后，由触发后尚未执行的剩余客户需求构成待重调度任务集，并与可用车辆集合共同形成动态优化子问题。求解时先进行容量与政策可行的初始路径分配，再通过词典序指标筛选候选方案；若局部路线迟到过大，则进一步采用路径拆分、车辆重分配和顺序调整等方式进行修复。

最后，将重调度结果与触发前固定部分合并，形成完整执行方案，并从总成本、迟到时长、碳排放成本和限行违规次数等角度评价动态响应效果。由此形成“状态识别—事件更新—局部重构—结果回填”的动态调度闭环。

6.3 模型求解

6.3.1 动态调度结果

五类突发事件均采用滚动重调度方式处理，仅调整触发时刻后尚未发车的任务。表 10 显示，各场景限行违规次数均为 0，动态路线数介于 51–84 条，动态总成本介于 33812.35–58985.78 元。

表 10 问题三动态调度场景结果

场景编号	事件类型	动态路线数	动态总成本/元	动态总迟到/h	动态碳成本/元
S1	订单取消	84	58985.78	7.5791	4443.81
S2	新增订单	76	52857.07	6.6931	3926.82
S3	配送地址变更	62	42313.79	12.8025	2997.54
S4	时间窗调整	57	39165.01	10.6764	2814.27
S5	车辆故障	51	33812.35	8.2261	2308.99

从服务表现看，S2 的动态总迟到最低，为 6.6931 h；S3 的动态总迟到最高，为 12.8025 h；S5 因触发较晚、剩余任务较少，动态总成本最低。

图 10 展示了路径响应结果。动态方案以局部重构为主，新增订单主要表现为邻近插入，地址变更和时间窗调整引发绕行或发车后移，车辆故障则通过备用车辆或后续释放车辆接续任务。

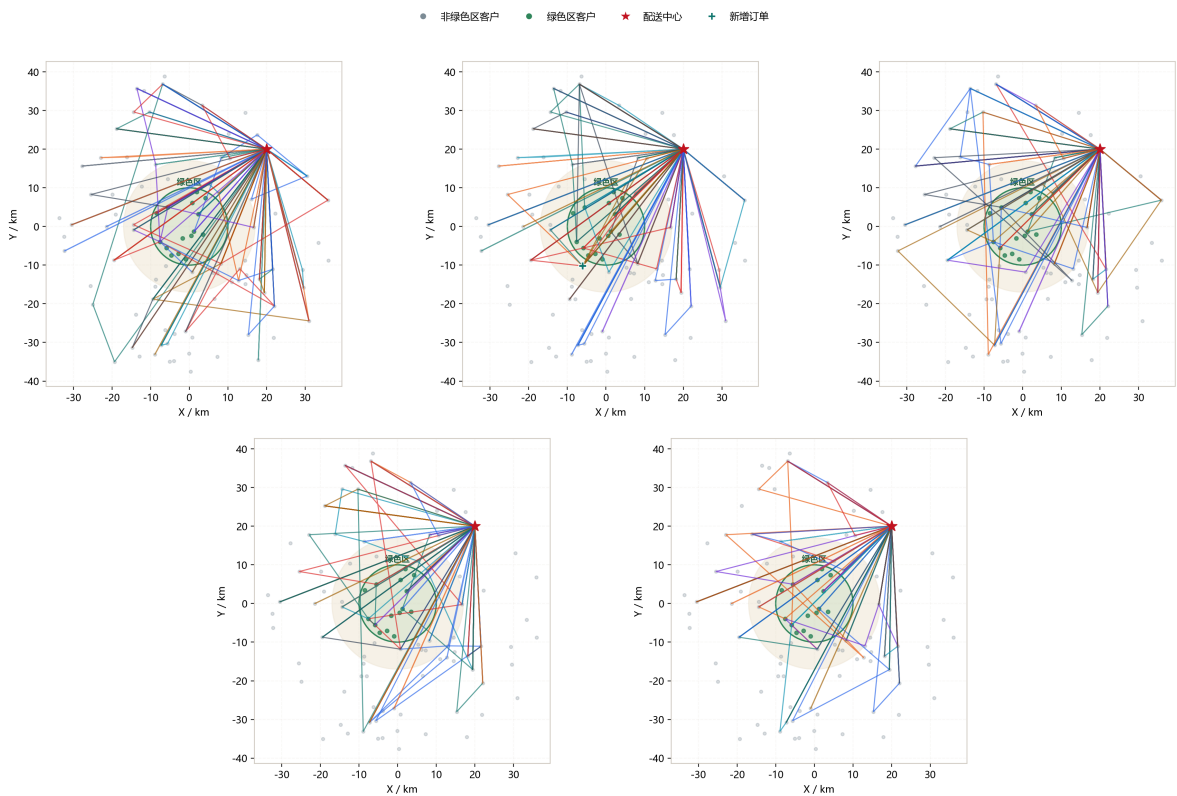


图 10 问题三事件响应路径图

图 11 给出了 S3 和 S5 两个典型场景的车辆响应甘特图。S3 体现空间扰动下的访问顺序重排，S5 体现故障车辆任务由备用或后续可用车辆接替。

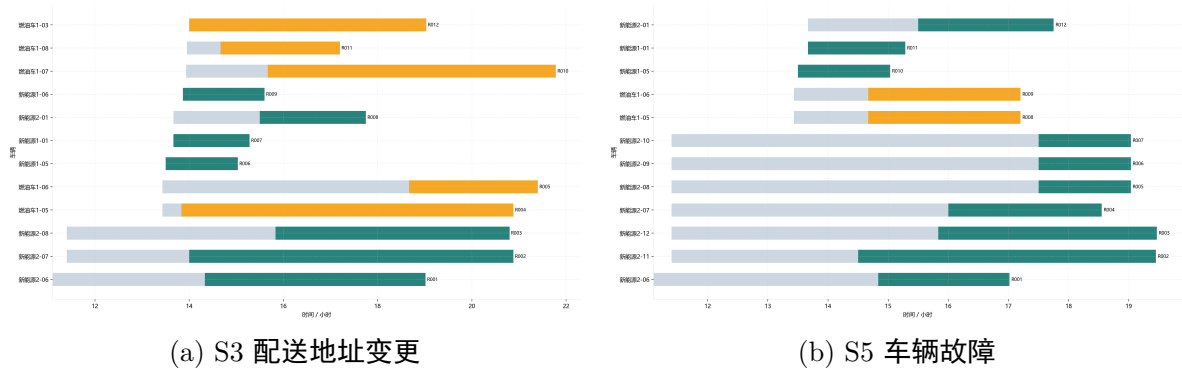


图 11 典型事件下的车辆响应甘特图

6.3.2 多情景对比分析

为评价动态调度效果，本文将各场景与局部基准方案及问题二全天基线方案进行对比。五个场景的平均动态总成本为 45426.80 元，平均动态总迟到为 9.1954 h；回填触发前固定部分后，平均全天总成本为 90114.00 元，低于问题二基准水平。结果见表 11。

表 11 问题三多场景关键结果汇总

场景编号	场景名称	全天总成本	成本变化	全天碳成本	碳成本变化
S1	订单取消	90369.08	-752.84	6679.30	-72.56
S2	新增订单	93334.70	+2212.78	6861.69	+109.83
S3	配送地址变更	88428.75	-2693.17	6439.38	-312.48
S4	时间窗调整	89273.26	-1848.66	6610.74	-141.12
S5	车辆故障	89164.20	-1957.72	6470.50	-281.36

由表 11可见，除新增订单场景 S2 外，其余四类场景的全天总成本均低于问题二基准方案，其中 S3 改善幅度最大。图 12 展示了各场景相对问题二基准方案的综合变化。

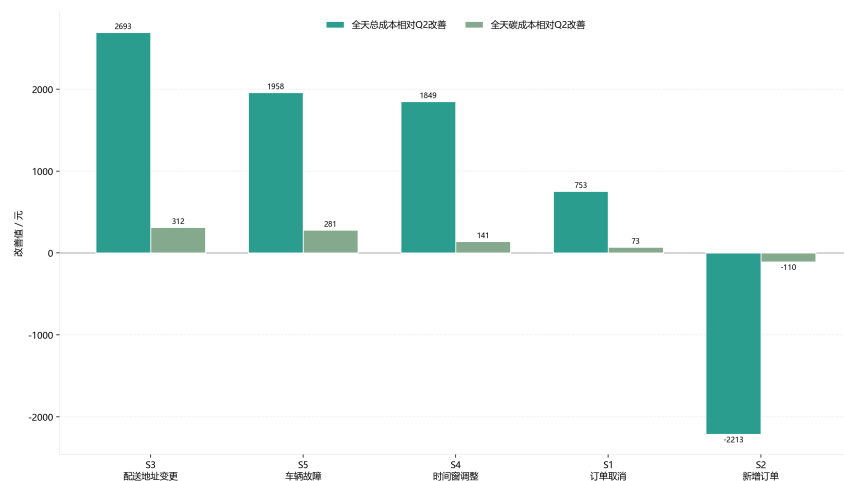
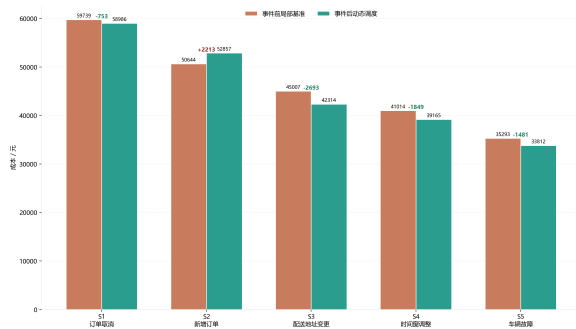
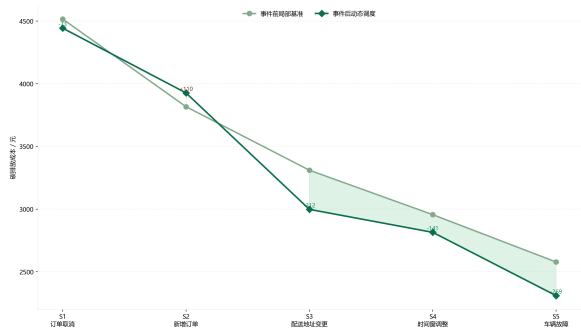


图 12 问题三综合改善瀑布图

事件前后成本对比与低碳响应结果如图 13 所示。动态调度通过局部重构控制事件后的成本增量；除 S2 因新增服务需求导致成本上升外，多数场景在全天回填后仍保持较好的成本和碳排放表现。



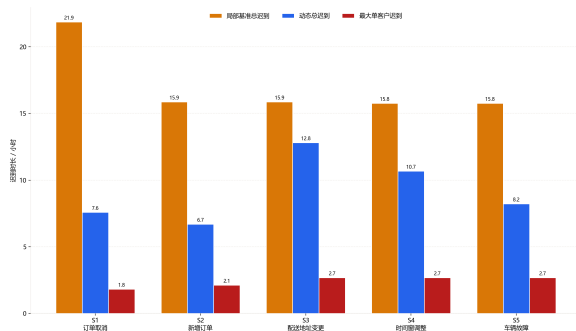
(a) 事件前后成本对比



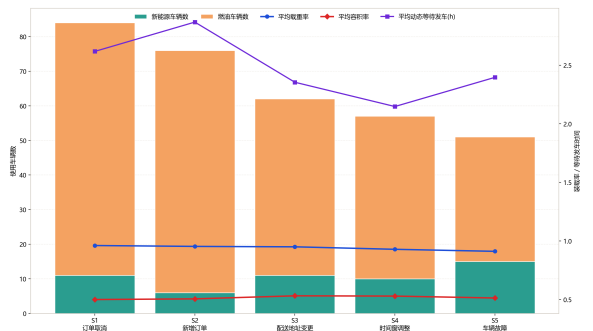
(b) 低碳响应对比

图 13 动态调度成本与低碳响应对比

服务水平与车辆结构见图 14。S2 迟到最小，S3 与 S4 迟到较高，说明地址变更和时间窗调整对时空匹配影响更明显。车辆结构会随事件类型自适应变化，各场景平均载重率均保持在 0.91 以上。



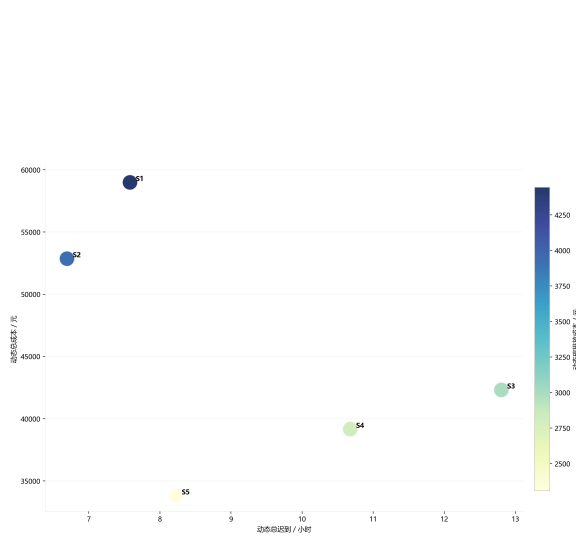
(a) 服务水平变化



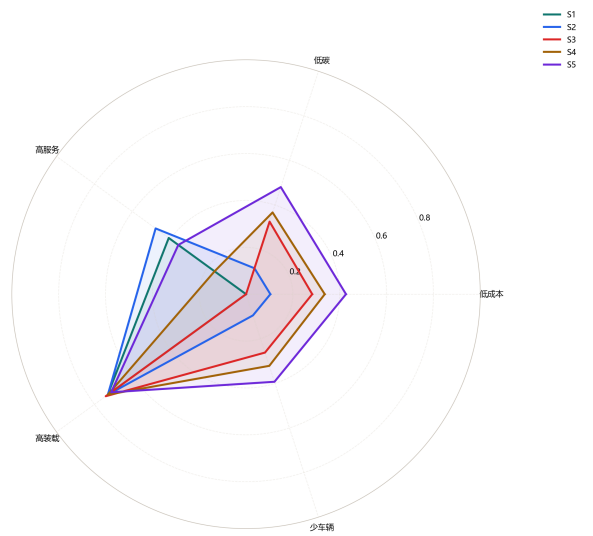
(b) 车辆结构与装载

图 14 动态调度服务水平与车辆结构对比

图 15 从迟到、成本和综合韧性角度刻画了五类场景差异：S2 服务表现较优，S3 对路径修复和时效保障要求最高。



(a) 迟到-成本气泡



(b) 情景韧性雷达

图 15 动态调度综合性能对比

综合以上结果，突发事件下的调度策略可概括如下。

1. 对订单取消事件，应及时释放对应客户需求、车辆容量和后续可用时间，并优先合并空间位置相近的未发车任务。该策略能够减少冗余发车，降低空驶和重复绕行带来的成本损失。
2. 对新增订单事件，应优先检查触发时刻后尚未发车且路径邻近的车辆，将新增客户插入边际距离和时间窗损失较小的路线；若插入后造成明显迟到，则调用备用车辆独立承运。
3. 对配送地址变更事件，应重新计算受影响客户与周边节点的距离关系，并优先修复原路线中的访问顺序。若地址变更导致单条路线迟到显著增加，则可通过路线拆分或客户迁移降低连锁延误。
4. 对时间窗调整事件，应将时间窗收紧或提前的客户前置服务，并同步调整相关车辆发车时刻。对无法通过顺序调整满足的任务，可采用局部换车或单独派车方式控制迟到惩罚。
5. 对车辆故障事件，应立即冻结故障车辆未完成任务，并在备用车辆与后续可释放车辆之间重新分配。若故障任务涉及绿色配送区，应优先由新能源车接替，以同时保证限行合规和低碳目标。

上述策略体现了动态调度的差异化响应思想：需求类扰动侧重任务增删与邻近插入，时空类扰动侧重路径顺序修复，运力类扰动侧重车辆接管与任务重分配。由此，问题三模型能够在限行合规前提下完成局部路径修复，并兼顾响应速度、服务水平与低碳目标。

七、敏感性与鲁棒性分析

7.1 敏感性分析

为检验关键成本参数变化对目标函数评价值的影响，本文以问题二限行政策调度方案为基准，对碳排放成本系数和迟到惩罚系数分别进行 $\pm 10\%$ 与 $\pm 20\%$ 扰动分析。该检验保持车辆路径、车型分配和发车时刻不变，仅重新计算对应成本分量，因此所得结论主要反映既定调度方案在参数扰动下的评价稳定性。基准方案总成本为 91121.92 元，其中碳排放成本为 6751.86 元，时间窗成本为 3573.96 元。

表 12 关键参数敏感性分析结果

扰动参数	扰动幅度	总成本/元	变化值/元	变化率/%
碳排放成本系数	-20%	89771.55	-1350.37	-1.48
碳排放成本系数	-10%	90446.73	-675.19	-0.74
碳排放成本系数	+10%	91797.11	675.19	0.74
碳排放成本系数	+20%	92472.29	1350.37	1.48
迟到惩罚系数	-20%	90407.13	-714.79	-0.78
迟到惩罚系数	-10%	90764.52	-357.40	-0.39
迟到惩罚系数	+10%	91479.32	357.40	0.39
迟到惩罚系数	+20%	91836.71	714.79	0.78

由表 12 和图 16 可见，在 $\pm 20\%$ 的参数扰动范围内，总成本变化率均控制在 1.5% 以内。其中，碳排放成本系数对总成本的影响相对更明显， $\pm 20\%$ 扰动对应总成本变化率约为 $\pm 1.48\%$ ；迟

到惩罚系数对应变化率约为 $\pm 0.78\%$ 。这表明在当前基准方案下，碳排放成本和时间窗成本会改变目标函数取值，但尚不足以主导总成本波动，既定调度方案的成本评价具有一定稳定性。

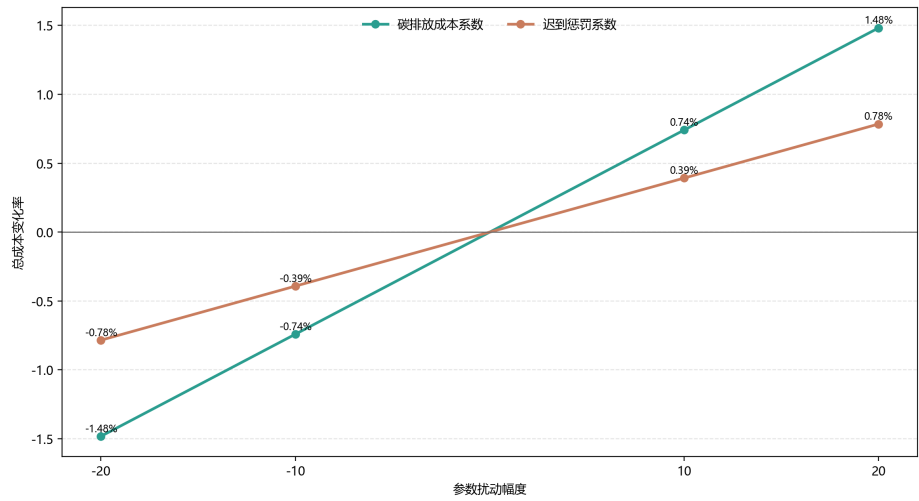


图 16 关键参数敏感性分析图

7.2 鲁棒性分析

鲁棒性分析主要考察模型在突发事件扰动下保持可行性和控制成本偏离的能力。本文以问题二方案作为基准，选取订单取消、新增订单、配送地址变更、时间窗调整和车辆故障五类事件进行检验，并从限行可行性、动态迟到、全天回填成本和碳成本变化等方面评价模型表现。表中“改善”以问题二基准方案为参照，正值表示成本下降，负值表示成本上升。

表 13 多情景鲁棒性分析结果

场景	事件类型	限行违规	动态迟到/h	总成本改善/元	碳成本改善/元
S1	订单取消	0	7.58	752.84	72.56
S2	新增订单	0	6.69	-2212.78	-109.83
S3	配送地址变更	0	12.80	2693.17	312.48
S4	时间窗调整	0	10.68	1848.66	141.12
S5	车辆故障	0	8.23	1957.72	281.36

由表 13 可知，五类突发事件下限行违规次数均为 0，说明绿色配送区约束在动态重调度中能够保持可行。从成本表现看，除新增订单场景 S2 因服务需求增加导致全天总成本上升 2212.78 元外，其余四类场景均实现全天总成本改善；其中 S3 的总成本改善最大，为 2693.17 元。碳成本方面，除 S2 外其余场景均低于问题二基准水平，表明动态重调度并未破坏低碳调度效果。图 17 进一步显示，S3、S4 和 S5 虽然存在较高动态迟到，但通过剩余任务重构仍能控制全天成本和碳排放成本，说明模型在空间扰动、时间窗扰动和车辆可用性扰动下具有一定恢复能力。

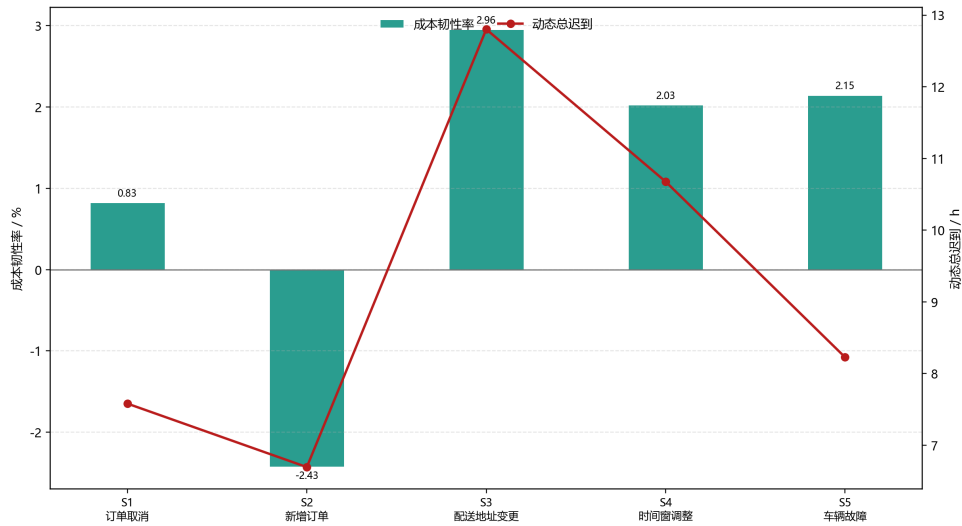


图 17 多情景鲁棒性分析图

综合来看，敏感性分析说明既定调度方案的成本评价对关键成本系数变化不敏感；鲁棒性分析则说明模型在多类典型突发事件下能够保持政策可行，并对成本和碳排放偏离进行有效控制。因此，本文所建模型不仅能给出单一场景下的优化方案，也具有一定的参数稳定性和动态适应能力。

八、模型评价

8.1 模型的优点

1. 模型框架较为完整。本文将车辆启用、容量约束、时变速度、软时间窗、能源消耗、碳排放和绿色配送区限行要求纳入统一调度框架，能够较全面地刻画城市绿色物流配送中的主要成本来源与政策约束。
2. 模型具有较好的层次性与可扩展性。三个问题分别对应静态调度、限行约束调度和突发事件动态调度，求解过程由基础路径优化逐步扩展到车型重分配与滚动重调度，逻辑递进清晰，便于针对不同业务场景进行调整。
3. 求解结果具有一定实践解释力。计算结果表明，模型能够在满足限行合规的前提下兼顾总成本、服务水平和低碳目标；在动态事件发生后，仅对未发车任务进行局部重构，既保持方案稳定性，也体现了较强的应急响应能力。
4. 结果分析具有较好的决策支持作用。本文从成本构成、车辆结构、碳排放、服务水平和动态响应等多个维度对方案进行比较，能够较直观地解释限行政策和突发事件对调度结果的影响，为企业车辆配置和应急调度策略制定提供参考。

8.2 模型的缺点

1. 速度参数主要依据分时段统计规律设定，对具体路段的实时拥堵、信号控制和道路通行状态刻画不足，因此到达时刻估计仍存在一定简化。
2. 动态调度场景以单一突发事件为主，对多事件连续发生、车辆状态频繁变化以及客户需求实时更新等复杂情形的描述仍不够充分。

3. 求解方法以启发式搜索和局部修复为主，适合较大规模算例的快速求解，但难以严格证明所得方案的全局最优性。

8.3 模型的改进

1. 可进一步引入实时交通数据和路段级速度预测模型，使旅行时间估计由分时段平均值扩展为随道路状态变化的动态参数 [4]。
2. 可在现有滚动重调度框架中加入鲁棒优化或随机规划思想，增强模型对连续扰动、复合事件和需求波动的适应能力 [8]。
3. 可结合精确算法、元启发式算法与并行计算策略，对关键子问题进行更充分搜索，从而提升大规模场景下的求解质量与稳定性。

参考文献

- [1] Dantzig G B, Ramser J H. The truck dispatching problem[J]. Management Science, 1959, 6(1): 80–91.
- [2] Solomon M M. Algorithms for the vehicle routing and scheduling problems with time window constraints[J]. Operations Research, 1987, 35(2): 254–265.
- [3] Bektaş T, Laporte G. The pollution-routing problem[J]. Transportation Research Part B: Methodological, 2011, 45(8): 1232–1250.
- [4] 曹庆奎, 杨凯文, 任向阳, 等. 基于交通流的多模糊时间窗车辆路径优化 [J]. 运筹与管理, 2018, 27(8): 20–26.
- [5] Laporte G. The vehicle routing problem: An overview of exact and approximate algorithms[J]. European Journal of Operational Research, 1992, 59(3): 345–358.
- [6] 唐慧玲, 唐恒书, 朱兴亮. 基于改进蚁群算法的低碳车辆路径问题研究 [J]. 中国管理科学, 2021, 29(7): 118–127.
- [7] Psaraftis H N. Dynamic vehicle routing problems[M]//Golden B L, Assad A A. Vehicle Routing: Methods and Studies. Amsterdam: North-Holland, 1988: 223–248.
- [8] Pillac V, Gendreau M, Guéret C, et al. A review of dynamic vehicle routing problems[J]. European Journal of Operational Research, 2013, 225(1): 1–11.

附录

附录 A 支撑材料列表

本文支撑材料包括源程序、过程结果文件和 AI 使用报告。

表 14 支撑材料列表

材料类别	文件路径	说明
Latex 源文件 可运行源程序	main.tex ../0-预处理.py; ../1-问题一.py; ../2-问题二.py; ../2-问题二数据后处 理.py; ../3-问题三.py; ../3-问题三后处 理.py; ../3-问题三情景生成.py; ../4-敏 感性与鲁棒性分析.py	论文源文件 数据预处理、模型求 解、结果后处理与稳定 性分析程序
问题一结果	../过程/1-问题一最优方案.xlsx; ../过 程/1-问题一求解流程图.png; ../过 程/1-问题一车辆使用方案图.png; ../过 程/1-问题一车辆行驶路径图.png; ../过 程/1-问题一成本构成图.png	静态调度方案与图表
问题二结果	../过程/2-问题二限行政策调度方案.xlsx; ../过程/2-问题一问题二综合对比.xlsx; ../过程/2-问题二求解流程图.png; ../过 程/2-问题二车辆使用结构图.png; ../过 程/2-问题二车辆行驶路径图.png; ../过 程/2-问题一问题二变化幅度图.png; ../过 程/2-问题一问题二车辆结构对比图.png; ../过程/2-问题一问题二成本构成对比 图.png; ../过程/2-问题一问题二关键指标 对比图.png	限行政策调度结果与对 比图表
问题三结果	../过程/3-问题三动态调度方案.xlsx; ../过程/3-问题三情景事件.xlsx; ../过 程/3-问题三情景说明.xlsx; ../过程/3-问 题三综合对比.xlsx; ../过程/3-问题三事 件响应路径图.png; ../过程/3-问题三综合 改善瀑布图.png; ../过程/3-问题三事件前 后成本对比图.png; ../过程/3-问题三低碳 响应对比图.png; ../过程/3-问题三服务水 平变化图.png; ../过程/3-问题三车辆结构 与装载图.png; ../过程/3-问题三迟到-成 本气泡图.png; ../过程/3-问题三情景韧性 雷达图.png; ../过程/3-问题三车辆响应甘 特图-S1.png 至 S5.png	动态场景、路径响应与 综合对比结果
敏感性与鲁棒性 结果	../过程/4-敏感性与鲁棒性分析.xlsx; ../过程/4-敏感性分析图.png; ../过 程/4-鲁棒性分析图.png	参数扰动与多场景鲁棒 性结果
其他支撑文件	../过程/0-客户坐标分布图.png; ../过 程/0-已填充订单数据.xlsx; ../AI 使用报 告.pdf	预处理结果与 AI 使用 报告

附录 B 核心程序说明

本附录给出当前工程中可运行源程序的核心代码。

B.1 数据预处理程序：0-预处理.py

```
import math
import os

import matplotlib.pyplot as plt
import pandas as pd
from matplotlib.patches import Circle

ATTACH_DIR = r".\附件"
PROC_DIR = r".\过程"
os.makedirs(PROC_DIR, exist_ok=True)

INPUT_FILE = os.path.join(ATTACH_DIR, " 订单信息.xlsx")
COORD_FILE = os.path.join(ATTACH_DIR, " 客户坐标信息.xlsx")
OUTPUT_FILE = os.path.join(PROC_DIR, "0-已填充订单数据.xlsx")
COORD_PNG = os.path.join(PROC_DIR, "0-客户坐标分布图.png")

GREEN_RADIUS = 10.0

def setup_chinese_font():
    try:
        plt.rcParams["font.sans-serif"] = [
            "Microsoft YaHei",
            "SimHei",
            "Noto Sans CJK SC",
            "Arial Unicode MS",
        ]
        plt.rcParams["axes.unicode_minus"] = False
    except Exception:
        pass

def read_orders():
    try:
        orders = pd.read_excel(INPUT_FILE)
    except Exception as exc:
        print(f" 错误: 无法读取文件 {INPUT_FILE}")
        raise exc

    required_cols = {" 订单编号", " 重量", " 体积", " 目标客户编号"}
    missing_cols = required_cols - set(orders.columns)
    if missing_cols:
        raise ValueError(f" 订单数据缺少必要字段: {missing_cols}")
    return orders

def fill_missing_values(orders):
    valid_mask = (
        orders[" 重量"].notna()
        & orders[" 体积"].notna()
        & (orders[" 重量"] > 0)
        & (orders[" 体积"] > 0)
    )

    if valid_mask.any():
        avg_density = (orders.loc[valid_mask, " 重量"] / orders.loc[valid_mask, " 体积"]).mean()
    else:
        avg_density = 250.0

    orders[" 重量"] = orders.apply(
        lambda row: row[" 体积"] * avg_density
```

```

        if (pd.isna(row["重量"]) or row["重量"] <= 0) and pd.notna(row["体积"]) and row["体积"] > 0
        else row["重量"],
        axis=1,
    )

orders["体积"] = orders.apply(
    lambda row: row["重量"] / avg_density
    if (pd.isna(row["体积"]) or row["体积"] <= 0) and pd.notna(row["重量"]) and row["重量"] > 0
    else row["体积"],
    axis=1,
)
return orders, float(avg_density)

def read_coordinates():
    coord_df = pd.read_excel(COORD_FILE)
    coord_df = coord_df.rename(columns={coord_df.columns[0]: "客户类型"})
    coord_df["半径"] = (coord_df["X (km)"] ** 2 + coord_df["Y (km)"] ** 2).pow(0.5)
    coord_df["区域属性"] = coord_df["半径"].apply(lambda x: "绿色配送区" if x <= GREEN_RADIUS + 1e-9 else "非绿色配送区")
    return coord_df

def draw_customer_map(coord_df):
    setup_chinese_font()

    center_df = coord_df[coord_df["ID"] == 0].copy()
    customer_df = coord_df[coord_df["ID"] != 0].copy()
    green_df = customer_df[customer_df["区域属性"] == "绿色配送区"].copy()
    normal_df = customer_df[customer_df["区域属性"] == "非绿色配送区"].copy()

    fig, ax = plt.subplots(figsize=(11.6, 9.6), dpi=220)
    fig.patch.set_facecolor("white")
    ax.set_facecolor("white")

    outer_ring = Circle((0, 0), GREEN_RADIUS * 1.7, color="#efe4c8", alpha=0.55, zorder=0)
    green_ring = Circle((0, 0), GREEN_RADIUS, color="#9ed8a3", alpha=0.24, zorder=0)
    border_ring = Circle((0, 0), GREEN_RADIUS, fill=False, color="#2f855a", lw=2.0, zorder=1)
    ax.add_patch(outer_ring)
    ax.add_patch(green_ring)
    ax.add_patch(border_ring)

    ax.scatter(
        normal_df["X (km)"],
        normal_df["Y (km)"],
        s=42,
        c="#7c8b95",
        alpha=0.62,
        edgecolors="white",
        linewidths=0.8,
        label="非绿色配送区客户",
        zorder=2,
    )

    ax.scatter(
        green_df["X (km)"],
        green_df["Y (km)"],
        s=54,
        c="#2f855a",
        alpha=0.90,
        edgecolors="white",
        linewidths=0.9,
        label="绿色配送区客户",
        zorder=3,
    )

    if not center_df.empty:
        ax.scatter(
            center_df["X (km)"],
            center_df["Y (km)"],
            s=320,

```

```

        c="#c1121f",
        marker="*",
        edgecolors="white",
        linewidths=1.1,
        label=" 配送中心",
        zorder=4,
    )

sample_df = customer_df.nlargest(min(10, len(customer_df)), " 半径")
for _, row in sample_df.iterrows():
    angle = math.atan2(float(row["Y (km)"]), float(row["X (km)"]))
    dx = 0.35 * math.cos(angle)
    dy = 0.35 * math.sin(angle)
    ax.text(
        float(row["X (km)"]) + dx,
        float(row["Y (km)"]) + dy,
        f"C{int(row['ID'])}",
        fontsize=8.5,
        color="#475569",
        alpha=0.92,
    )

ax.text(0, GREEN_RADIUS + 0.9, " 绿色配送区", ha="center", fontsize=12, color="#276749", weight="bold")
ax.text(0, GREEN_RADIUS * 1.48, " 城市客户空间分布", ha="center", fontsize=11, color="#7c6f64")

ax.grid(True, linestyle="--", alpha=0.16)
ax.set_xlabel("X / km")
ax.set_ylabel("Y / km")
ax.legend(frameon=False, loc="upper right")
ax.set_aspect("equal", adjustable="box")
fig.tight_layout()
fig.savefig(COORD_PNG, bbox_inches="tight")
plt.close(fig)

def main():
    orders = read_orders()
    orders, avg_density = fill_missing_values(orders)
    orders.to_excel(OUTPUT_FILE, index=False)

    customer_summary = (
        orders.groupby(" 目标客户编号", as_index=False)[[" 重量", " 体积"]]
        .sum()
        .rename(columns={" 重量": " 客户总重量", " 体积": " 客户总体积"})
    )

    coord_df = read_coordinates()
    draw_customer_map(coord_df)
    green_customers = int(((coord_df["ID"] != 0) & (coord_df[" 区域属性"] == " 绿色配送区")).sum())
    normal_customers = int(((coord_df["ID"] != 0) & (coord_df[" 区域属性"] == " 非绿色配送区")).sum())

    print("-" * 56)
    print(f" 已生成: {OUTPUT_FILE}")
    print(f" 客户坐标图: {COORD_PNG}")
    print(f" 订单数: {len(orders)}")
    print(f" 客户数: {orders[' 目标客户编号'].nunique()}")
    print(f" 平均密度估计: {avg_density:.4f} kg/m3")
    print(f" 填充后总重量: {orders[' 重量'].sum():.4f} kg")
    print(f" 填充后总体积: {orders[' 体积'].sum():.4f} m3")
    print(f" 客户最大总重量: {customer_summary[' 客户总重量'].max():.4f} kg")
    print(f" 客户最大总体积: {customer_summary[' 客户总体积'].max():.4f} m3")
    print(f" 绿色配送区客户数: {green_customers}")
    print(f" 非绿色配送区客户数: {normal_customers}")
    print("-" * 56)

if __name__ == "__main__":
    main()

```

B.2 问题一求解程序：1-问题一.py

```
import math
import os
import random
from collections import defaultdict

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import font_manager

# =====
# 1. 基本参数
# =====
ATTACH_DIR = r".\附件"
PROC_DIR = r".\过程"
os.makedirs(PROC_DIR, exist_ok=True)

RANDOM_SEED = 20260423
random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)

START_HOUR = 8.0
SERVICE_TIME = 20.0 / 60.0
START_COST = 400.0
FUEL_PRICE = 7.61
EV_PRICE = 1.64
CARBON_PRICE = 0.65
WAIT_PENALTY = 20.0
LATE_PENALTY = 50.0

# 速度服从正态分布。表中写法按  $N(\text{mean}, \text{sigma}^2)$  理解。
# 题目表 1 写作  $v(t) \sim N(\mu, \sigma^2)$ , 第二个参数按“方差”解释, 因此需转为标准差采样。
# 结合题意补充假设: 17:00-19:00 为一般时段, 19:00 之后为顺畅时段。
SPEED_PERIODS = [
    (8.0, 9.0, " 拥堵", 9.8, 4.7),
    (9.0, 10.0, " 顺畅", 55.3, 0.1),
    (10.0, 11.5, " 一般", 35.4, 5.2),
    (11.5, 13.0, " 拥堵", 9.8, 4.7),
    (13.0, 15.0, " 顺畅", 55.3, 0.1),
    (15.0, 17.0, " 一般", 35.4, 5.2),
    (17.0, 19.0, " 一般", 35.4, 5.2),
    (19.0, 24.0, " 顺畅", 55.3, 0.1),
]

NUM_SPEED_SCENARIOS = 7
MIN_SPEED = 3.0
DEPARTURE_STEP = 10.0 / 60.0
LOCAL_SEARCH_PASSES = 3
CROSS_ROUTE_PASSES = 5
BAD_ROUTE_SA_ITER = 450
BAD_ROUTE_GROUP_MAX = 3
# 题面未给出“24:00 前必须返场”的硬约束, 这里不再额外施加返场超时罚金,
# 仅保留字段以兼容后续汇总与输出。
RETURN_DEADLINE = None
RETURN_OVERTIME_PENALTY = 0.0

VEHICLES = {
    " 新能源 1": {"power": "ev", "cap_w": 3000.0, "cap_v": 15.0, "count": 10, "load_coef": 0.35},
    " 新能源 2": {"power": "ev", "cap_w": 1250.0, "cap_v": 8.5, "count": 15, "load_coef": 0.35},
    " 燃油车 1": {"power": "fuel", "cap_w": 3000.0, "cap_v": 13.5, "count": 60, "load_coef": 0.40},
    " 燃油车 2": {"power": "fuel", "cap_w": 1500.0, "cap_v": 10.8, "count": 50, "load_coef": 0.40},
    " 燃油车 3": {"power": "fuel", "cap_w": 1250.0, "cap_v": 6.5, "count": 50, "load_coef": 0.40},
}

FLEET_ORDER = [" 新能源 1", " 燃油车 1", " 燃油车 2", " 新能源 2", " 燃油车 3"]
SPLIT_CAP_W = min(v["cap_w"] for v in VEHICLES.values())
```

```

SPLIT_CAP_V = min(v["cap_v"] for v in VEHICLES.values())

# =====
# 2. 数据读取与任务生成
# =====

def parse_time_to_hour(value):
    if pd.isna(value):
        return START_HOUR
    if hasattr(value, "hour"):
        return value.hour + value.minute / 60.0 + value.second / 3600.0
    parts = str(value).strip().split(":")
    hour = int(parts[0])
    minute = int(parts[1]) if len(parts) > 1 else 0
    second = int(parts[2]) if len(parts) > 2 else 0
    return hour + minute / 60.0 + second / 3600.0

def hour_to_time(value):
    total_minutes = int(round(value * 60))
    hour = total_minutes // 60
    minute = total_minutes % 60
    return f"{hour:02d}:{minute:02d}"

def read_distance_matrix():
    dist_df = pd.read_excel(os.path.join(ATTACH_DIR, " 距离矩阵.xlsx"), index_col=0)
    return dist_df.values.astype(float)

def read_time_windows():
    tw_df = pd.read_excel(os.path.join(ATTACH_DIR, " 时间窗.xlsx"))
    tw = {}
    for _, row in tw_df.iterrows():
        customer = int(row[" 客户编号"])
        tw[customer] = {
            "early": parse_time_to_hour(row[" 开始时间"]),
            "late": parse_time_to_hour(row[" 结束时间"]),
        }
    return tw

def read_coordinates():
    coord_df = pd.read_excel(os.path.join(ATTACH_DIR, " 客户坐标信息.xlsx"))
    coord_df = coord_df.rename(columns={coord_df.columns[0]: " 类型"})
    coords = {}
    for _, row in coord_df.iterrows():
        coords[int(row["ID"])] = {"x": float(row["X (km)"]), "y": float(row["Y (km)"]), "type": row[" 类型"]}
    return coords

def build_delivery_tasks():
    filled_path = os.path.join(PROC_DIR, "0-已填充订单数据.xlsx")
    if not os.path.exists(filled_path):
        filled_path = os.path.join(ATTACH_DIR, " 订单信息.xlsx")

    orders = pd.read_excel(filled_path)
    required_cols = {" 目标客户编号", " 重量", " 体积"}
    missing_cols = required_cols - set(orders.columns)
    if missing_cols:
        raise ValueError(f" 订单数据缺少列: {missing_cols}")

    customer_demands = orders.groupby(" 目标客户编号", as_index=False)[[" 重量", " 体积"]].sum()
    tasks = []
    for _, row in customer_demands.iterrows():
        customer = int(row[" 目标客户编号"])
        weight = float(row[" 重量"])
        volume = float(row[" 体积"])
        split_count = max(1, math.ceil(max(weight / SPLIT_CAP_W, volume / SPLIT_CAP_V)))

```

```

        for idx in range(split_count):
            tasks.append(
                {
                    "task_id": f"{customer}_{idx + 1}",
                    "customer": customer,
                    "weight": weight / split_count,
                    "volume": volume / split_count,
                    "source_order_count": int(split_count),
                }
            )
        return tasks, customer_demands

# =====
# 3. 正态速度、时间与成本模型
# =====
def generate_speed_scenarios():
    rng = np.random.default_rng(RANDOM_SEED)
    scenarios = []
    for sid in range(NUM_SPEED_SCENARIOS):
        periods = []
        for start, end, label, mean, variance in SPEED_PERIODS:
            std = math.sqrt(variance)
            speed = max(MIN_SPEED, float(rng.normal(mean, std)))
            periods.append({"start": start, "end": end, "label": label, "speed": speed})
        scenarios.append({"scenario_id": sid + 1, "periods": periods})
    return scenarios

def mean_speed_periods():
    return [
        {"start": start, "end": end, "label": label, "speed": mean}
        for start, end, label, mean, _ in SPEED_PERIODS
    ]

def get_period(current_hour, periods):
    for period in periods:
        if period["start"] <= current_hour < period["end"]:
            return period
    return periods[-1]

def travel_segments(distance_km, depart_hour, periods):
    if distance_km <= 1e-9:
        return depart_hour, []

    current = depart_hour
    remaining = float(distance_km)
    segments = []
    while remaining > 1e-9:
        period = get_period(current, periods)
        speed = period["speed"]
        period_end = period["end"]
        if current >= period_end:
            period_end = current + 24.0

        available_time = max(1e-9, period_end - current)
        possible_distance = speed * available_time
        if remaining <= possible_distance + 1e-9:
            duration = remaining / speed
            segments.append({"distance": remaining, "speed": speed, "duration": duration})
            current += duration
            remaining = 0.0
        else:
            segments.append({"distance": possible_distance, "speed": speed, "duration": available_time})
            remaining -= possible_distance
            current = period_end
    return current, segments

```



```

def energy_and_carbon_cost(distance, speed, vehicle_spec, load_ratio):
    load_factor = 1.0 + vehicle_spec["load_coef"] * max(0.0, min(1.0, load_ratio))
    if vehicle_spec["power"] == "fuel":
        amount_per_100km = (0.0025 * speed**2 - 0.2554 * speed + 31.75) * load_factor
        amount = amount_per_100km * distance / 100.0
        return amount * FUEL_PRICE, amount * 2.547 * CARBON_PRICE

    amount_per_100km = (0.0014 * speed**2 - 0.12 * speed + 36.19) * load_factor
    amount = amount_per_100km * distance / 100.0
    return amount * EV_PRICE, amount * 0.501 * CARBON_PRICE

def route_load(stops):
    return sum(s["weight"] for s in stops), sum(s["volume"] for s in stops)

def route_distance(stops, dist_matrix):
    if not stops:
        return 0.0
    total = 0.0
    prev = 0
    for stop in stops:
        total += dist_matrix[prev][stop["customer"]]
        prev = stop["customer"]
    total += dist_matrix[prev][0]
    return total

def evaluate_route_once(stops, vehicle_name, dist_matrix, time_windows, scenario, depart_hour, with_detail=False):
    vehicle = VEHICLES[vehicle_name]
    total_weight, total_volume = route_load(stops)
    if total_weight > vehicle["cap_w"] + 1e-8 or total_volume > vehicle["cap_v"] + 1e-8:
        return None

    current_time = depart_hour
    current_loc = 0
    current_load = total_weight
    energy_cost = 0.0
    carbon_cost = 0.0
    time_window_cost = 0.0
    total_wait = 0.0
    total_late = 0.0
    detail_rows = []

    for seq, stop in enumerate(stops, start=1):
        customer = stop["customer"]
        distance = dist_matrix[current_loc][customer]
        arrive_time, segments = travel_segments(distance, current_time, scenario["periods"])
        load_ratio = current_load / vehicle["cap_w"]
        for segment in segments:
            e_cost, c_cost = energy_and_carbon_cost(
                segment["distance"], segment["speed"], vehicle, load_ratio
            )
            energy_cost += e_cost
            carbon_cost += c_cost

        window = time_windows.get(customer, {"early": START_HOUR, "late": 24.0})
        wait_hours = max(0.0, window["early"] - arrive_time)
        late_hours = max(0.0, arrive_time - window["late"])
        total_wait += wait_hours
        total_late += late_hours
        time_window_cost += wait_hours * WAIT_PENALTY + late_hours * LATE_PENALTY
        service_start = max(arrive_time, window["early"])
        depart_time = service_start + SERVICE_TIME

        if with_detail:
            detail_rows.append(

```

```

        {
            "seq": seq,
            "customer": customer,
            "arrival": arrive_time,
            "service_start": service_start,
            "departure": depart_time,
            "wait_hours": wait_hours,
            "late_hours": late_hours,
            "travel_distance": distance,
        }
    )

    current_time = depart_time
    current_load -= stop["weight"]
    current_loc = customer

    back_distance = dist_matrix[current_loc][0]
    return_time, back_segments = travel_segments(back_distance, current_time, scenario["periods"])
    for segment in back_segments:
        e_cost, c_cost = energy_and_carbon_cost(segment["distance"], segment["speed"], vehicle, 0.0)
        energy_cost += e_cost
        carbon_cost += c_cost

    overtime_hours = 0.0
    overtime_cost = 0.0

    result = {
        "depart": depart_hour,
        "return": return_time,
        "start": START_COST,
        "energy": energy_cost,
        "carbon": carbon_cost,
        "time_window": time_window_cost,
        "overtime_hours": overtime_hours,
        "overtime_cost": overtime_cost,
        "wait_hours": total_wait,
        "late_hours": total_late,
        "total": START_COST + energy_cost + carbon_cost + time_window_cost + overtime_cost,
    }
    if with_detail:
        result["details"] = detail_rows
    return result

def estimate_departure_bounds(stops, dist_matrix, time_windows):
    if not stops:
        return START_HOUR, START_HOUR

    mean_scenario = {"scenario_id": 0, "periods": mean_speed_periods()}
    travel_and_service = 0.0
    current_loc = 0
    for stop in stops:
        arrive, _ = travel_segments(dist_matrix[current_loc][stop["customer"]], START_HOUR + travel_and_service,
        ↪ mean_scenario["periods"])
        travel_and_service = arrive - START_HOUR + SERVICE_TIME
        current_loc = stop["customer"]

    first_late = min(time_windows.get(stop["customer"], {"late": 24.0})["late"] for stop in stops)
    first_early = min(time_windows.get(stop["customer"], {"early": START_HOUR})["early"] for stop in stops)
    lower = max(START_HOUR, first_early - travel_and_service - 0.5)
    upper = min(22.0, first_late + 0.5)
    if upper < lower:
        upper = lower
    return lower, upper

def best_departure_time(stops, vehicle_name, dist_matrix, time_windows, speed_scenarios):
    lower, upper = estimate_departure_bounds(stops, dist_matrix, time_windows)
    candidates = set()

```

```

current = math.floor(lower / DEPARTURE_STEP) * DEPARTURE_STEP
while current <= upper + 1e-9:
    candidates.add(round(max(START_HOUR, current), 6))
    current += DEPARTURE_STEP
candidates.add(START_HOUR)

best_depart = START_HOUR
best_total = float("inf")
for depart in sorted(candidates):
    values = []
    feasible = True
    for scenario in speed_scenarios:
        result = evaluate_route_once(stops, vehicle_name, dist_matrix, time_windows, scenario, depart)
        if result is None:
            feasible = False
            break
        values.append(result["total"])
    if feasible:
        avg_total = float(np.mean(values))
        if avg_total < best_total:
            best_total = avg_total
            best_depart = depart
return best_depart

def evaluate_route(stops, vehicle_name, dist_matrix, time_windows, speed_scenarios, with_detail=False, depart_hour=None):
    if not stops:
        return {
            "depart": START_HOUR,
            "return": START_HOUR,
            "total": 0.0,
            "start": 0.0,
            "energy": 0.0,
            "carbon": 0.0,
            "time_window": 0.0,
            "wait_hours": 0.0,
            "late_hours": 0.0,
        }

    vehicle = VEHICLES[vehicle_name]
    total_weight, total_volume = route_load(stops)
    if total_weight > vehicle["cap_w"] + 1e-8 or total_volume > vehicle["cap_v"] + 1e-8:
        return None

    if depart_hour is None:
        depart_hour = best_departure_time(stops, vehicle_name, dist_matrix, time_windows, speed_scenarios)

    scenario_results = []
    for scenario in speed_scenarios:
        scenario_results.append(
            evaluate_route_once(stops, vehicle_name, dist_matrix, time_windows, scenario, depart_hour, with_detail)
        )

    avg = {
        key: float(np.mean([result[key] for result in scenario_results]))
        for key in [
            "depart",
            "return",
            "start",
            "energy",
            "carbon",
            "time_window",
            "overtime_hours",
            "overtime_cost",
            "wait_hours",
            "late_hours",
            "total",
        ]
    }
}

```

```

if with_detail:
    merged_details = []
    for seq in range(len(stops)):
        rows = [result["details"][seq] for result in scenario_results]
        base = rows[0].copy()
        for key in ["arrival", "service_start", "departure", "wait_hours", "late_hours"]:
            base[key] = float(np.mean([row[key] for row in rows]))
        merged_details.append(base)
    avg["details"] = merged_details
return avg

# =====
# 4. 构造与改进可行解
# =====

def combine_same_customer(tasks):
    grouped = {}
    for task in tasks:
        customer = task["customer"]
        if customer not in grouped:
            grouped[customer] = {
                "customer": customer,
                "weight": 0.0,
                "volume": 0.0,
                "task_ids": [],
            }
        grouped[customer]["weight"] += task["weight"]
        grouped[customer]["volume"] += task["volume"]
        if "task_id" in task:
            grouped[customer]["task_ids"].append(task["task_id"])
        else:
            grouped[customer]["task_ids"].extend(task.get("task_ids", []))
    return list(grouped.values())

def normalize_route_stops(stops):
    return combine_same_customer(stops)

def order_stops(stops, dist_matrix, time_windows, periods):
    remaining = stops[:]
    ordered = []
    current_loc = 0
    current_time = START_HOUR

    while remaining:
        best_idx = 0
        best_score = float("inf")
        for idx, stop in enumerate(remaining):
            customer = stop["customer"]
            arrive, _ = travel_segments(dist_matrix[current_loc][customer], current_time, periods)
            window = time_windows.get(customer, {"early": START_HOUR, "late": 24.0})
            wait = max(0.0, window["early"] - arrive)
            late = max(0.0, arrive - window["late"])
            score = late * 1000.0 + wait * 20.0 + dist_matrix[current_loc][customer] + 0.05 * window["late"]
            if score < best_score:
                best_score = score
                best_idx = idx

        chosen = remaining.pop(best_idx)
        customer = chosen["customer"]
        arrive, _ = travel_segments(dist_matrix[current_loc][customer], current_time, periods)
        window = time_windows.get(customer, {"early": START_HOUR, "late": 24.0})
        current_time = max(arrive, window["early"]) + SERVICE_TIME
        current_loc = customer
        ordered.append(chosen)

    return ordered

```

```

def try_two_opt(stops, dist_matrix, time_windows, periods):
    if len(stops) < 4:
        return stops

    best = stops[:]
    best_distance = route_distance(best, dist_matrix)
    improved = True
    while improved:
        improved = False
        for i in range(0, len(best) - 2):
            for j in range(i + 2, len(best)):
                candidate = best[:i] + list(reversed(best[i:j + 1])) + best[j + 1:]
                candidate_distance = route_distance(candidate, dist_matrix)
                if candidate_distance + 1e-8 < best_distance:
                    best = candidate
                    best_distance = candidate_distance
                    improved = True
                    break
            if improved:
                break
    return order_stops(best, dist_matrix, time_windows, periods)

def build_capacity_focused_routes(customer_demands, dist_matrix, time_windows):
    """ 按实际可用车型容量动态拆分客户需求，目标是先压低启动车辆数。 """
    residual = {}
    for _, row in customer_demands.iterrows():
        customer = int(row["目标客户编号"])
        residual[customer] = {
            "weight": float(row["重量"]),
            "volume": float(row["体积"]),
            "part": 0,
        }

    fleet = []
    for vehicle_name in FLEET_ORDER:
        fleet.extend([vehicle_name] * VEHICLES[vehicle_name]["count"])

    routes = []
    mean_periods = mean_speed_periods()
    eps = 1e-7

    for vehicle_name in fleet:
        if sum(item["weight"] for item in residual.values()) <= eps:
            break

        vehicle = VEHICLES[vehicle_name]
        remain_w = vehicle["cap_w"]
        remain_v = vehicle["cap_v"]
        current_loc = 0
        current_time = START_HOUR
        raw_stops = []
        used_customers = set()

        while remain_w > eps and remain_v > eps:
            best_customer = None
            best_take = None
            best_score = float("inf")

            for customer, item in residual.items():
                if customer in used_customers:
                    continue
                if item["weight"] <= eps or item["volume"] <= eps:
                    continue

            fraction = min(1.0, remain_w / item["weight"], remain_v / item["volume"])
            if fraction <= eps:

```

```

        continue

    take_w = item["weight"] * fraction
    take_v = item["volume"] * fraction
    fill_ratio = max(take_w / vehicle["cap_w"], take_v / vehicle["cap_v"])
    window = time_windows.get(customer, {"early": START_HOUR, "late": 24.0})
    arrive, _ = travel_segments(dist_matrix[current_loc][customer], current_time, mean_periods)
    wait = max(0.0, window["early"] - arrive)
    late = max(0.0, arrive - window["late"])

    if not raw_stops:
        # 首站优先选能高比例装满车辆且时间窗更早的客户。
        score = -10000.0 * fill_ratio + 20.0 * window["late"] + dist_matrix[0][customer]
    else:
        score = (
            -8000.0 * fill_ratio
            + 800.0 * late
            + 10.0 * wait
            + dist_matrix[current_loc][customer]
            + 5.0 * abs(window["late"] - time_windows.get(raw_stops[-1]["customer"], window["late"]))
        )

    if score < best_score:
        best_score = score
        best_customer = customer
        best_take = (take_w, take_v, fraction)

    if best_customer is None:
        break

    take_w, take_v, fraction = best_take
    residual[best_customer]["part"] += 1
    raw_stops.append(
        {
            "customer": best_customer,
            "weight": take_w,
            "volume": take_v,
            "task_ids": [f"{best_customer}_part{residual[best_customer]['part']}"],
        }
    )
    residual[best_customer]["weight"] -= take_w
    residual[best_customer]["volume"] -= take_v
    if residual[best_customer]["weight"] < eps:
        residual[best_customer]["weight"] = 0.0
    if residual[best_customer]["volume"] < eps:
        residual[best_customer]["volume"] = 0.0

    remain_w -= take_w
    remain_v -= take_v
    used_customers.add(best_customer)

    arrive, _ = travel_segments(dist_matrix[current_loc][best_customer], current_time, mean_periods)
    window = time_windows.get(best_customer, {"early": START_HOUR, "late": 24.0})
    current_time = max(arrive, window["early"]) + SERVICE_TIME
    current_loc = best_customer

    if raw_stops:
        stops = combine_same_customer(raw_stops)
        stops = order_stops(stops, dist_matrix, time_windows, mean_periods)
        stops = try_two_opt(stops, dist_matrix, time_windows, mean_periods)
        routes.append({"vehicle": vehicle_name, "stops": stops})

    remaining_weight = sum(item["weight"] for item in residual.values())
    if remaining_weight > 1e-5:
        raise RuntimeError(
            f" 车辆数量硬约束下仍有 {remaining_weight:.4f} kg 需求未分配，请检查数据或车辆参数。"
        )

    return routes

```

```

def candidate_fleet_orders():
    n1, n2, f1, f2, f3 = " 新能源 1", " 新能源 2", " 燃油车 1", " 燃油车 2", " 燃油车 3"
    return [
        [n1, f1, f2, n2, f3],
        [f1, n1, f2, n2, f3],
        [f1, f2, n1, n2, f3],
        [n1, f1, n2, f2, f3],
        [n1, n2, f1, f2, f3],
        [f1, f2, f3, n1, n2],
    ]

def select_best_fleet_order(customer_demands, dist_matrix, time_windows, speed_scenarios):
    global FLEET_ORDER
    best_order = FLEET_ORDER[:]
    best_cost = float("inf")
    best_used = float("inf")

    for order in candidate_fleet_orders():
        FLEET_ORDER = order[:]
        routes = build_capacity_focused_routes(customer_demands, dist_matrix, time_windows)
        routes = assign_vehicle_types(routes, dist_matrix, time_windows, speed_scenarios)
        total_cost = sum(route["cost"]["total"] for route in routes)
        used = len(routes)
        if used < best_used or (used == best_used and total_cost < best_cost):
            best_order = order[:]
            best_cost = total_cost
            best_used = used

    FLEET_ORDER = best_order[:]
    return best_order, best_used, best_cost

def build_initial_routes(tasks, dist_matrix, time_windows):
    unassigned = sorted(
        tasks,
        key=lambda x: (
            time_windows.get(x["customer"], {"late": 24.0})["late"],
            -x["weight"],
            -x["volume"],
        ),
    )
    routes = []
    mean_periods = mean_speed_periods()

    fleet = []
    for vehicle_name in FLEET_ORDER:
        fleet.extend([vehicle_name] * VEHICLES[vehicle_name]["count"])

    for vehicle_name in fleet:
        if not unassigned:
            break

        vehicle = VEHICLES[vehicle_name]
        route_tasks = []
        load_w = 0.0
        load_v = 0.0
        current_loc = 0
        current_time = START_HOUR

        while True:
            best_idx = None
            best_score = float("inf")
            for idx, task in enumerate(unassigned):
                if load_w + task["weight"] > vehicle["cap_w"] + 1e-8:
                    continue
                if load_v + task["volume"] > vehicle["cap_v"] + 1e-8:

```

```

        continue

        customer = task["customer"]
        arrive, _ = travel_segments(dist_matrix[current_loc][customer], current_time, mean_periods)
        window = time_windows.get(customer, {"early": START_HOUR, "late": 24.0})
        late = max(0.0, arrive - window["late"])
        wait = max(0.0, window["early"] - arrive)
        fill_reward = 0.04 * (task["weight"] / vehicle["cap_w"] + task["volume"] / vehicle["cap_v"])
        score = (
            late * 1000.0
            + wait * 10.0
            + dist_matrix[current_loc][customer]
            + 0.05 * window["late"]
            - fill_reward
        )
        if score < best_score:
            best_score = score
            best_idx = idx

    if best_idx is None:
        break

    chosen = unassigned.pop(best_idx)
    route_tasks.append(chosen)
    load_w += chosen["weight"]
    load_v += chosen["volume"]
    customer = chosen["customer"]
    arrive, _ = travel_segments(dist_matrix[current_loc][customer], current_time, mean_periods)
    window = time_windows.get(customer, {"early": START_HOUR, "late": 24.0})
    current_time = max(arrive, window["early"]) + SERVICE_TIME
    current_loc = customer

    if route_tasks:
        stops = combine_same_customer(route_tasks)
        stops = order_stops(stops, dist_matrix, time_windows, mean_periods)
        stops = try_two_opt(stops, dist_matrix, time_windows, mean_periods)
        routes.append({"vehicle": vehicle_name, "stops": stops})

    if unassigned:
        raise RuntimeError(
            f" 车辆数量硬约束下未能分配全部任务, 剩余 {len(unassigned)} 个任务。"
            " 请检查需求总量或拆分粒度。"
        )

    return routes

def max_vehicle_feasible(stops):
    load_w, load_v = route_load(stops)
    return any(load_w <= v["cap_w"] + 1e-8 and load_v <= v["cap_v"] + 1e-8 for v in VEHICLES.values())

def merge_light_routes(routes, dist_matrix, time_windows):
    mean_periods = mean_speed_periods()
    changed = True
    while changed:
        changed = False
        routes.sort(key=lambda r: (route_load(r["stops"])[0], route_load(r["stops"])[1]))

    for source_idx, source in enumerate(routes[:]):
        if len(routes) <= 1:
            return routes

        source_stops = source["stops"]
        target_indices = [idx for idx in range(len(routes)) if idx != source_idx]
        target_indices.sort(key=lambda idx: route_distance(routes[idx]["stops"], dist_matrix))

        trial_routes = [r["stops"][:] for r in routes]
        success = True

```



```

    for stop in source_stops:
        best_target = None
        best_distance = float("inf")
        for target_idx in target_indices:
            candidate = trial_routes[target_idx] + [stop]
            if not max_vehicle_feasible(candidate):
                continue
            distance = route_distance(candidate, dist_matrix)
            if distance < best_distance:
                best_distance = distance
                best_target = target_idx

        if best_target is None:
            success = False
            break
        trial_routes[best_target].append(stop)

    if success:
        new_routes = []
        for idx, route in enumerate(routes):
            if idx == source_idx:
                continue
            stops = combine_same_customer(trial_routes[idx])
            stops = order_stops(stops, dist_matrix, time_windows, mean_periods)
            stops = try_two_opt(stops, dist_matrix, time_windows, mean_periods)
            new_routes.append({"vehicle": route["vehicle"], "stops": stops})
        routes = new_routes
        changed = True
        break

return routes

def assign_vehicle_types(routes, dist_matrix, time_windows, speed_scenarios):
    remaining = {name: spec["count"] for name, spec in VEHICLES.items()}
    assigned = [None] * len(routes)

    def feasible_types(stops):
        load_w, load_v = route_load(stops)
        return [
            name
            for name, spec in VEHICLES.items()
            if load_w <= spec["cap_w"] + 1e-8 and load_v <= spec["cap_v"] + 1e-8
        ]

    route_order = sorted(
        range(len(routes)),
        key=lambda idx: (
            len(feasible_types(routes[idx]["stops"])),
            -route_distance(routes[idx]["stops"], dist_matrix),
            -route_load(routes[idx]["stops"])[0],
        ),
    )

    for idx in route_order:
        stops = routes[idx]["stops"]
        choices = [name for name in feasible_types(stops) if remaining[name] > 0]
        if not choices:
            raise RuntimeError(" 车辆数量硬约束下车型分配失败。")

        costs = []
        for name in choices:
            cost = evaluate_route(stops, name, dist_matrix, time_windows, speed_scenarios)
            costs.append((cost["total"], name, cost))
        _, chosen_name, chosen_cost = min(costs, key=lambda x: x[0])
        remaining[chosen_name] -= 1
        assigned[idx] = {"vehicle": chosen_name, "cost": chosen_cost}

    final_routes = []

```

```

used_counter = defaultdict(int)
for route, item in zip(routes, assigned):
    vehicle_name = item["vehicle"]
    used_counter[vehicle_name] += 1
    final_routes.append(
        {
            "route_id": f"R{len(final_routes) + 1:03d}",
            "vehicle": vehicle_name,
            "vehicle_no": f"{vehicle_name}-{used_counter[vehicle_name]:02d}",
            "stops": route["stops"],
            "cost": item["cost"],
        }
    )
)

return final_routes

def renumber_routes(routes):
    used_counter = defaultdict(int)
    for idx, route in enumerate(routes, start=1):
        route["route_id"] = f"R{idx:03d}"
        vehicle_name = route["vehicle"]
        used_counter[vehicle_name] += 1
        route["vehicle_no"] = f"{vehicle_name}-{used_counter[vehicle_name]:02d}"
    return routes

def assign_vehicle_types_with_limits(routes, vehicle_limits, dist_matrix, time_windows, speed_scenarios):
    old_counts = {name: spec["count"] for name, spec in VEHICLES.items()}
    try:
        for name, spec in VEHICLES.items():
            spec["count"] = int(vehicle_limits.get(name, 0))
        assigned = assign_vehicle_types(routes, dist_matrix, time_windows, speed_scenarios)
    finally:
        for name, old_count in old_counts.items():
            VEHICLES[name]["count"] = old_count
    return renumber_routes(assigned)

def route_signature(stops):
    return tuple((stop["customer"], round(stop["weight"], 4), round(stop["volume"], 4)) for stop in stops)

def optimized_route_cost(stops, vehicle_name, dist_matrix, time_windows, speed_scenarios, cache):
    key = (vehicle_name, route_signature(stops))
    if key not in cache:
        result = evaluate_route(stops, vehicle_name, dist_matrix, time_windows, speed_scenarios)
        cache[key] = result["total"] if result is not None else float("inf")
    return cache[key]

def improve_route_sequence(route, dist_matrix, time_windows, speed_scenarios):
    stops = normalize_route_stops(route["stops"][:])
    if len(stops) <= 1:
        route["stops"] = stops
        route["cost"] = evaluate_route(stops, route["vehicle"], dist_matrix, time_windows, speed_scenarios)
        return route

    vehicle_name = route["vehicle"]
    cache = {}
    best_cost = optimized_route_cost(stops, vehicle_name, dist_matrix, time_windows, speed_scenarios, cache)
    improved = True
    passes = 0

    while improved and passes < LOCAL_SEARCH_PASSES:
        improved = False
        passes += 1
        n = len(stops)
        candidates = []

```

```

    for i in range(n - 1):
        for j in range(i + 1, n):
            candidate = stops[:]
            candidate[i], candidate[j] = candidate[j], candidate[i]
            candidates.append(normalize_route_stops(candidate))

    for i in range(n - 1):
        for j in range(i + 1, n):
            candidate = stops[:i] + list(reversed(stops[i:j + 1])) + stops[j + 1:]
            candidates.append(normalize_route_stops(candidate))

    for i in range(n):
        for j in range(n):
            if i == j:
                continue
            candidate = stops[:]
            moved = candidate.pop(i)
            candidate.insert(j, moved)
            candidates.append(normalize_route_stops(candidate))

    best_candidate = None
    for candidate in candidates:
        cost = optimized_route_cost(candidate, vehicle_name, dist_matrix, time_windows, speed_scenarios, cache)
        if cost + 1e-6 < best_cost:
            best_cost = cost
            best_candidate = candidate

    if best_candidate is not None:
        stops = normalize_route_stops(best_candidate)
        improved = True

    route["stops"] = normalize_route_stops(stops)
    route["cost"] = evaluate_route(stops, vehicle_name, dist_matrix, time_windows, speed_scenarios)
    return route

def improve_all_routes(routes, dist_matrix, time_windows, speed_scenarios):
    improved = []
    before = 0.0
    after = 0.0
    for route in routes:
        old_cost = evaluate_route(route["stops"], route["vehicle"], dist_matrix, time_windows, speed_scenarios)
        before += old_cost["total"]
        improved_route = improve_route_sequence(route, dist_matrix, time_windows, speed_scenarios)
        after += improved_route["cost"]["total"]
        improved.append(improved_route)
    return improved, before, after

def route_is_feasible_for_vehicle(stops, vehicle_name):
    load_w, load_v = route_load(stops)
    vehicle = VEHICLES[vehicle_name]
    return load_w <= vehicle["cap_w"] + 1e-8 and load_v <= vehicle["cap_v"] + 1e-8

def improve_cross_route_swaps(routes, dist_matrix, time_windows, speed_scenarios):
    routes = [dict(route) for route in routes]
    total_before = sum(route["cost"]["total"] for route in routes)

    for _ in range(CROSS_ROUTE_PASSES):
        changed = False
        routes.sort(key=lambda r: r["cost"]["time_window"], reverse=True)
        for i in range(len(routes)):
            for j in range(i + 1, len(routes)):
                route_a = routes[i]
                route_b = routes[j]
                if not route_a["stops"] or not route_b["stops"]:
                    continue

```

```

base_cost = route_a["cost"]["total"] + route_b["cost"]["total"]
best_move = None
best_cost = base_cost

for ia, stop_a in enumerate(route_a["stops"]):
    for ib, stop_b in enumerate(route_b["stops"]):
        new_a = route_a["stops"][:]
        new_b = route_b["stops"][:]
        new_a[ia], new_b[ib] = stop_b, stop_a
        new_a = normalize_route_stops(new_a)
        new_b = normalize_route_stops(new_b)
        if not route_is_feasible_for_vehicle(new_a, route_a["vehicle"]):
            continue
        if not route_is_feasible_for_vehicle(new_b, route_b["vehicle"]):
            continue
        cost_a = evaluate_route(new_a, route_a["vehicle"], dist_matrix, time_windows, speed_scenarios)
        cost_b = evaluate_route(new_b, route_b["vehicle"], dist_matrix, time_windows, speed_scenarios)
        total_cost = cost_a["total"] + cost_b["total"]
        if total_cost + 1e-6 < best_cost:
            best_cost = total_cost
            best_move = (new_a, new_b, cost_a, cost_b)

if best_move is not None:
    new_a, new_b, cost_a, cost_b = best_move
    route_a["stops"] = new_a
    route_b["stops"] = new_b
    route_a = improve_route_sequence(route_a, dist_matrix, time_windows, speed_scenarios)
    route_b = improve_route_sequence(route_b, dist_matrix, time_windows, speed_scenarios)
    routes[i] = route_a
    routes[j] = route_b
    changed = True
    break
if changed:
    break
if not changed:
    break

routes.sort(key=lambda r: r["route_id"])
total_after = sum(route["cost"]["total"] for route in routes)
return routes, total_before, total_after

def anneal_bad_route_sequence(route, dist_matrix, time_windows, speed_scenarios):
    stops = normalize_route_stops(route["stops"])
    if len(stops) <= 3:
        route["stops"] = stops
        route["cost"] = evaluate_route(stops, route["vehicle"], dist_matrix, time_windows, speed_scenarios)
        return route

    vehicle_name = route["vehicle"]
    rng = random.Random(RANDOM_SEED + sum(stop["customer"] for stop in stops) + len(stops))
    current = stops[:]
    current_cost = evaluate_route(current, vehicle_name, dist_matrix, time_windows, speed_scenarios)["total"]
    best = current[:]
    best_cost = current_cost
    temp = 60.0

    for _ in range(BAD_ROUTE_SA_ITER):
        candidate = current[:]
        op = rng.random()
        if op < 0.4 and len(candidate) >= 2:
            i, j = sorted(rng.sample(range(len(candidate)), 2))
            candidate[i : j + 1] = reversed(candidate[i : j + 1])
        elif op < 0.75 and len(candidate) >= 2:
            i, j = rng.sample(range(len(candidate)), 2)
            candidate[i], candidate[j] = candidate[j], candidate[i]
        else:
            i, j = rng.sample(range(len(candidate)), 2)

```

```

        moved = candidate.pop(i)
        candidate.insert(j, moved)

    candidate = normalize_route_stops(candidate)
    candidate_cost = evaluate_route(candidate, vehicle_name, dist_matrix, time_windows, speed_scenarios)["total"]
    delta = candidate_cost - current_cost
    if delta < 0 or rng.random() < math.exp(-delta / max(temp, 1e-6)):
        current = candidate
        current_cost = candidate_cost
        if candidate_cost < best_cost:
            best = candidate
            best_cost = candidate_cost
    temp *= 0.985

route["stops"] = best
route["cost"] = evaluate_route(best, vehicle_name, dist_matrix, time_windows, speed_scenarios)
return improve_route_sequence(route, dist_matrix, time_windows, speed_scenarios)

def bad_route_groups(route, time_windows, max_group_size=BAD_ROUTE_GROUP_MAX):
    stops = route["stops"]
    groups = []
    if len(stops) <= 1:
        return groups

    # 优先迁移后半段和时间窗较晚的客户，这些客户最容易造成尾部迟到。
    tail_start = max(0, len(stops) // 2)
    if route.get("cost", {}).get("overtime_hours", 0.0) > 0:
        # 若已跨过 24 点，则更激进地拆尾部。
        groups.append(tuple(range(tail_start, len(stops))))
        if tail_start + 1 < len(stops):
            groups.append(tuple(range(tail_start + 1, len(stops))))
    for size in range(2, max_group_size + 1):
        for start in range(tail_start, max(0, len(stops) - size + 1)):
            idxs = tuple(range(start, start + size))
            groups.append(idxs)

    ranked_single = sorted(
        range(len(stops)),
        key=lambda idx: (
            time_windows.get(stops[idx]["customer"], {"late": 24.0})["late"],
            stops[idx]["weight"],
        ),
        reverse=True,
    )
    groups.extend((idx,) for idx in ranked_single[: min(8, len(ranked_single))])

    seen = set()
    unique_groups = []
    for group in groups:
        if group not in seen and len(group) < len(stops):
            seen.add(group)
            unique_groups.append(group)
    return unique_groups

def remove_group(stops, indices):
    idx_set = set(indices)
    moved = [stop for idx, stop in enumerate(stops) if idx in idx_set]
    remaining = [stop for idx, stop in enumerate(stops) if idx not in idx_set]
    return normalize_route_stops(remaining), normalize_route_stops(moved)

def best_insert_group(target_stops, group, vehicle_name, dist_matrix, time_windows, speed_scenarios):
    best_stops = None
    best_cost = None
    base = target_stops[:]
    for pos in range(len(base) + 1):
        candidate = normalize_route_stops(base[:pos] + group + base[pos:])

```

```

        if not route_is_feasible_for_vehicle(candidate, vehicle_name):
            continue
        cost = evaluate_route(candidate, vehicle_name, dist_matrix, time_windows, speed_scenarios)
        if best_cost is None or cost["total"] < best_cost["total"]:
            best_cost = cost
            best_stops = candidate
    return best_stops, best_cost

def migrate_group_from_route(routes, idx, group_indices, dist_matrix, time_windows, speed_scenarios):
    route = routes[idx]
    remaining_stops, moved_group = remove_group(route["stops"], group_indices)
    if not remaining_stops:
        return False
    if not route_is_feasible_for_vehicle(remaining_stops, route["vehicle"]):
        return False

    remaining_cost = evaluate_route(
        remaining_stops, route["vehicle"], dist_matrix, time_windows, speed_scenarios
    )
    best = None
    best_gain = 0.0

    for j, target in enumerate(routes):
        if j == idx:
            continue
        candidate_target, candidate_cost = best_insert_group(
            target["stops"], moved_group, target["vehicle"], dist_matrix, time_windows, speed_scenarios
        )
        if candidate_target is None:
            continue

        old_cost = route["cost"]["total"] + target["cost"]["total"]
        new_cost = remaining_cost["total"] + candidate_cost["total"]
        gain = old_cost - new_cost
        if gain > best_gain + 1e-6:
            best_gain = gain
            best = (j, remaining_stops, candidate_target, remaining_cost, candidate_cost)

    if best is None:
        return False

    j, remaining_stops, candidate_target, remaining_cost, candidate_cost = best
    routes[idx]["stops"] = remaining_stops
    routes[idx]["cost"] = remaining_cost
    routes[j]["stops"] = candidate_target
    routes[j]["cost"] = candidate_cost
    routes[idx] = improve_route_sequence(routes[idx], dist_matrix, time_windows, speed_scenarios)
    routes[j] = improve_route_sequence(routes[j], dist_matrix, time_windows, speed_scenarios)
    return True

def improve_bad_routes(routes, dist_matrix, time_windows, speed_scenarios):
    routes = [dict(route) for route in routes]
    total_before = sum(route["cost"]["total"] for route in routes)

    def ranked_bad_indices():
        return sorted(
            range(len(routes)),
            key=lambda ridx: (
                routes[ridx]["cost"]["overtime_hours"],
                routes[ridx]["cost"]["late_hours"],
                len(routes[ridx]["stops"]),
                routes[ridx]["cost"]["total"],
            ),
            reverse=True,
        )

    target_indices = ranked_bad_indices()[: min(8, len(routes))]

```

```

# 1) 长坏路线先做小规模退火/禁忌式路线内重排。
for idx in target_indices:
    if len(routes[idx]["stops"]) >= 6 or routes[idx]["cost"]["late_hours"] > 1.0:
        routes[idx] = anneal_bad_route_sequence(routes[idx], dist_matrix, time_windows, speed_scenarios)

# 2) 对最差路线做“尾部半段/2-3 客户组”迁移。
for _ in range(2):
    changed = False
    for idx in ranked_bad_indices()[0: min(6, len(routes))]:
        route = routes[idx]
        if len(route["stops"]) <= 3 and route["cost"]["late_hours"] <= 0.1:
            continue

        groups = bad_route_groups(route, time_windows)
        # 越大的组优先, 体现“二分拆开/成组迁移”的策略。
        groups.sort(key=lambda g: (len(g), g[0]), reverse=True)
        for group_indices in groups:
            if migrate_group_from_route(routes, idx, group_indices, dist_matrix, time_windows, speed_scenarios):
                changed = True
                break
        if changed:
            break
    if not changed:
        break

# 3) 单客户迁移作为兜底, 处理组迁移后剩余的局部坏点。
for idx in ranked_bad_indices()[0: min(8, len(routes))]:
    route = routes[idx]
    if len(route["stops"]) <= 3 and route["cost"]["late_hours"] <= 0.1:
        continue

    changed = True
    while changed and len(route["stops"]) > 1:
        changed = False
        stop_order = sorted(
            range(len(route["stops"])),
            key=lambda sidx: (
                time_windows.get(route["stops"][sidx]["customer"], {"late": 24.0})["late"],
                route["stops"][sidx]["weight"],
            ),
            reverse=True,
        )

        for sidx in stop_order:
            moved_stop = route["stops"][sidx]
            remaining_stops = normalize_route_stops(route["stops"][:sidx] + route["stops"][sidx + 1 :])
            if not remaining_stops:
                continue
            if not route_is_feasible_for_vehicle(remaining_stops, route["vehicle"]):
                continue

            current_pair_cost = route["cost"]["total"]
            best_target = None
            best_gain = 0.0

            for j, target in enumerate(routes):
                if j == idx:
                    continue
                candidate_target = normalize_route_stops(target["stops"] + [moved_stop])
                if not route_is_feasible_for_vehicle(candidate_target, target["vehicle"]):
                    continue

                new_route_cost = evaluate_route(
                    remaining_stops, route["vehicle"], dist_matrix, time_windows, speed_scenarios
                )
                new_target_cost = evaluate_route(
                    candidate_target, target["vehicle"], dist_matrix, time_windows, speed_scenarios
                )

```

```

        gain = (route["cost"]["total"] + target["cost"]["total"]) - (
            new_route_cost["total"] + new_target_cost["total"]
        )
        if gain > best_gain + 1e-6:
            best_gain = gain
            best_target = (j, remaining_stops, candidate_target, new_route_cost, new_target_cost)

    if best_target is not None:
        j, remaining_stops, candidate_target, new_route_cost, new_target_cost = best_target
        route["stops"] = remaining_stops
        route["cost"] = new_route_cost
        routes[j]["stops"] = candidate_target
        routes[j]["cost"] = new_target_cost
        route = improve_route_sequence(route, dist_matrix, time_windows, speed_scenarios)
        routes[j] = improve_route_sequence(routes[j], dist_matrix, time_windows, speed_scenarios)
        routes[idx] = route
        changed = True
        break

    total_after = sum(route["cost"]["total"] for route in routes)
    return routes, total_before, total_after

def route_fragment_penalty(route):
    tiny_weight = sum(stop["weight"] for stop in route["stops"] if stop["weight"] < 200.0)
    tiny_count = sum(1 for stop in route["stops"] if stop["weight"] < 200.0)
    return (
        route["cost"]["time_window"] * 2.5
        + route["cost"]["energy"]
        + route["cost"]["carbon"]
        + 0.8 * route["cost"]["total"]
        + 0.6 * tiny_weight
        + 80.0 * tiny_count
    )

def collect_route_demands(routes):
    demand = defaultdict(lambda: {"weight": 0.0, "volume": 0.0})
    for route in routes:
        for stop in route["stops"]:
            demand[stop["customer"]]["weight"] += stop["weight"]
            demand[stop["customer"]]["volume"] += stop["volume"]
    rows = []
    for customer, item in demand.items():
        rows.append({"目标客户编号": customer, "重量": item["weight"], "体积": item["volume"]})
    return pd.DataFrame(rows)

def vehicle_limit_from_routes(routes):
    used = defaultdict(int)
    for route in routes:
        used[route["vehicle"]] += 1
    return used

def build_routes_for_demands_with_limits(customer_demands, vehicle_limits, dist_matrix, time_windows, fleet_order):
    if customer_demands.empty:
        return []
    old_order = FLEET_ORDER[:]
    old_counts = {name: spec["count"] for name, spec in VEHICLES.items()}
    try:
        FLEET_ORDER[:] = fleet_order[:]
        for name, spec in VEHICLES.items():
            spec["count"] = int(vehicle_limits.get(name, 0))
        routes = build_capacity_focused_routes(customer_demands, dist_matrix, time_windows)
    finally:
        FLEET_ORDER[:] = old_order
        for name, old_count in old_counts.items():
            VEHICLES[name]["count"] = old_count

```



```

return routes

def reoptimize_fragmented_routes(routes, dist_matrix, time_windows, speed_scenarios, iterations=4, group_size=8):
    routes = [dict(route) for route in routes]
    total_before = sum(route["cost"]["total"] for route in routes)

    for _ in range(iterations):
        routes.sort(key=route_fragment_penalty, reverse=True)
        candidate_idx = list(range(min(group_size, len(routes))))
        selected = [routes[idx] for idx in candidate_idx]
        other_routes = [routes[idx] for idx in range(len(routes)) if idx not in candidate_idx]
        if len(selected) <= 1:
            break

        local_demands = collect_route_demands(selected)
        vehicle_limits = vehicle_limit_from_routes(selected)
        local_order = [name for name in FLEET_ORDER if vehicle_limits.get(name, 0) > 0]
        if not local_order:
            continue

        try:
            rebuilt = build_routes_for_demands_with_limits(
                local_demands,
                vehicle_limits,
                dist_matrix,
                time_windows,
                local_order,
            )
            rebuilt = assign_vehicle_types_with_limits(
                rebuilt,
                vehicle_limits,
                dist_matrix,
                time_windows,
                speed_scenarios,
            )
            rebuilt, _, _ = improve_all_routes(rebuilt, dist_matrix, time_windows, speed_scenarios)
            rebuilt, _, _ = improve_cross_route_swaps(rebuilt, dist_matrix, time_windows, speed_scenarios)
            rebuilt, _, _ = improve_bad_routes(rebuilt, dist_matrix, time_windows, speed_scenarios)
        except Exception:
            continue

        old_cost = sum(route["cost"]["total"] for route in selected)
        new_cost = sum(route["cost"]["total"] for route in rebuilt)
        if new_cost + 1e-6 < old_cost:
            routes = other_routes + rebuilt
        else:
            routes = other_routes + selected

    routes = renumber_routes(routes)
    total_after = sum(route["cost"]["total"] for route in routes)
    return routes, total_before, total_after

def rebalance_customer_fragments(routes, dist_matrix, time_windows, speed_scenarios, tiny_threshold=250.0):
    routes = [dict(route) for route in routes]
    total_before = sum(route["cost"]["total"] for route in routes)

    customer_map = defaultdict(list)
    for ridx, route in enumerate(routes):
        for sidx, stop in enumerate(route["stops"]):
            customer_map[stop["customer"]].append((ridx, sidx))

    improved = True
    while improved:
        improved = False
        customers = sorted(
            customer_map.keys(),
            key=lambda cid: (

```

```

        sum(routes[ridx]["stops"][sidx]["weight"] < tiny_threshold for ridx, sidx in customer_map[cid]),
        len(customer_map[cid]),
    ),
    reverse=True,
)

for customer in customers:
    refs = customer_map[customer]
    if len(refs) <= 1:
        continue
    refs = sorted(refs, key=lambda item: routes[item[0]]["stops"][item[1]]["weight"])
    source_rix, source_sidx = refs[0]
    moved_stop = routes[source_rix]["stops"][source_sidx]
    if moved_stop["weight"] >= tiny_threshold:
        continue

    best_move = None
    best_gain = 0.0
    for target_rix, target_sidx in refs[1:]:
        if target_rix == source_rix:
            continue
        source_route = routes[source_rix]
        target_route = routes[target_rix]
        target_stop = target_route["stops"][target_sidx]

        candidate_target_stop = {
            "customer": customer,
            "weight": target_stop["weight"] + moved_stop["weight"],
            "volume": target_stop["volume"] + moved_stop["volume"],
            "task_ids": target_stop.get("task_ids", []) + moved_stop.get("task_ids", []),
        }
        candidate_target_stops = target_route["stops"][:]
        candidate_target_stops[target_sidx] = candidate_target_stop
        candidate_target_stops = normalize_route_stops(candidate_target_stops)

        candidate_source_stops = normalize_route_stops(
            source_route["stops"][:source_sidx] + source_route["stops"][source_sidx + 1 :]
        )
        if not candidate_source_stops:
            continue
        if not route_is_feasible_for_vehicle(candidate_source_stops, source_route["vehicle"]):
            continue
        if not route_is_feasible_for_vehicle(candidate_target_stops, target_route["vehicle"]):
            continue

        new_source_cost = evaluate_route(
            candidate_source_stops, source_route["vehicle"], dist_matrix, time_windows, speed_scenarios
        )
        new_target_cost = evaluate_route(
            candidate_target_stops, target_route["vehicle"], dist_matrix, time_windows, speed_scenarios
        )
        gain = (source_route["cost"]["total"] + target_route["cost"]["total"]) - (
            new_source_cost["total"] + new_target_cost["total"]
        )
        if gain > best_gain + 1e-6:
            best_gain = gain
            best_move = (
                source_rix,
                target_rix,
                candidate_source_stops,
                candidate_target_stops,
                new_source_cost,
                new_target_cost,
            )

    if best_move is not None:
        source_rix, target_rix, source_stops, target_stops, source_cost, target_cost = best_move
        routes[source_rix]["stops"] = source_stops
        routes[source_rix]["cost"] = source_cost

```

```

        routes[target_idx]["stops"] = target_stops
        routes[target_idx]["cost"] = target_cost
        routes[source_idx] = improve_route_sequence(routes[source_idx], dist_matrix, time_windows, speed_scenarios)
        routes[target_idx] = improve_route_sequence(routes[target_idx], dist_matrix, time_windows, speed_scenarios)

        customer_map = defaultdict(list)
        for ridx, route in enumerate(routes):
            for sidx, stop in enumerate(route["stops"]):
                customer_map[stop["customer"]].append((ridx, sidx))
        improved = True
        break
    if improved:
        continue

    routes = [route for route in routes if route["stops"]]
    routes = renumber_routes(routes)
    total_after = sum(route["cost"]["total"] for route in routes)
    return routes, total_before, total_after

def setup_chinese_font():
    candidates = [
        "Microsoft YaHei",
        "SimHei",
        "SimSun",
        "Noto Sans CJK SC",
        "Source Han Sans SC",
    ]
    available = {font.name for font in font_manager.fontManager.ttflist}
    for name in candidates:
        if name in available:
            plt.rcParams["font.sans-serif"] = [name]
            break
    plt.rcParams["axes.unicode_minus"] = False

def plot_routes(routes, coords, out_path):
    setup_chinese_font()
    fig, ax = plt.subplots(figsize=(14, 12), dpi=180)
    ax.set_facecolor("#f7f3ea")
    fig.patch.set_facecolor("#f7f3ea")

    center = coords[0]
    green_circle = plt.Circle((0, 0), 10, color="#64a96b", alpha=0.12, lw=2, fill=True)
    green_border = plt.Circle((0, 0), 10, color="#3d7f45", alpha=0.8, lw=1.5, fill=False)
    ax.add_patch(green_circle)
    ax.add_patch(green_border)
    ax.text(0, 10.8, " 绿色配送区", ha="center", va="bottom", color="#2e6f39", fontsize=11)

    customers = [cid for cid in coords if cid != 0]
    xs = [coords[cid]["x"] for cid in customers]
    ys = [coords[cid]["y"] for cid in customers]
    in_green = [(x**2 + y**2) ** 0.5 <= 10 for x, y in zip(xs, ys)]
    ax.scatter(
        [x for x, g in zip(xs, in_green) if not g],
        [y for y, g in zip(ys, in_green) if not g],
        s=22,
        c="#44546a",
        alpha=0.75,
        label=" 普通客户",
        zorder=3,
    )
    ax.scatter(
        [x for x, g in zip(xs, in_green) if g],
        [y for y, g in zip(ys, in_green) if g],
        s=26,
        c="#2f855a",
        alpha=0.9,
        label=" 绿色区客户",
    )

```

```

        zorder=4,
    )
    ax.scatter(center["x"], center["y"], s=180, marker="*", c="#c0392b", label=" 配送中心", zorder=6)
    ax.text(center["x"] + 0.8, center["y"] + 0.8, " 配送中心", color="#8b1e16", fontsize=11, weight="bold")

    colors = {
        " 新能源 1": "#0f766e",
        " 新能源 2": "#14b8a6",
        " 燃油车 1": "#c2410c",
        " 燃油车 2": "#f97316",
        " 燃油车 3": "#a16207",
    }

    for route in routes:
        if not route["stops"]:
            continue
        vehicle = route["vehicle"]
        path_ids = [0] + [stop["customer"] for stop in route["stops"]] + [0]
        path_x = [coords[cid]["x"] for cid in path_ids]
        path_y = [coords[cid]["y"] for cid in path_ids]
        alpha = 0.32 if len(route["stops"]) <= 2 else 0.62
        lw = 0.8 if len(route["stops"]) <= 2 else 1.4
        ax.plot(path_x, path_y, color=colors.get(vehicle, "#555555"), lw=lw, alpha=alpha, zorder=2)

    for cid in customers:
        if cid % 5 == 0:
            ax.text(coords[cid]["x"] + 0.25, coords[cid]["y"] + 0.25, str(cid), fontsize=6, color="#333333")

    ax.set_xlabel("X 坐标 / km")
    ax.set_ylabel("Y 坐标 / km")
    ax.grid(True, color="#d7cfbd", linestyle="--", linewidth=0.6, alpha=0.8)
    ax.legend(loc="upper right", frameon=True, facecolor="white", edgecolor="#c8c0ae")
    ax.set_aspect("equal", adjustable="box")
    fig.tight_layout()
    fig.savefig(out_path, bbox_inches="tight")
    plt.close(fig)

def plot_vehicle_usage(type_df, out_path):
    setup_chinese_font()
    fig, ax = plt.subplots(figsize=(10, 6), dpi=180)
    fig.patch.set_facecolor("white")
    ax.set_facecolor("white")
    x = np.arange(len(type_df))
    used = type_df[" 使用数量"].to_numpy()
    available = type_df[" 可用数量"].to_numpy()
    labels = type_df[" 车辆类型"].tolist()
    ax.bar(x, available, color="#d6d3d1", label=" 可用数量", width=0.62)
    ax.bar(x, used, color=["#0f766e", "#14b8a6", "#c2410c", "#f97316", "#a16207"], label=" 使用数量", width=0.62)
    for idx, value in enumerate(used):
        ax.text(idx, value + 1, str(int(value)), ha="center", va="bottom", fontsize=10)
    ax.set_xticks(x)
    ax.set_xticklabels(labels)
    ax.set_ylabel(" 车辆数")
    ax.legend()
    ax.grid(axis="y", linestyle="--", alpha=0.35)
    fig.tight_layout()
    fig.savefig(out_path, bbox_inches="tight")
    plt.close(fig)

def plot_cost_breakdown(cost_row, out_path):
    setup_chinese_font()
    labels = [" 启动成本", " 能源成本", " 碳排放成本", " 时间窗成本"]
    values = [cost_row[label] for label in labels]
    colors = ["#334155", "#0f766e", "#65a30d", "#ea580c"]
    fig, ax = plt.subplots(figsize=(9, 6), dpi=180)
    fig.patch.set_facecolor("white")
    ax.set_facecolor("white")
    wedges, texts, autotexts = ax.pie(

```

```

        values,
        labels=labels,
        colors=colors,
        startangle=120,
        autopct=lambda pct: f"{pct:.1f}%",
        pctdistance=0.78,
        textprops={"fontsize": 10},
        wedgeprops={"linewidth": 1.2, "edgecolor": "white"},
    )

ax.set_title(f" 问题一: 成本构成 (总成本 {cost_row['总成本']:.2f} 元)", fontsize=16, weight="bold")
fig.tight_layout()
fig.savefig(out_path, bbox_inches="tight")
plt.close(fig)

# =====
# 5. 输出结果
# =====
def write_results(routes, customer_demands, dist_matrix, time_windows, speed_scenarios, coords, fleet_order):
    summary_rows = []
    detail_rows = []
    type_rows = []
    total = defaultdict(float)

    for route in routes:
        detail_cost = evaluate_route(
            route["stops"], route["vehicle"], dist_matrix, time_windows, speed_scenarios, with_detail=True
        )
        route["cost"] = detail_cost
        load_w, load_v = route_load(route["stops"])
        distance = route_distance(route["stops"], dist_matrix)
        path = "0 -> " + " -> ".join(str(stop["customer"]) for stop in route["stops"]) + " -> 0"
        arrival = ", ".join(hour_to_time(row["arrival"]) for row in detail_cost["details"])

        summary_rows.append(
            {
                "路线编号": route["route_id"],
                "车辆编号": route["vehicle_no"],
                "车辆类型": route["vehicle"],
                "发车时间": hour_to_time(detail_cost["depart"]),
                "返场时间": hour_to_time(detail_cost["return"]),
                "载重 (kg)": round(load_w, 4),
                "体积 (m3)": round(load_v, 4),
                "载重率": round(load_w / VEHICLES[route["vehicle"]]["cap_w"], 4),
                "容积率": round(load_v / VEHICLES[route["vehicle"]]["cap_v"], 4),
                "行驶距离 (km)": round(distance, 4),
                "访问客户数": len(route["stops"]),
                "路径": path,
                "平均到达时间": arrival,
                "启动成本": round(detail_cost["start"], 2),
                "能源成本": round(detail_cost["energy"], 2),
                "碳排放成本": round(detail_cost["carbon"], 2),
                "碳排放量 (kg)": round(detail_cost["carbon"] / CARBON_PRICE, 4),
                "时间窗成本": round(detail_cost["time_window"], 2),
                "超时返场 (h)": round(detail_cost["overtime_hours"], 4),
                "超时返场成本": round(detail_cost["overtime_cost"], 2),
                "等待总时长 (h)": round(detail_cost["wait_hours"], 4),
                "迟到总时长 (h)": round(detail_cost["late_hours"], 4),
                "总成本": round(detail_cost["total"], 2),
            }
        )

    for key in ["start", "energy", "carbon", "time_window", "overtime_cost", "total"]:
        total[key] += detail_cost[key]

    for stop, detail in zip(route["stops"], detail_cost["details"]):
        window = time_windows.get(stop["customer"], {"early": START_HOUR, "late": 24.0})
        detail_rows.append(
            {

```

```

        " 路线编号": route["route_id"],
        " 车辆编号": route["vehicle_no"],
        " 车辆类型": route["vehicle"],
        " 访问顺序": detail["seq"],
        " 客户编号": stop["customer"],
        " 配送重量 (kg)": round(stop["weight"], 4),
        " 配送体积 (m3)": round(stop["volume"], 4),
        " 时间窗开始": hour_to_time(window["early"]),
        " 时间窗结束": hour_to_time(window["late"]),
        " 平均到达": hour_to_time(detail["arrival"]),
        " 平均服务开始": hour_to_time(detail["service_start"]),
        " 平均离开": hour_to_time(detail["departure"]),
        " 平均等待 (h)": round(detail["wait_hours"], 4),
        " 平均迟到 (h)": round(detail["late_hours"], 4),
        " 上一点到本点距离 (km)": round(detail["travel_distance"], 4),
        " 包含拆分任务": ", ".join(stop["task_ids"]),
    }
)

used = defaultdict(int)
for route in routes:
    used[route["vehicle"]] += 1
for name, spec in VEHICLES.items():
    type_rows.append(
        {
            " 车辆类型": name,
            " 动力类型": " 新能源" if spec["power"] == "ev" else " 燃油",
            " 可用数量": spec["count"],
            " 使用数量": used[name],
            " 剩余数量": spec["count"] - used[name],
            " 载重上限 (kg)": spec["cap_w"],
            " 容积上限 (m3)": spec["cap_v"],
        }
    )

customer_check = customer_demands.rename(
    columns={" 目标客户编号": " 客户编号", " 重量": " 总重量 (kg)", " 体积": " 总体积 (m3)" }
)

speed_rows = []
for scenario in speed_scenarios:
    for period in scenario["periods"]:
        speed_rows.append(
            {
                " 场景": scenario["scenario_id"],
                " 时段": f"{hour_to_time(period['start'])}--{hour_to_time(period['end'])}",
                " 交通状态": period["label"],
                " 抽样速度 (km/h)": round(period["speed"], 4),
            }
        )

total_distance = float(sum(route_distance(route["stops"], dist_matrix) for route in routes))
total_customers = int(sum(len(route["stops"]) for route in routes))
total_wait = float(sum(route["cost"]["wait_hours"] for route in routes))
total_late = float(sum(route["cost"]["late_hours"] for route in routes))
type_df = pd.DataFrame(type_rows)
ev_used = int(type_df[type_df[" 动力类型"] == " 新能源"][" 使用数量"].sum())
fuel_used = int(type_df[type_df[" 动力类型"] == " 燃油"][" 使用数量"].sum())

overview_rows = [
    {" 指标": " 问题", " 数值": " 问题一: 静态环境下的车辆调度"},
    {" 指标": " 使用车辆数", " 数值": len(routes)},
    {" 指标": " 新能源车使用数", " 数值": ev_used},
    {" 指标": " 燃油车使用数", " 数值": fuel_used},
    {" 指标": " 客户数", " 数值": len(customer_demands)},
    {" 指标": " 总访问客户点", " 数值": total_customers},
    {" 指标": " 总行驶距离 (km)", " 数值": round(total_distance, 2)},
    {" 指标": " 总等待时长 (h)", " 数值": round(total_wait, 4)},
    {" 指标": " 总迟到时长 (h)", " 数值": round(total_late, 4)},

```

```

        {"指标": "启动成本", "数值": round(total["start"], 2)},
        {"指标": "能源成本", "数值": round(total["energy"], 2)},
        {"指标": "碳排放成本", "数值": round(total["carbon"], 2)},
        {"指标": "碳排放总量 (kg)", "数值": round(total["carbon"] / CARBON_PRICE, 4)},
        {"指标": "时间窗成本", "数值": round(total["time_window"], 2)},
        {"指标": "超时返场成本", "数值": round(total["overtime_cost"], 2)},
        {"指标": "总成本", "数值": round(total["total"], 2)},
        {"指标": "速度场景数", "数值": NUM_SPEED_SCENARIOS},
        {"指标": "随机种子", "数值": RANDOM_SEED},
        {"指标": "自动选中车队展开顺序", "数值": " -> ".join(fleet_order)},
    ]
    for _, row in type_df.iterrows():
        overview_rows.append(
            {"指标": f"{row['车辆类型']} 使用/可用", "数值": f"{int(row['使用数量'])}/{int(row['可用数量'])}"
            }
        )
    cost_total = pd.DataFrame(overview_rows)

    out_path = os.path.join(PROC_DIR, "1-问题一最优方案.xlsx")
    route_png = os.path.join(PROC_DIR, "1-问题一车辆行驶路径图.png")
    vehicle_png = os.path.join(PROC_DIR, "1-问题一车辆使用方案图.png")
    cost_png = os.path.join(PROC_DIR, "1-问题一成本构成图.png")

    cost_row = {
        "使用车辆数": len(routes),
        "新能源车使用数": ev_used,
        "燃油车使用数": fuel_used,
        "客户数": len(customer_demands),
        "总访问客户点": total_customers,
        "总行驶距离 (km)": round(total_distance, 2),
        "总等待时长 (h)": round(total_wait, 4),
        "总迟到时长 (h)": round(total_late, 4),
        "启动成本": round(total["start"], 2),
        "能源成本": round(total["energy"], 2),
        "碳排放成本": round(total["carbon"], 2),
        "碳排放总量 (kg)": round(total["carbon"] / CARBON_PRICE, 4),
        "时间窗成本": round(total["time_window"], 2),
        "超时返场成本": round(total["overtime_cost"], 2),
        "总成本": round(total["total"], 2),
        "速度场景数": NUM_SPEED_SCENARIOS,
        "随机种子": RANDOM_SEED,
    }

    with pd.ExcelWriter(out_path, engine="openpyxl") as writer:
        cost_total.to_excel(writer, sheet_name=" 方案总览", index=False)
        type_df.to_excel(writer, sheet_name=" 车辆使用统计", index=False)
        pd.DataFrame(summary_rows).to_excel(writer, sheet_name=" 车辆路径", index=False)
        pd.DataFrame(detail_rows).to_excel(writer, sheet_name=" 到达时间明细", index=False)
        customer_check.to_excel(writer, sheet_name=" 客户需求汇总", index=False)
        pd.DataFrame(speed_rows).to_excel(writer, sheet_name=" 速度抽样场景", index=False)

    plot_routes(routes, coords, route_png)
    plot_vehicle_usage(type_df, vehicle_png)
    plot_cost_breakdown(cost_row, cost_png)

    return out_path, cost_row, type_df, cost_total

def relative_proc_path(path):
    try:
        return os.path.relpath(path, os.path.dirname(os.path.abspath(__file__)))
    except ValueError:
        return path

def print_terminal_summary_q1(total_row, type_df, out_path):
    total_cost = float(total_row["总成本"])
    start_cost = float(total_row["启动成本"])
    energy_cost = float(total_row["能源成本"])
    carbon_cost = float(total_row["碳排放成本"])
    carbon_kg = float(total_row["碳排放总量 (kg)"])

```

```

time_window_cost = float(total_row[" 时间窗成本"])
total_distance = float(total_row[" 总行驶距离 (km)"])
used_routes = int(total_row[" 使用车辆数"])
ev_used = int(total_row[" 新能源车使用数"])
fuel_used = int(total_row[" 燃油车使用数"])
total_wait = float(total_row[" 总等待时长 (h)"])
total_late = float(total_row[" 总迟到时长 (h)"])

print("\n" + "=" * 72)
print(" 问题一求解完成")
print("=" * 72)
print(" 规模")
print(f" 使用车辆数: {used_routes}")
print(f" 新能源车: {ev_used}")
print(f" 燃油车: {fuel_used}")
print(f" 总访问客户点: {int(total_row['总访问客户点'])}")
print(f" 总行驶距离: {total_distance:.2f} km")
print(f" 总等待时长: {total_wait:.4f} h")
print(f" 总迟到时长: {total_late:.4f} h")
print(" 成本")
print(f" 总成本: {total_cost:.2f}")
print(f" 成本构成: 启动 {start_cost:.2f} | 能源 {energy_cost:.2f} | 碳排放 {carbon_cost:.2f} | 时间窗
↪ {time_window_cost:.2f}")
print(f" 碳排放总量: {carbon_kg:.4f} kg")
print(" 车辆")
for _, row in type_df.iterrows():
    print(
        f" {row['车辆类型']}: 已用 {int(row['使用数量'])} / 总量 {int(row['可用数量'])} / 剩余 {int(row['剩余数量'])}"
    )
print(" 运行结果")
print(f" 结果文件: {relative_proc_path(out_path)}")
print(f" 路径图: {relative_proc_path(os.path.join(PROC_DIR, '1-问题一车辆行驶路径图.png'))}")
print(f" 车辆方案图: {relative_proc_path(os.path.join(PROC_DIR, '1-问题一车辆使用方案图.png'))}")
print(f" 成本构成图: {relative_proc_path(os.path.join(PROC_DIR, '1-问题一成本构成图.png'))}")
print("=" * 72)

def main():
    print(" 读取附件与预处理订单数据...")
    dist_matrix = read_distance_matrix()
    time_windows = read_time_windows()
    coords = read_coordinates()
    _, customer_demands = build_delivery_tasks()
    speed_scenarios = generate_speed_scenarios()

    print(f" 客户数: {len(customer_demands)}")
    print(" 自动筛选较优车队展开顺序...")
    best_order, best_used, best_order_cost = select_best_fleet_order(customer_demands, dist_matrix, time_windows,
↪ speed_scenarios)
    print(f" 选中顺序: {' ' -> '.join(best_order)} | 初始车辆数: {best_used} | 初始成本: {best_order_cost:.2f}")
    print(" 按车型容量动态拆分需求, 构造满足车辆数量硬约束的低车辆数方案...")
    routes = build_capacity_focused_routes(customer_demands, dist_matrix, time_windows)

    print(" 在车辆数量硬约束下分配车型并计算正态速度场景平均成本...")
    routes = assign_vehicle_types(routes, dist_matrix, time_windows, speed_scenarios)

    print(" 固定车辆数, 优化各线路访问顺序与发车时间...")
    routes, before_cost, after_cost = improve_all_routes(routes, dist_matrix, time_windows, speed_scenarios)
    print(f" 局部优化前成本: {before_cost:.2f}, 优化后成本: {after_cost:.2f}")

    print(" 尝试跨线路客户交换, 继续压缩时间窗与能源成本...")
    routes, cross_before, cross_after = improve_cross_route_swaps(routes, dist_matrix, time_windows, speed_scenarios)
    print(f" 跨线路优化前成本: {cross_before:.2f}, 优化后成本: {cross_after:.2f}")

    print(" 定向拆解重组长坏路线, 继续压缩尾部高迟到成本...")
    routes, bad_before, bad_after = improve_bad_routes(routes, dist_matrix, time_windows, speed_scenarios)
    print(f" 坏路线重组前成本: {bad_before:.2f}, 优化后成本: {bad_after:.2f}")

    print(" 针对同客户极小尾货做跨路线重平衡...")

```



```

    routes, frag_before, frag_after = rebalance_customer_fragments(
        routes, dist_matrix, time_windows, speed_scenarios
    )
    print(f" 尾货重平衡前成本: {frag_before:.2f}, 优化后成本: {frag_after:.2f}")

    out_path, total_row, type_df, overview_df = write_results(
        routes, customer_demands, dist_matrix, time_windows, speed_scenarios, coords, best_order
    )
    print_terminal_summary_q1(total_row, type_df, out_path)

if __name__ == "__main__":
    main()

```

B.3 问题二求解程序：2-问题二.py

```

import importlib.util
import math
import os
from collections import defaultdict
from collections import deque

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# =====
# 1. 复用问题一基础模型
# =====
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
PROC_DIR = os.path.join(BASE_DIR, " 过程")
os.makedirs(PROC_DIR, exist_ok=True)

Q1_PATH = os.path.join(BASE_DIR, "1-问题一.py")
spec = importlib.util.spec_from_file_location("q1", Q1_PATH)
q1 = importlib.util.module_from_spec(spec)
spec.loader.exec_module(q1)

GREEN_RADIUS = 10.0
FUEL_BAN_START = 8.0
FUEL_BAN_END = 16.0
POLICY_VIOLATION_PENALTY = 1e8
Q2_FLEET_ORDER = [" 新能源 1", " 新能源 2", " 燃油车 1", " 燃油车 2", " 燃油车 3"]
GREEN_PRIORITY_BONUS = 2500.0
CARBON_PRIORITY_FACTOR = 12.0
CARBON_SEARCH_LAMBDAS = [0, 5, 10, 20, 30, 40, 50, 60, 80, 100, 120]

# =====
# 2. 政策约束与问题二评估
# =====
def green_customers(coords):
    return {
        cid
        for cid, item in coords.items()
        if cid != 0 and (item["x"] ** 2 + item["y"] ** 2) ** 0.5 <= GREEN_RADIUS + 1e-9
    }

def is_policy_feasible(stops, vehicle_name, coords, time_windows, speed_scenarios, depart_hour):
    if q1.VEHICLES[vehicle_name]["power"] != "fuel":
        return True

    green_set = green_customers(coords)
    if not any(stop["customer"] in green_set for stop in stops):
        return True

    for scenario in speed_scenarios:

```

```

        current_time = depart_hour
        current_loc = 0
        for stop in stops:
            customer = stop["customer"]
            arrive_time, _ = q1.travel_segments(
                q1.dist_matrix_global[current_loc][customer], current_time, scenario["periods"]
            )
            if customer in green_set and FUEL_BAN_START <= arrive_time < FUEL_BAN_END:
                return False
            window = time_windows.get(customer, {"early": q1.START_HOUR, "late": 24.0})
            current_time = max(arrive_time, window["early"]) + q1.SERVICE_TIME
            current_loc = customer
        return True

def evaluate_route_policy(stops, vehicle_name, dist_matrix, time_windows, speed_scenarios, coords, with_detail=False):
    if not stops:
        return q1.evaluate_route(stops, vehicle_name, dist_matrix, time_windows, speed_scenarios, with_detail)

    vehicle = q1.VEHICLES[vehicle_name]
    load_w, load_v = q1.route_load(stops)
    if load_w > vehicle["cap_w"] + 1e-8 or load_v > vehicle["cap_v"] + 1e-8:
        return None

    candidate_departures = set()
    lower, upper = q1.estimate_departure_bounds(stops, dist_matrix, time_windows)
    current = math.floor(lower / q1.DEPARTURE_STEP) * q1.DEPARTURE_STEP
    while current <= upper + 1e-9:
        candidate_departures.add(round(max(q1.START_HOUR, current), 6))
        current += q1.DEPARTURE_STEP
    candidate_departures.add(q1.START_HOUR)

    if vehicle["power"] == "fuel":
        green_set = green_customers(coords)
        if any(stop["customer"] in green_set for stop in stops):
            # 允许燃油车晚出发, 在 16:00 后进入绿色配送区。
            current = FUEL_BAN_END
            while current <= 22.0 + 1e-9:
                candidate_departures.add(round(current, 6))
                current += q1.DEPARTURE_STEP

    best = None
    best_depart = None
    for depart in sorted(candidate_departures):
        if not is_policy_feasible(stops, vehicle_name, coords, time_windows, speed_scenarios, depart):
            continue
        result = q1.evaluate_route(stops, vehicle_name, dist_matrix, time_windows, speed_scenarios, with_detail, depart)
        if result is None:
            continue
        if best is None or result["total"] < best["total"]:
            best = result
            best_depart = depart

    if best is None:
        return None
    best["depart"] = best_depart
    return best

def route_has_green_customer(stops, coords):
    green_set = green_customers(coords)
    return any(stop["customer"] in green_set for stop in stops)

def feasible_types_policy(stops, coords):
    load_w, load_v = q1.route_load(stops)
    choices = []
    for name, spec in q1.VEHICLES.items():
        if load_w <= spec["cap_w"] + 1e-8 and load_v <= spec["cap_v"] + 1e-8:

```

```

        choices.append(name)
    return choices

def can_fuel_visit_customer(customer, current_time, current_loc, dist_matrix, coords, time_windows, speed_scenarios):
    if customer not in green_customers(coords):
        return True
    # 对问题二建路阶段使用均值速度进行可行性过滤，避免燃油车在限行时段选绿区点。
    mean_periods = q1.mean_speed_periods()
    arrive, _ = q1.travel_segments(dist_matrix[current_loc][customer], current_time, mean_periods)
    return arrive >= FUEL_BAN_END

def policy_score(cost_result, vehicle_name, has_green_customer):
    """ 问题二使用“总成本 + 碳排放偏好”的综合打分，保证限行前提下优先减排。"""
    if cost_result is None:
        return float("inf")
    score = float(cost_result["total"]) + CARBON_PRIORITY_FACTOR * float(cost_result["carbon"])
    if has_green_customer and q1.VEHICLES[vehicle_name]["power"] == "ev":
        score -= GREEN_PRIORITY_BONUS
    return score

class FlowEdge:
    __slots__ = ("to", "rev", "cap", "cost")

    def __init__(self, to, rev, cap, cost):
        self.to = to
        self.rev = rev
        self.cap = cap
        self.cost = cost

def add_flow_edge(graph, u, v, cap, cost):
    graph[u].append(FlowEdge(v, len(graph[v]), cap, cost))
    graph[v].append(FlowEdge(u, len(graph[u]) - 1, 0, -cost))

def min_cost_max_flow(graph, source, sink, target_flow):
    n = len(graph)
    flow = 0
    total_cost = 0
    inf = 10**18

    while flow < target_flow:
        dist = [inf] * n
        in_queue = [False] * n
        prev_node = [-1] * n
        prev_edge = [-1] * n
        dist[source] = 0
        queue = deque([source])
        in_queue[source] = True

        while queue:
            u = queue.popleft()
            in_queue[u] = False
            for edge_idx, edge in enumerate(graph[u]):
                if edge.cap <= 0:
                    continue
                new_cost = dist[u] + edge.cost
                if new_cost >= dist[edge.to]:
                    continue
                dist[edge.to] = new_cost
                prev_node[edge.to] = u
                prev_edge[edge.to] = edge_idx
                if not in_queue[edge.to]:
                    queue.append(edge.to)
                in_queue[edge.to] = True

    return flow, total_cost

```

```

        if dist[sink] >= inf:
            break

        add_flow = target_flow - flow
        v = sink
        while v != source:
            edge = graph[prev_node[v]][prev_edge[v]]
            add_flow = min(add_flow, edge.cap)
            v = prev_node[v]

        v = sink
        while v != source:
            edge = graph[prev_node[v]][prev_edge[v]]
            edge.cap -= add_flow
            graph[v][edge.rev].cap += add_flow
            v = prev_node[v]

        flow += add_flow
        total_cost += add_flow * dist[sink]

    return flow, total_cost

def split_demands_by_policy(customer_demands, coords):
    green_set = green_customers(coords)
    green_df = customer_demands[customer_demands["目标客户编号"].isin(green_set)].copy()
    outer_df = customer_demands[~customer_demands["目标客户编号"].isin(green_set)].copy()
    return green_df, outer_df

def make_vehicle_pool(vehicle_names):
    pool = []
    for name in vehicle_names:
        pool.extend([name] * q1.VEHICLES[name]["count"])
    return pool

def build_routes_for_subset(customer_subset, dist_matrix, time_windows, fleet_order, vehicle_limits=None):
    if customer_subset.empty:
        return []
    old_order = q1.FLEET_ORDER[:]
    old_counts = {name: spec["count"] for name, spec in q1.VEHICLES.items()}
    q1.FLEET_ORDER = fleet_order[:]
    if vehicle_limits is not None:
        for name, spec in q1.VEHICLES.items():
            spec["count"] = int(vehicle_limits.get(name, 0))
    try:
        routes = q1.build_capacity_focused_routes(customer_subset, dist_matrix, time_windows)
    finally:
        q1.FLEET_ORDER = old_order
        for name, count in old_counts.items():
            q1.VEHICLES[name]["count"] = count
    return routes

def fleet_order_from_remaining(remaining):
    order = []
    for name in Q2_FLEET_ORDER:
        order.extend([name] * int(remaining.get(name, 0)))
    return order

def try_assign_with_pool(routes, dist_matrix, time_windows, speed_scenarios, coords, remaining, green_only=False):
    assigned = [None] * len(routes)
    used_counter = defaultdict(int)
    normalized_routes = [q1.normalize_route_stops(route["stops"]) for route in routes]
    pending = set(range(len(routes)))

    while pending:

```

```

candidate_rows = []
for idx in pending:
    stops = normalized_routes[idx]
    choices = [
        name
        for name in feasible_types_policy(stops, coords)
        if remaining.get(name, 0) > 0
    ]
    if green_only:
        choices = [name for name in choices if q1.VEHICLES[name]["power"] == "ev"]
    evaluated = []
    has_green = route_has_green_customer(stops, coords)
    for name in choices:
        cost = evaluate_route_policy(stops, name, dist_matrix, time_windows, speed_scenarios, coords)
        if cost is not None:
            evaluated.append((policy_score(cost, name, has_green), name, cost))
    if not evaluated:
        return None
    evaluated.sort(key=lambda item: item[0])
    load_w, load_v = q1.route_load(stops)
    candidate_rows.append(
        (
            len(evaluated),
            -load_w,
            -load_v,
            evaluated[0][0],
            idx,
            evaluated,
        )
    )

_, _, _, _, chosen_idx, evaluated = min(candidate_rows, key=lambda item: item[5])
_, best_name, best_cost = evaluated[0]
remaining[best_name] -= 1
used_counter[best_name] += 1
assigned[chosen_idx] = {
    "route_id": f"R{chosen_idx + 1:03d}",
    "vehicle": best_name,
    "vehicle_no": f"{best_name}-{used_counter[best_name]:02d}",
    "stops": normalized_routes[chosen_idx],
    "cost": best_cost,
}
pending.remove(chosen_idx)

return assigned

def benchmark_q1_carbon_cost():
    q1_path = os.path.join(PROC_DIR, "1-问题一最优方案.xlsx")
    if not os.path.exists(q1_path):
        return None
    overview = pd.read_excel(q1_path, sheet_name=0)
    hit = overview.loc[overview["指标"] == "碳排放成本", "数值"]
    if hit.empty:
        return None
    return float(hit.iloc[0])

def renumber_routes(routes):
    used_counter = defaultdict(int)
    for idx, route in enumerate(routes, start=1):
        route["route_id"] = f"R{idx:03d}"
        used_counter[route["vehicle"]] += 1
        route["vehicle_no"] = f"{route['vehicle']}-{used_counter[route['vehicle']]:02d}"
    return routes

def global_assign_vehicle_types(routes, dist_matrix, time_windows, speed_scenarios, coords, carbon_lambda=0.0):
    vehicle_names = list(q1.VEHICLES.keys())

```

```

route_count = len(routes)
source = 0
route_offset = 1
vehicle_offset = route_offset + route_count
sink = vehicle_offset + len(vehicle_names)
graph = [[] for _ in range(sink + 1)]

for ridx in range(route_count):
    add_flow_edge(graph, source, route_offset + ridx, 1, 0)

for vidx, vehicle_name in enumerate(vehicle_names):
    add_flow_edge(
        graph,
        vehicle_offset + vidx,
        sink,
        q1.VEHICLES[vehicle_name]["count"],
        0,
    )

edge_map = {}
route_cost_cache = {}
for ridx, route in enumerate(routes):
    stops = q1.normalize_route_stops(route["stops"])
    has_green = route_has_green_customer(stops, coords)
    for vidx, vehicle_name in enumerate(vehicle_names):
        cost_result = evaluate_route_policy(
            stops,
            vehicle_name,
            dist_matrix,
            time_windows,
            speed_scenarios,
            coords,
        )
        if cost_result is None:
            continue
        route_cost_cache[(ridx, vehicle_name)] = cost_result
        score = policy_score(cost_result, vehicle_name, has_green) + carbon_lambda * float(cost_result["carbon"])
        edge_map[(ridx, vehicle_name)] = (route_offset + ridx, len(graph[route_offset + ridx]))
        add_flow_edge(
            graph,
            route_offset + ridx,
            vehicle_offset + vidx,
            1,
            int(round(score * 100)),
        )

flow, _ = min_cost_max_flow(graph, source, sink, route_count)
if flow < route_count:
    return None

assigned = []
for ridx, route in enumerate(routes):
    chosen_vehicle = None
    chosen_cost = None
    for vehicle_name in vehicle_names:
        key = (ridx, vehicle_name)
        if key not in edge_map:
            continue
        u, edge_idx = edge_map[key]
        if graph[u][edge_idx].cap == 0:
            chosen_vehicle = vehicle_name
            chosen_cost = route_cost_cache[key]
            break
    if chosen_vehicle is None:
        return None
    assigned.append(
        {
            "route_id": route.get("route_id", f"R{ridx + 1:03d}"),
            "vehicle": chosen_vehicle,

```

```

        "vehicle_no": route.get("vehicle_no", chosen_vehicle),
        "stops": q1.normalize_route_stops(route["stops"]),
        "cost": chosen_cost,
    }
)
return renumber_routes(assigned)

def total_carbon_cost(routes):
    return float(sum(route["cost"]["carbon"] for route in routes))

def total_route_cost(routes):
    return float(sum(route["cost"]["total"] for route in routes))

def search_low_carbon_assignment(routes, dist_matrix, time_windows, speed_scenarios, coords, carbon_target):
    candidates = []
    for carbon_lambda in CARBON_SEARCH_LAMBDA:
        assigned = global_assign_vehicle_types(
            routes,
            dist_matrix,
            time_windows,
            speed_scenarios,
            coords,
            carbon_lambda=carbon_lambda,
        )
        if assigned is None:
            continue
        candidates.append(
            {
                "lambda": carbon_lambda,
                "routes": assigned,
                "total": total_route_cost(assigned),
                "carbon": total_carbon_cost(assigned),
            }
        )

    if not candidates:
        return routes, None

    feasible = [item for item in candidates if carbon_target is not None and item["carbon"] <= carbon_target + 1e-8]
    if feasible:
        best = min(feasible, key=lambda item: (item["total"], item["carbon"], item["lambda"]))
    else:
        best = min(candidates, key=lambda item: (item["carbon"], item["total"], item["lambda"]))
    return best["routes"], best

def build_initial_routes_for_q2(customer_demands, dist_matrix, time_windows, coords):
    green_df, outer_df = split_demands_by_policy(customer_demands, coords)

    green_routes = build_routes_for_subset(
        green_df,
        dist_matrix,
        time_windows,
        ["新能源 1", "新能源 2"],
        {"新能源 1": q1.VEHICLES["新能源 1"]["count"], "新能源 2": q1.VEHICLES["新能源 2"]["count"]},
    )
    outer_routes = build_routes_for_subset(
        outer_df,
        dist_matrix,
        time_windows,
        Q2_FLEET_ORDER,
    )
    return green_routes, outer_routes

def assign_vehicle_types_policy(route_groups, dist_matrix, time_windows, speed_scenarios, coords):

```

```

green_routes, outer_routes = route_groups
remaining = {name: spec["count"] for name, spec in q1.VEHICLES.items()}

green_assigned = try_assign_with_pool(
    green_routes,
    dist_matrix,
    time_windows,
    speed_scenarios,
    coords,
    remaining,
    green_only=True,
)
if green_assigned is None:
    raise RuntimeError(" 问题二: 绿色配送区客户需求无法由现有新能源车独立承接。")

# 绿色区锁定新能源车后, 外围需求应基于剩余车池重新建路, 避免先建路再分车型导致容量结构失配。
outer_stops = []
for route in outer_routes:
    for stop in route["stops"]:
        outer_stops.append({"目标客户编号": stop["customer"], "重量": stop["weight"], "体积": stop["volume"]})
if outer_stops:
    outer_df = pd.DataFrame(outer_stops).groupby("目标客户编号", as_index=False)[["重量", "体积"]].sum()
    remaining_order = fleet_order_from_remaining(remaining)
    if not remaining_order:
        raise RuntimeError(" 问题二: 绿色区分配后已无剩余车辆可服务外围客户。")
    outer_routes = build_routes_for_subset(outer_df, dist_matrix, time_windows, Q2_FLEET_ORDER, remaining)

outer_assigned = try_assign_with_pool(
    outer_routes,
    dist_matrix,
    time_windows,
    speed_scenarios,
    coords,
    remaining,
    green_only=False,
)
if outer_assigned is None:
    raise RuntimeError(" 问题二: 外围客户的车型分配失败。")

final_routes = green_assigned + outer_assigned
final_routes.sort(
    key=lambda route: (
        0 if route_has_green_customer(route["stops"], coords) else 1,
        route["vehicle"],
        route["route_id"],
    )
)
return renumber_routes(final_routes)

def improve_route_sequence_policy(route, dist_matrix, time_windows, speed_scenarios, coords):
    stops = q1.normalize_route_stops(route["stops"])
    if len(stops) <= 1:
        route["stops"] = stops
        route["cost"] = evaluate_route_policy(stops, route["vehicle"], dist_matrix, time_windows, speed_scenarios, coords)
        return route

    best = stops[:]
    best_cost = evaluate_route_policy(best, route["vehicle"], dist_matrix, time_windows, speed_scenarios, coords)["total"]
    improved = True
    passes = 0
    while improved and passes < q1.LOCAL_SEARCH_PASSES:
        improved = False
        passes += 1
        n = len(best)
        candidates = []
        for i in range(n - 1):
            for j in range(i + 1, n):
                cand = best[:i]

```



```

        cand[i], cand[j] = cand[j], cand[i]
        candidates.append(q1.normalize_route_stops(cand))
        cand = best[:i] + list(reversed(best[i : j + 1])) + best[j + 1 :]
        candidates.append(q1.normalize_route_stops(cand))
    for i in range(n):
        for j in range(n):
            if i == j:
                continue
            cand = best[:]
            moved = cand.pop(i)
            cand.insert(j, moved)
            candidates.append(q1.normalize_route_stops(cand))

    best_candidate = None
    for cand in candidates:
        result = evaluate_route_policy(cand, route["vehicle"], dist_matrix, time_windows, speed_scenarios, coords)
        if result is not None and result["total"] + 1e-6 < best_cost:
            best_cost = result["total"]
            best_candidate = cand
    if best_candidate is not None:
        best = best_candidate
        improved = True

    route["stops"] = best
    route["cost"] = evaluate_route_policy(best, route["vehicle"], dist_matrix, time_windows, speed_scenarios, coords)
    return route

def improve_all_routes_policy(routes, dist_matrix, time_windows, speed_scenarios, coords):
    improved = []
    before = sum(route["cost"]["total"] for route in routes)
    for route in routes:
        improved.append(improve_route_sequence_policy(route, dist_matrix, time_windows, speed_scenarios, coords))
    after = sum(route["cost"]["total"] for route in improved)
    return improved, before, after

def policy_violation_count(routes, coords):
    green_set = green_customers(coords)
    count = 0
    for route in routes:
        if q1.VEHICLES[route["vehicle"]]["power"] != "fuel":
            continue
        for detail in route["cost"].get("details", []):
            if detail["customer"] in green_set and FUEL_BAN_START <= detail["arrival"] < FUEL_BAN_END:
                count += 1
    return count

# =====
# 3. 输出
# =====
def hour_to_time(value):
    return q1.hour_to_time(value)

def write_results_policy(routes, customer_demands, dist_matrix, time_windows, speed_scenarios, coords):
    green_set = green_customers(coords)
    summary_rows = []
    detail_rows = []
    type_rows = []
    total = defaultdict(float)

    for route in routes:
        detail_cost = evaluate_route_policy(
            route["stops"], route["vehicle"], dist_matrix, time_windows, speed_scenarios, coords, with_detail=True
        )
        route["cost"] = detail_cost
        load_w, load_v = q1.route_load(route["stops"])

```

```

distance = q1.route_distance(route["stops"], dist_matrix)
path = "0 -> " + " -> ".join(str(stop["customer"]) for stop in route["stops"]) + " -> 0"
arrival = ", ".join(hour_to_time(row["arrival"]) for row in detail_cost["details"])

summary_rows.append(
    {
        "路线编号": route["route_id"],
        "车辆编号": route["vehicle_no"],
        "车辆类型": route["vehicle"],
        "是否含绿色区客户": "是" if route_has_green_customer(route["stops"], coords) else "否",
        "发车时间": hour_to_time(detail_cost["depart"]),
        "返场时间": hour_to_time(detail_cost["return"]),
        "载重 (kg)": round(load_w, 4),
        "体积 (m3)": round(load_v, 4),
        "载重率": round(load_w / q1.VEHICLES[route["vehicle"]]["cap_w"], 4),
        "容积率": round(load_v / q1.VEHICLES[route["vehicle"]]["cap_v"], 4),
        "行驶距离 (km)": round(distance, 4),
        "访问客户数": len(route["stops"]),
        "路径": path,
        "平均到达时间": arrival,
        "启动成本": round(detail_cost["start"], 2),
        "能源成本": round(detail_cost["energy"], 2),
        "碳排放成本": round(detail_cost["carbon"], 2),
        "碳排放量 (kg)": round(detail_cost["carbon"] / q1.CARBON_PRICE, 4),
        "时间窗成本": round(detail_cost["time_window"], 2),
        "等待总时长 (h)": round(detail_cost["wait_hours"], 4),
        "迟到总时长 (h)": round(detail_cost["late_hours"], 4),
        "总成本": round(detail_cost["total"], 2),
    }
)

for key in ["start", "energy", "carbon", "time_window", "total"]:
    total[key] += detail_cost[key]

for stop, detail in zip(route["stops"], detail_cost["details"]):
    customer = stop["customer"]
    window = time_windows.get(customer, {"early": q1.START_HOUR, "late": 24.0})
    fuel_forbidden = (
        q1.VEHICLES[route["vehicle"]]["power"] == "fuel"
        and customer in green_set
        and FUEL_BAN_START <= detail["arrival"] < FUEL_BAN_END
    )
    detail_rows.append(
        {
            "路线编号": route["route_id"],
            "车辆编号": route["vehicle_no"],
            "车辆类型": route["vehicle"],
            "访问顺序": detail["seq"],
            "客户编号": customer,
            "是否绿色区客户": "是" if customer in green_set else "否",
            "燃油限行是否违规": "是" if fuel_forbidden else "否",
            "配送重量 (kg)": round(stop["weight"], 4),
            "配送体积 (m3)": round(stop["volume"], 4),
            "时间窗开始": hour_to_time(window["early"]),
            "时间窗结束": hour_to_time(window["late"]),
            "平均到达": hour_to_time(detail["arrival"]),
            "平均服务开始": hour_to_time(detail["service_start"]),
            "平均离开": hour_to_time(detail["departure"]),
            "平均等待 (h)": round(detail["wait_hours"], 4),
            "平均迟到 (h)": round(detail["late_hours"], 4),
            "上一点到本点距离 (km)": round(detail["travel_distance"], 4),
            "包含拆分任务": ", ".join(stop["task_ids"]),
        }
    )

used = defaultdict(int)
for route in routes:
    used[route["vehicle"]] += 1
for name, spec in q1.VEHICLES.items():
    type_rows.append(

```

```

        {
            " 车辆类型": name,
            " 动力类型": " 新能源" if spec["power"] == "ev" else " 燃油",
            " 可用数量": spec["count"],
            " 使用数量": used[name],
            " 剩余数量": spec["count"] - used[name],
            " 载重上限 (kg)": spec["cap_w"],
            " 容积上限 (m3)": spec["cap_v"],
        }
    )

summary_df = pd.DataFrame(summary_rows)
detail_df = pd.DataFrame(detail_rows)
type_df = pd.DataFrame(type_rows)
green_route_df = summary_df[summary_df[" 是否含绿色区客户"] == " 是"].copy()
green_ev_count = int(green_route_df[" 车辆类型"].astype(str).str.contains(" 新能源", regex=False).sum())
green_fuel_count = int(green_route_df[" 车辆类型"].astype(str).str.contains(" 燃油", regex=False).sum())
customer_check = customer_demands.rename(
    columns={" 目标客户编号": " 客户编号", " 重量": " 总重量 (kg)", " 体积": " 总体积 (m3)"})
)

overview = pd.DataFrame(
    [
        {" 指标": " 问题", " 数值": " 问题二: 环保政策影响下的车辆调度策略"},
        {" 指标": " 使用车辆数", " 数值": len(routes)},
        {" 指标": " 新能源车使用数", " 数值": int(type_df[type_df[" 动力类型"] == " 新能源"][" 使用数量"].sum())},
        {" 指标": " 燃油车使用数", " 数值": int(type_df[type_df[" 动力类型"] == " 燃油"][" 使用数量"].sum())},
        {" 指标": " 总行驶距离 (km)", " 数值": round(summary_df[" 行驶距离 (km)"].sum(), 2)},
        {" 指标": " 启动成本", " 数值": round(total["start"], 2)},
        {" 指标": " 能源成本", " 数值": round(total["energy"], 2)},
        {" 指标": " 碳排放成本", " 数值": round(total["carbon"], 2)},
        {" 指标": " 碳排放总量 (kg)", " 数值": round(total["carbon"] / q1.CARBON_PRICE, 4)},
        {" 指标": " 时间窗成本", " 数值": round(total["time_window"], 2)},
        {" 指标": " 总成本", " 数值": round(total["total"], 2)},
        {" 指标": " 政策", " 数值": "8:00-16:00 禁止燃油车进入半径 10km 绿色配送区"},
        {" 指标": " 绿色区客户数", " 数值": len(green_set)},
        {" 指标": " 涉及绿色区的路线数", " 数值": len(green_route_df)},
        {" 指标": " 绿色区路线中新能源车数", " 数值": green_ev_count},
        {" 指标": " 绿色区路线中燃油车数", " 数值": green_fuel_count},
        {" 指标": " 限行违规次数", " 数值": int((detail_df[" 燃油限行是否违规"] == " 是").sum())},
        {" 指标": " 速度场景数", " 数值": q1.NUM_SPEED_SCENARIOS},
        {" 指标": " 随机种子", " 数值": q1.RANDOM_SEED},
    ]
)

speed_rows = []
for scenario in speed_scenarios:
    for period in scenario["periods"]:
        speed_rows.append(
            {
                " 场景": scenario["scenario_id"],
                " 时段": f"{hour_to_time(period['start'])}-{hour_to_time(period['end'])}",
                " 交通状态": period["label"],
                " 抽样速度 (km/h)": round(period["speed"], 4),
            }
        )

compare_df = build_policy_impact_comparison(overview, type_df)

out_path = os.path.join(PROC_DIR, "2-问题二限行政策调度方案.xlsx")
final_out_path = out_path

def write_excel(target_path):
    with pd.ExcelWriter(target_path, engine="openpyxl") as writer:
        overview.to_excel(writer, sheet_name=" 方案总览", index=False)
        compare_df.to_excel(writer, sheet_name=" 政策影响对比", index=False)
        type_df.to_excel(writer, sheet_name=" 车辆使用统计", index=False)
        summary_df.to_excel(writer, sheet_name=" 车辆路径", index=False)
        detail_df.to_excel(writer, sheet_name=" 到达时间明细", index=False)

```

```

        customer_check.to_excel(writer, sheet_name=" 客户需求汇总", index=False)
        pd.DataFrame(speed_rows).to_excel(writer, sheet_name=" 速度抽样场景", index=False)

    try:
        write_excel(out_path)
    except PermissionError:
        final_out_path = os.path.join(PROC_DIR, "2-问题二限行政策调度方案-新.xlsx")
        write_excel(final_out_path)

    plot_routes_policy(routes, coords, os.path.join(PROC_DIR, "2-问题二车辆行驶路径图.png"))
    plot_vehicle_usage_policy(type_df, os.path.join(PROC_DIR, "2-问题二车辆使用结构图.png"))
    plot_cost_policy(overview, os.path.join(PROC_DIR, "2-问题二成本构成图.png"))
    return final_out_path, overview, type_df

def overview_value(overview, key, default=None):
    hit = overview.loc[overview[" 指标"] == key, " 数值"]
    if hit.empty:
        return default
    return hit.iloc[0]

def overview_number(overview, key, default=0.0, cast=float):
    value = overview_value(overview, key, default)
    if value is None:
        return cast(default)
    return cast(value)

def relative_proc_path(path):
    try:
        return os.path.relpath(path, BASE_DIR)
    except ValueError:
        return path

def print_terminal_summary(overview, type_df, out_path, before, after):
    total_cost = overview_number(overview, " 总成本", 0.0, float)
    start_cost = overview_number(overview, " 启动成本", 0.0, float)
    energy_cost = overview_number(overview, " 能源成本", 0.0, float)
    carbon_cost = overview_number(overview, " 碳排放成本", 0.0, float)
    time_window_cost = overview_number(overview, " 时间窗成本", 0.0, float)
    total_distance = overview_number(overview, " 总行驶距离 (km)", 0.0, float)
    used_routes = overview_number(overview, " 使用车辆数", 0, int)
    green_customer_count = overview_number(overview, " 绿色区客户数", 0, int)
    green_route_count = overview_number(overview, " 涉及绿色区的路线数", 0, int)
    green_ev_count = overview_number(overview, " 绿色区路线中新能源车数", 0, int)
    green_fuel_count = overview_number(overview, " 绿色区路线中燃油车数", 0, int)
    violation = overview_number(overview, " 限行违规次数", 0, int)

    ev_used = int(type_df.loc[type_df[" 动力类型"] == " 新能源", " 使用数量"].sum())
    fuel_used = int(type_df.loc[type_df[" 动力类型"] == " 燃油", " 使用数量"].sum())
    improvement = before - after
    improvement_pct = improvement / before * 100 if before > 1e-9 else 0.0
    green_ev_share = green_ev_count / green_route_count * 100 if green_route_count else 0.0

    print("\n" + "=" * 72)
    print(" 问题二求解完成")
    print("=" * 72)
    print(" 可行性")
    print(f" 限行违规次数: {violation}")
    print(f" 绿色配送区客户数: {green_customer_count}")
    print(f" 涉及绿色区的路线数: {green_route_count}")
    print(f" 绿色区路线分配: 新能源 {green_ev_count} 条, 燃油 {green_fuel_count} 条, 新能源占比 {green_ev_share:.2f}%")
    print(" 成本")
    print(f" 优化前成本: {before:.2f}")
    print(f" 优化后成本: {after:.2f}")
    print(f" 本轮改进: {improvement:.2f} ({improvement_pct:.2f}%)")
    print(f" 总成本: {total_cost:.2f}")

```

```

print(f" 成本构成: 启动 {start_cost:.2f} | 能源 {energy_cost:.2f} | 碳排放 {carbon_cost:.2f} | 时间窗
↳ {time_window_cost:.2f}")
print(" 车辆")
print(f" 使用车辆数: {used_routes}")
print(f" 新能源车: {ev_used}")
print(f" 燃油车: {fuel_used}")
for _, row in type_df.iterrows():
    print(
        f" {row['车辆类型']}: 已用 {int(row['使用数量'])} / 总量 {int(row['可用数量'])} / 剩余 {int(row['剩余数量'])}"
    )
print(" 运行结果")
print(f" 总行驶距离: {total_distance:.2f} km")
print(f" 结果文件: {relative_proc_path(out_path)}")
print(f" 路径图: {relative_proc_path(os.path.join(PROC_DIR, '2-问题二车辆行驶路径图.png'))}")
print(f" 车辆结构图: {relative_proc_path(os.path.join(PROC_DIR, '2-问题二车辆使用结构图.png'))}")
print(f" 成本构成图: {relative_proc_path(os.path.join(PROC_DIR, '2-问题二成本构成图.png'))}")
print("=" * 72)

def build_policy_impact_comparison(overview2, type_df2):
    q1_path = os.path.join(PROC_DIR, "1-问题一最优方案.xlsx")
    if not os.path.exists(q1_path):
        return pd.DataFrame(
            [{"指标": "说明", "问题一": "未找到问题一输出文件", "问题二": "", "变化": ""}]
        )

    overview1 = pd.read_excel(q1_path, sheet_name=0)
    type_df1 = pd.read_excel(q1_path, sheet_name=1)
    rows = []
    for key in ["使用车辆数", "总行驶距离(km)", "启动成本", "能源成本", "碳排放成本", "碳排放总量(kg)", "时间窗成本", "总成本"]:
        v1 = float(overview_value(overview1, key, 0))
        v2 = float(overview_value(overview2, key, 0))
        rows.append({"指标": key, "问题一": round(v1, 4), "问题二": round(v2, 4), "变化": round(v2 - v1, 4)})

    for power in ["新能源", "燃油"]:
        v1 = int(type_df1[type_df1["动力类型"] == power]["使用数量"].sum())
        v2 = int(type_df2[type_df2["动力类型"] == power]["使用数量"].sum())
        rows.append({"指标": f"{power} 车使用数", "问题一": v1, "问题二": v2, "变化": v2 - v1})

    return pd.DataFrame(rows)

def plot_routes_policy(routes, coords, out_path):
    q1.setup_chinese_font()
    fig, ax = plt.subplots(figsize=(14, 12), dpi=180)
    fig.patch.set_facecolor("#f7f3ea")
    ax.set_facecolor("#f7f3ea")
    green_circle = plt.Circle((0, 0), GREEN_RADIUS, color="#64a96b", alpha=0.16, fill=True)
    green_border = plt.Circle((0, 0), GREEN_RADIUS, color="#2f855a", alpha=0.9, lw=1.5, fill=False)
    ax.add_patch(green_circle)
    ax.add_patch(green_border)
    ax.text(0, GREEN_RADIUS + 0.8, "绿色配送区 (燃油车 8:00-16:00 禁入)", ha="center", fontsize=11, color="#276749")

    green_set = green_customers(coords)
    customers = [cid for cid in coords if cid != 0]
    ax.scatter(
        [coords[cid]["x"] for cid in customers if cid not in green_set],
        [coords[cid]["y"] for cid in customers if cid not in green_set],
        s=22,
        c="#475569",
        alpha=0.75,
        label="普通客户",
    )
    ax.scatter(
        [coords[cid]["x"] for cid in customers if cid in green_set],
        [coords[cid]["y"] for cid in customers if cid in green_set],
        s=28,
        c="#16a34a",
        alpha=0.9,
    )

```

```

        label=" 绿色区客户",
    )
    ax.scatter(coords[0]["x"], coords[0]["y"], s=180, marker="*", c="#c0392b", label=" 配送中心", zorder=6)

for route in routes:
    path_ids = [0] + [stop["customer"] for stop in route["stops"]] + [0]
    xs = [coords[cid]["x"] for cid in path_ids]
    ys = [coords[cid]["y"] for cid in path_ids]
    color = "#0f766e" if q1.VEHICLES[route["vehicle"]]["power"] == "ev" else "#ea580c"
    ax.plot(xs, ys, color=color, lw=0.9, alpha=0.42)

ax.set_xlabel("X 坐标 / km")
ax.set_ylabel("Y 坐标 / km")
ax.grid(True, color="#d7cfbd", linestyle="--", linewidth=0.6, alpha=0.8)
ax.legend(loc="upper right")
ax.set_aspect("equal", adjustable="box")
fig.tight_layout()
fig.savefig(out_path, bbox_inches="tight")
plt.close(fig)

def plot_vehicle_usage_policy(type_df, out_path):
    q1.setup_chinese_font()
    fig, ax = plt.subplots(figsize=(10, 6), dpi=180)
    fig.patch.set_facecolor("white")
    ax.set_facecolor("white")
    colors = ["#0f766e" if p == " 新能源" else "#ea580c" for p in type_df[" 动力类型"]]
    x = np.arange(len(type_df))
    ax.bar(x, type_df[" 使用数量"], color=colors, width=0.62)
    for idx, value in enumerate(type_df[" 使用数量"]):
        ax.text(idx, value + 1, str(int(value)), ha="center", va="bottom")
    ax.set_xticks(x)
    ax.set_xticklabels(type_df[" 车辆类型"])
    ax.set_ylabel(" 使用车辆数")
    ax.grid(axis="y", linestyle="--", alpha=0.35)
    fig.tight_layout()
    fig.savefig(out_path, bbox_inches="tight")
    plt.close(fig)

def plot_cost_policy(overview, out_path):
    q1.setup_chinese_font()
    values = {}
    for _, row in overview.iterrows():
        values[row[" 指标"]] = row[" 数值"]
    labels = [" 启动成本", " 能源成本", " 碳排放成本", " 时间窗成本"]
    costs = [float(values[label]) for label in labels]
    fig, ax = plt.subplots(figsize=(9, 6), dpi=180)
    fig.patch.set_facecolor("white")
    ax.set_facecolor("white")
    ax.pie(
        costs,
        labels=labels,
        colors=["#334155", "#0f766e", "#65a30d", "#ea580c"],
        startangle=120,
        autopct=lambda pct: f"{pct:.1f}%",
        wedgeprops={"linewidth": 1.2, "edgecolor": "white"},
    )
    ax.set_title(f" 问题二: 成本构成 (总成本 {float(values['总成本']):.2f} 元)", fontsize=16, weight="bold")
    fig.tight_layout()
    fig.savefig(out_path, bbox_inches="tight")
    plt.close(fig)

def main():
    print(" 读取数据并构造问题二模型...")
    dist_matrix = q1.read_distance_matrix()
    q1.dist_matrix_global = dist_matrix
    time_windows = q1.read_time_windows()

```

```

coords = q1.read_coordinates()
_, customer_demands = q1.build_delivery_tasks()
speed_scenarios = q1.generate_speed_scenarios()

print(f" 绿色配送区客户数: {len(green_customers(coords))}")
best_order, _, _ = q1.select_best_fleet_order(customer_demands, dist_matrix, time_windows, speed_scenarios)
print(f" 沿用问题一较优车队展开顺序: {' -> '.join(best_order)}")

print(" 问题二专用建路: 新能源车优先吸纳绿色区客户, 燃油车在禁行时段不选绿色区点...")
routes = build_initial_routes_for_q2(customer_demands, dist_matrix, time_windows, coords)
routes = assign_vehicle_types_policy(routes, dist_matrix, time_windows, speed_scenarios, coords)
routes, before, after = improve_all_routes_policy(routes, dist_matrix, time_windows, speed_scenarios, coords)
print(f" 路线优化完成: 成本 {before:.2f} -> {after:.2f}")

q1_carbon_target = benchmark_q1_carbon_cost()
routes, search_result = search_low_carbon_assignment(
    routes,
    dist_matrix,
    time_windows,
    speed_scenarios,
    coords,
    q1_carbon_target,
)
if search_result is not None:
    target_text = " 无" if q1_carbon_target is None else f"{q1_carbon_target:.2f}"
    print(
        f" 全局车型重分配完成: lambda={search_result['lambda']} | "
        f" 总成本 {search_result['total']:.2f} | 碳排放成本 {search_result['carbon']:.2f} | "
        f" 问题一碳目标 {target_text}"
    )

out_path, overview, type_df = write_results_policy(routes, customer_demands, dist_matrix, time_windows, speed_scenarios,
↪ coords)
print_terminal_summary(overview, type_df, out_path, before, after)

if __name__ == "__main__":
    main()

```

B.4 问题三情景生成程序: 3-问题三情景生成.py

```

import importlib.util
import math
import os

import numpy as np
import pandas as pd

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
PROC_DIR = os.path.join(BASE_DIR, " 过程")
os.makedirs(PROC_DIR, exist_ok=True)

Q1_PATH = os.path.join(BASE_DIR, "1-问题一.py")
Q2_RESULT_PATH = os.path.join(PROC_DIR, "2-问题二限行政策调度方案.xlsx")
ORDER_PATH = os.path.join(PROC_DIR, "0-已填充订单数据.xlsx")

SCENE_BOOK_PATH = os.path.join(PROC_DIR, "3-问题三情景事件.xlsx")
SCENE_DESC_PATH = os.path.join(PROC_DIR, "3-问题三情景说明.xlsx")

RANDOM_SEED = 20260424
RNG = np.random.default_rng(RANDOM_SEED)

def load_module(module_name, path):
    spec = importlib.util.spec_from_file_location(module_name, path)
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module

```

```

q1 = load_module("q1_for_q3_scene_v2", Q1_PATH)

def parse_hour(value):
    return q1.parse_time_to_hour(value)

def hour_to_time(value):
    return q1.hour_to_time(value)

def load_baseline():
    q2_book = pd.read_excel(Q2_RESULT_PATH, sheet_name=None)
    summary_df = q2_book[" 车辆路径"].copy()
    detail_df = q2_book[" 到达时间明细"].copy()
    overview_df = q2_book[" 方案总览"].copy()
    coords = q1.read_coordinates()
    time_windows = q1.read_time_windows()
    orders = pd.read_excel(ORDER_PATH)
    demand_df = orders.groupby(" 目标客户编号", as_index=False)[[" 重量", " 体积"]].sum()
    demand_df = demand_df.rename(columns={" 目标客户编号": " 客户编号", " 重量": " 总重量 (kg)", " 体积": " 总体积 (m3)"})
    return summary_df, detail_df, overview_df, coords, time_windows, demand_df

def build_future_pool(summary_df, detail_df, time_windows, coords):
    rows = []
    for _, route_row in summary_df.iterrows():
        route_id = route_row[" 路线编号"]
        depart = parse_hour(route_row[" 发车时间"])
        route_detail = detail_df[detail_df[" 路线编号"] == route_id].sort_values(" 访问顺序")
        for _, stop in route_detail.iterrows():
            cid = int(stop[" 客户编号"])
            tw = time_windows.get(cid, {"early": q1.START_HOUR, "late": 24.0})
            rows.append(
                {
                    " 路线编号": route_id,
                    " 车辆编号": route_row[" 车辆编号"],
                    " 车辆类型": route_row[" 车辆类型"],
                    " 发车小时": depart,
                    " 发车时间": route_row[" 发车时间"],
                    " 客户编号": cid,
                    " 配送重量 (kg)": float(stop[" 配送重量 (kg)"]),
                    " 配送体积 (m3)": float(stop[" 配送体积 (m3)"]),
                    " 是否绿色区客户": str(stop[" 是否绿色区客户"]),
                    " 时间窗开始": hour_to_time(tw["early"]),
                    " 时间窗结束": hour_to_time(tw["late"]),
                    " 时间窗宽度 (h)": round(float(tw["late"] - tw["early"]), 4),
                    "X(km)": float(coords[cid]["x"]),
                    "Y(km)": float(coords[cid]["y"]),
                }
            )
    pool_df = pd.DataFrame(rows)
    pool_df[" 候选评分"] = (
        pool_df[" 配送重量 (kg)"] * 0.6
        + pool_df[" 配送体积 (m3)"] * 95.0
        + (pool_df[" 发车小时"] - q1.START_HOUR) * 28.0
        + (pool_df[" 是否绿色区客户"] == " 是").astype(float) * 50.0
    )
    return pool_df.sort_values(["候选评分", "配送重量(kg)", "客户编号"], ascending=[False, False, True]).reset_index(drop=True)

def choose_candidate(pool_df, trigger_hour, used_customers=None, require_green=None, min_width=None, late_depart=False):
    used_customers = used_customers or set()
    subset = pool_df[pool_df[" 发车小时"] >= trigger_hour].copy()
    if late_depart:
        subset = subset[subset[" 发车小时"] >= trigger_hour + 0.5]
    if require_green is not None:

```



```

        subset = subset[subset[" 是否绿色区客户"] == (" 是" if require_green else " 否")]
    if min_width is not None:
        subset = subset[subset[" 时间窗宽度 (h)" ] >= min_width]
    subset = subset[~subset[" 客户编号"].isin(used_customers)]
    if subset.empty:
        subset = pool_df[~pool_df[" 客户编号"].isin(used_customers)]
    if subset.empty:
        return None
    return subset.iloc[0].to_dict()

def base_event(scene_id, scene_name, trigger_hour, event_order, event_type):
    return {
        " 场景编号": scene_id,
        " 场景名称": scene_name,
        " 触发时间": hour_to_time(trigger_hour),
        " 事件顺序": event_order,
        " 事件类型": event_type,
        " 目标客户编号": np.nan,
        " 目标路线编号": "",
        " 目标车辆编号": "",
        " 虚拟客户编号": np.nan,
        " 新重量 (kg)": np.nan,
        " 新体积 (m3)": np.nan,
        " 原时间窗开始": "",
        " 原时间窗结束": "",
        " 新时间窗开始": "",
        " 新时间窗结束": "",
        " 原 X(km)": np.nan,
        " 原 Y(km)": np.nan,
        " 新 X(km)": np.nan,
        " 新 Y(km)": np.nan,
        " 附加信息": "",
        " 说明": "",
    }

def event_new_order(scene_id, scene_name, trigger_hour, pool_df, coords, used_customers, event_order):
    anchor = choose_candidate(pool_df, trigger_hour, used_customers=used_customers)
    if anchor is None:
        raise RuntimeError(f"{scene_id} 无法生成新增订单事件。")
    used_customers.add(int(anchor[" 客户编号"]))

    event = base_event(scene_id, scene_name, trigger_hour, event_order, "new_order")
    new_customer = max(coords) + 100 + event_order
    radius = float(RNG.uniform(2.2, 4.8))
    theta = float(RNG.uniform(0, 2 * math.pi))
    early = max(trigger_hour + 0.5, parse_hour(anchor[" 时间窗开始"])) + float(RNG.uniform(0.2, 0.7))
    late = min(23.5, early + float(RNG.uniform(1.1, 2.1)))
    event.update(
        {
            " 虚拟客户编号": new_customer,
            " 新重量 (kg)": round(max(650.0, min(2800.0, anchor[" 配送重量 (kg)"] * float(RNG.uniform(0.55, 0.9)))), 4),
            " 新体积 (m3)": round(max(2.0, min(9.8, anchor[" 配送体积 (m3)"] * float(RNG.uniform(0.6, 1.0)))), 4),
            " 新时间窗开始": hour_to_time(early),
            " 新时间窗结束": hour_to_time(late),
            " 新 X(km)": round(float(anchor["X(km)"]) + radius * math.cos(theta), 4),
            " 新 Y(km)": round(float(anchor["Y(km)"]) + radius * math.sin(theta), 4),
            " 附加信息": f" 锚点客户 ={int(anchor['客户编号'])}",
            " 说明": f" 基于客户 {int(anchor['客户编号'])} 周边生成临时新增订单, 模拟实时加单。",
        }
    )
    return event

def event_cancel(scene_id, scene_name, trigger_hour, pool_df, used_customers, event_order):
    target = choose_candidate(pool_df, trigger_hour, used_customers=used_customers)
    if target is None:
        raise RuntimeError(f"{scene_id} 无法生成取消订单事件。")

```

```

used_customers.add(int(target[" 客户编号"]))

event = base_event(scene_id, scene_name, trigger_hour, event_order, "cancel_order")
event.update(
    {
        " 目标客户编号": int(target[" 客户编号"]),
        " 目标路线编号": target[" 路线编号"],
        " 目标车辆编号": target[" 车辆编号"],
        " 原时间窗开始": target[" 时间窗开始"],
        " 原时间窗结束": target[" 时间窗结束"],
        " 原 X(km)": float(target["X(km)"]),
        " 原 Y(km)": float(target["Y(km)"]),
        " 说明": f" 客户 {int(target['客户编号'])} 临时取消订单, 模拟门店计划中止。",
    }
)
return event

def event_time_window_adjustment(scene_id, scene_name, trigger_hour, pool_df, used_customers, event_order):
    target = choose_candidate(pool_df, trigger_hour, used_customers=used_customers, min_width=1.6)
    if target is None:
        raise RuntimeError(f"{scene_id} 无法生成时间窗调整事件。")
    used_customers.add(int(target[" 客户编号"]))

    old_early = parse_hour(target[" 时间窗开始"])
    old_late = parse_hour(target[" 时间窗结束"])
    width = old_late - old_early
    new_width = max(0.75, width * float(RNG.uniform(0.35, 0.55)))
    center = (old_early + old_late) / 2
    new_early = max(trigger_hour + 0.25, center - new_width / 2)
    new_late = min(23.5, new_early + new_width)

    event = base_event(scene_id, scene_name, trigger_hour, event_order, "time_window_adjustment")
    event.update(
        {
            " 目标客户编号": int(target[" 客户编号"]),
            " 目标路线编号": target[" 路线编号"],
            " 目标车辆编号": target[" 车辆编号"],
            " 原时间窗开始": target[" 时间窗开始"],
            " 原时间窗结束": target[" 时间窗结束"],
            " 新时间窗开始": hour_to_time(new_early),
            " 新时间窗结束": hour_to_time(new_late),
            " 原 X(km)": float(target["X(km)"]),
            " 原 Y(km)": float(target["Y(km)"]),
            " 新 X(km)": float(target["X(km)"]),
            " 新 Y(km)": float(target["Y(km)"]),
            " 说明": f" 客户 {int(target['客户编号'])} 收货时间窗发生调整, 模拟预约时间变更。",
        }
    )
    return event

def event_address_change(scene_id, scene_name, trigger_hour, pool_df, used_customers, event_order):
    target = choose_candidate(pool_df, trigger_hour, used_customers=used_customers, require_green=False)
    if target is None:
        raise RuntimeError(f"{scene_id} 无法生成配送地址变更事件。")
    used_customers.add(int(target[" 客户编号"]))

    radius = float(RNG.uniform(2.0, 5.2))
    theta = float(RNG.uniform(0, 2 * math.pi))
    event = base_event(scene_id, scene_name, trigger_hour, event_order, "address_change")
    event.update(
        {
            " 目标客户编号": int(target[" 客户编号"]),
            " 目标路线编号": target[" 路线编号"],
            " 目标车辆编号": target[" 车辆编号"],
            " 原时间窗开始": target[" 时间窗开始"],
            " 原时间窗结束": target[" 时间窗结束"],
            " 新时间窗开始": target[" 时间窗开始"],

```

```

        " 新时间窗结束": target[" 时间窗结束"],
        " 原 X(km)": float(target["X(km)"]),
        " 原 Y(km)": float(target["Y(km)"]),
        " 新 X(km)": round(float(target["X(km)"]) + radius * math.cos(theta), 4),
        " 新 Y(km)": round(float(target["Y(km)"]) + radius * math.sin(theta), 4),
        " 说明": f" 客户 {int(target['客户编号'])} 配送地址发生变更, 模拟园区换门或临时改址。",
    }
)
return event

def event_vehicle_breakdown(scene_id, scene_name, trigger_hour, pool_df, used_customers, event_order):
    target = choose_candidate(pool_df, trigger_hour, used_customers=used_customers, late_depart=True)
    if target is None:
        target = choose_candidate(pool_df, trigger_hour, used_customers=used_customers)
    if target is None:
        raise RuntimeError(f"{scene_id} 无法生成车辆故障事件。")
    used_customers.add(int(target[" 客户编号"]))

    fault_hours = float(RNG.uniform(1.0, 2.5))
    event = base_event(scene_id, scene_name, trigger_hour, event_order, "vehicle_breakdown")
    event.update(
        {
            " 目标客户编号": int(target[" 客户编号"]),
            " 目标路线编号": target[" 路线编号"],
            " 目标车辆编号": target[" 车辆编号"],
            " 原时间窗开始": target[" 时间窗开始"],
            " 原时间窗结束": target[" 时间窗结束"],
            " 原 X(km)": float(target["X(km)"]),
            " 原 Y(km)": float(target["Y(km)"]),
            " 附加信息": f" 故障车辆类型={target['车辆类型']}; 预计停运时长 (h)={fault_hours:.2f}",
            " 说明": f" 车辆 {target['车辆编号']} 在发车前发生故障, 模拟车辆临时停运与任务转移。",
        }
    )
    return event

def build_scene_events(scene_id, scene_name, trigger_hour, mode, pool_df, coords):
    used_customers = set()
    if mode == "new_order":
        return [event_new_order(scene_id, scene_name, trigger_hour, pool_df, coords, used_customers, 1)]
    if mode == "cancel_order":
        return [event_cancel(scene_id, scene_name, trigger_hour, pool_df, used_customers, 1)]
    if mode == "address_change":
        return [event_address_change(scene_id, scene_name, trigger_hour, pool_df, used_customers, 1)]
    if mode == "time_window_adjustment":
        return [event_time_window_adjustment(scene_id, scene_name, trigger_hour, pool_df, used_customers, 1)]
    if mode == "vehicle_breakdown":
        return [event_vehicle_breakdown(scene_id, scene_name, trigger_hour, pool_df, used_customers, 1)]
    raise ValueError(f" 未知场景模式: {mode}")

def generate_scenarios(summary_df, detail_df, coords, time_windows):
    pool_df = build_future_pool(summary_df, detail_df, time_windows, coords)

    configs = [
        ("S1", " 订单取消", 13.0, "cancel_order"),
        ("S2", " 新增订单", 13.3, "new_order"),
        ("S3", " 配送地址变更", 14.0, "address_change"),
        ("S4", " 时间窗调整", 14.4, "time_window_adjustment"),
        ("S5", " 车辆故障", 15.0, "vehicle_breakdown"),
    ]

    scene_rows = []
    event_rows = []
    candidate_rows = []

    for scene_id, scene_name, trigger_hour, mode in configs:
        candidates = pool_df[pool_df[" 发车小时"] >= trigger_hour].copy()

```

```

        if candidates.empty:
            raise RuntimeError(f"{scene_id} 在 {hour_to_time(trigger_hour)} 后没有候选未发车任务。")

        preview = candidates.head(12).copy()
        preview.insert(0, " 场景编号", scene_id)
        preview.insert(1, " 场景名称", scene_name)
        candidate_rows.extend(preview.to_dict("records"))

        scene_events = build_scene_events(scene_id, scene_name, trigger_hour, mode, pool_df, coords)
        event_rows.extend(scene_events)

        affected_routes = summary_df[summary_df[" 发车时间"].map(parse_hour) >= trigger_hour][" 路线编号"].nunique()
        candidate_customers = int(candidates[" 客户编号"].nunique())
        scene_rows.append(
            {
                " 场景编号": scene_id,
                " 场景名称": scene_name,
                " 触发时间": hour_to_time(trigger_hour),
                " 场景模式": mode,
                " 触发后未发车路线数": int(affected_routes),
                " 触发后候选客户数": candidate_customers,
                " 事件数量": len(scene_events),
                " 场景说明": " 仅对触发时刻后尚未发车的任务执行滚动重调度，原始附件与问题二结果保持不变。",
            }
        )

    return pd.DataFrame(scene_rows), pd.DataFrame(event_rows), pd.DataFrame(candidate_rows)

def build_description(scene_df, event_df, overview_df, demand_df):
    total_cost = float(overview_df.loc[overview_df[" 指标"] == " 总成本", " 数值"].iloc[0])
    carbon_cost = float(overview_df.loc[overview_df[" 指标"] == " 碳排放成本", " 数值"].iloc[0])
    return pd.DataFrame(
        [
            {"项目": " 问题三基线", "说明": " 问题三建立在问题二限行政策调度结果之上，触发时刻前路线固定，触发时刻后采用动态重调度。"},
            {"项目": " 基础突发事件类型", "说明": " 订单取消、新增订单、配送地址变更、时间窗调整、车辆故障"},
            {"项目": " 事件类别数", "说明": " 5"},
            {"项目": " 叠加事件场景数", "说明": " 0"},
            {"项目": " 基准总成本", "说明": total_cost},
            {"项目": " 基准碳排放成本", "说明": carbon_cost},
            {"项目": " 基准客户数", "说明": int(demand_df[" 客户编号"].nunique())},
            {"项目": " 随机种子", "说明": RANDOM_SEED},
        ]
    )

def main():
    print(" 读取问题二结果，生成问题三五类基础突发事件场景...")
    summary_df, detail_df, overview_df, coords, time_windows, demand_df = load_baseline()
    scene_df, event_df, candidate_df = generate_scenarios(summary_df, detail_df, coords, time_windows)
    desc_df = build_description(scene_df, event_df, overview_df, demand_df)

    with pd.ExcelWriter(SCENE_BOOK_PATH, engine="openpyxl") as writer:
        scene_df.to_excel(writer, sheet_name=" 情景总览", index=False)
        event_df.to_excel(writer, sheet_name=" 事件明细", index=False)
        candidate_df.to_excel(writer, sheet_name=" 候选客户池", index=False)

    with pd.ExcelWriter(SCENE_DESC_PATH, engine="openpyxl") as writer:
        desc_df.to_excel(writer, sheet_name=" 情景设计说明", index=False)
        scene_df.to_excel(writer, sheet_name=" 情景摘要", index=False)
        (
            event_df.groupby([" 场景编号", " 事件类型"], as_index=False)
            .size()
            .rename(columns={"size": " 数量"})
            .to_excel(writer, sheet_name=" 事件结构", index=False)
        )

    print(f" 情景事件文件已输出: {os.path.relpath(SCENE_BOOK_PATH, BASE_DIR)}")
    print(f" 情景说明文件已输出: {os.path.relpath(SCENE_DESC_PATH, BASE_DIR)}")

```

```

print(scene_df[[" 场景编号", " 场景名称", " 触发时间", " 事件数量", " 场景模式"]].to_string(index=False))

if __name__ == "__main__":
    main()

```

B.5 问题三求解程序：3-问题三.py

```

import importlib.util
import os
from collections import defaultdict
from copy import deepcopy

import numpy as np
import pandas as pd

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
PROC_DIR = os.path.join(BASE_DIR, " 过程")
os.makedirs(PROC_DIR, exist_ok=True)

Q1_PATH = os.path.join(BASE_DIR, "1-问题一.py")
Q2_PATH = os.path.join(BASE_DIR, "2-问题二.py")
Q2_RESULT_PATH = os.path.join(PROC_DIR, "2-问题二限行政策调度方案.xlsx")
SCENE_BOOK_PATH = os.path.join(PROC_DIR, "3-问题三情景事件.xlsx")

OUT_PATH = os.path.join(PROC_DIR, "3-问题三动态调度方案.xlsx")
COMPARE_PATH = os.path.join(PROC_DIR, "3-问题三事件前后对比.xlsx")

ROUTE_EVAL_CACHE = {}
LATE_REPAIR_THRESHOLD = 2.0
MAX_LATE_ROUTE_REPAIRS = 6

def load_module(module_name, path):
    spec = importlib.util.spec_from_file_location(module_name, path)
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module

q1 = load_module("q1_for_q3_clean", Q1_PATH)
q2 = load_module("q2_for_q3_clean", Q2_PATH)

def parse_hour(value):
    return q1.parse_time_to_hour(value)

def hour_to_time(value):
    return q1.hour_to_time(value)

def route_load(stops):
    return sum(stop["weight"] for stop in stops), sum(stop["volume"] for stop in stops)

def euclidean_distance(a, b):
    return float(((a["x"] - b["x"]) ** 2 + (a["y"] - b["y"]) ** 2) ** 0.5)

def load_inputs():
    q2_book = pd.read_excel(Q2_RESULT_PATH, sheet_name=None)
    scene_book = pd.read_excel(SCENE_BOOK_PATH, sheet_name=None)
    dist_matrix = q1.read_distance_matrix()
    q1.dist_matrix_global = dist_matrix
    q2.q1.dist_matrix_global = dist_matrix
    time_windows = q1.read_time_windows()
    coords = q1.read_coordinates()

```

```

speed_scenarios = q1.generate_speed_scenarios()
return q2_book, scene_book, dist_matrix, time_windows, coords, speed_scenarios

def build_baseline_routes(summary_df, detail_df, dist_matrix, time_windows, speed_scenarios, coords):
    routes = []
    for _, row in summary_df.sort_values(" 路线编号").iterrows():
        route_id = row[" 路线编号"]
        route_detail = detail_df[detail_df[" 路线编号"] == route_id].sort_values(" 访问顺序")
        stops = []
        for _, stop_row in route_detail.iterrows():
            task_ids = []
            if " 包含拆分任务" in stop_row and pd.notna(stop_row[" 包含拆分任务"]):
                task_ids = [item.strip() for item in str(stop_row[" 包含拆分任务"]).split(",") if item.strip()]
            stops.append(
                {
                    "customer": int(stop_row[" 客户编号"]),
                    "weight": float(stop_row[" 配送重量 (kg)"]),
                    "volume": float(stop_row[" 配送体积 (m3)"]),
                    "task_ids": task_ids or [f"{int(stop_row[' 客户编号'])}_base"],
                }
            )
        cost = q2.evaluate_route_policy(
            q1.normalize_route_stops(stops),
            row[" 车辆类型"],
            dist_matrix,
            time_windows,
            speed_scenarios,
            coords,
            with_detail=True,
        )
        routes.append(
            {
                "route_id": route_id,
                "vehicle": row[" 车辆类型"],
                "vehicle_no": row[" 车辆编号"],
                "stops": q1.normalize_route_stops(stops),
                "cost": cost,
            }
        )
    return routes

def lexical_metrics(cost_result, vehicle_name, vehicle_status=None):
    if cost_result is None:
        return (10**9, 10**9, 10**9, 10**9, 10**9)
    release_hour = 0.0 if vehicle_status is None else float(vehicle_status.get("available_hour", q1.START_HOUR))
    return (
        int(cost_result.get("policy_violation_count", 0)),
        float(cost_result["late_hours"] > 1e-9),
        round(float(cost_result["late_hours"]), 6),
        round(float(cost_result["total"]), 6),
        round(float(cost_result["carbon"] + 0.01 * release_hour), 6),
    )

def better_cost(cost_a, cost_b, vehicle_name, vehicle_status=None):
    return lexical_metrics(cost_a, vehicle_name, vehicle_status) < lexical_metrics(cost_b, vehicle_name, vehicle_status)

def renumber_routes(routes):
    for idx, route in enumerate(routes, start=1):
        route["route_id"] = f"R{idx:03d}"
    return routes

def evaluate_route_with_status(stops, vehicle_name, dist_matrix, time_windows, speed_scenarios, coords, vehicle_status=None,
↪ with_detail=False):
    if not stops:

```

```

depart = q1.START_HOUR if vehicle_status is None else float(vehicle_status.get("available_hour", q1.START_HOUR))
return {
    "depart": depart,
    "return": depart,
    "start": 0.0,
    "energy": 0.0,
    "carbon": 0.0,
    "time_window": 0.0,
    "wait_hours": 0.0,
    "late_hours": 0.0,
    "total": 0.0,
    "policy_violation_count": 0,
    "details": [] if with_detail else None,
}

available_hour = q1.START_HOUR if vehicle_status is None else float(vehicle_status.get("available_hour", q1.START_HOUR))
cache_key = None
if not with_detail:
    cache_key = (
        tuple((int(stop["customer"]), round(float(stop["weight"]), 4), round(float(stop["volume"]), 4)) for stop in stops),
        vehicle_name,
        round(float(available_hour), 4),
    )
    if cache_key in ROUTE_EVAL_CACHE:
        return deepcopy(ROUTE_EVAL_CACHE[cache_key])

vehicle = q1.VEHICLES[vehicle_name]
load_w, load_v = route_load(stops)
if load_w > vehicle["cap_w"] + 1e-8 or load_v > vehicle["cap_v"] + 1e-8:
    return None

lower, upper = q1.estimate_departure_bounds(stops, dist_matrix, time_windows)
start = max(q1.START_HOUR, available_hour, lower)

candidate_departures = {round(start, 6)}
current = np.floor(start / q1.DEPARTURE_STEP) * q1.DEPARTURE_STEP
while current < start - 1e-9:
    current += q1.DEPARTURE_STEP
while current <= max(start, upper) + 1e-9:
    candidate_departures.add(round(current, 6))
    current += q1.DEPARTURE_STEP

if vehicle["power"] == "fuel":
    green_set = q2.green_customers(coords)
    if any(stop["customer"] in green_set for stop in stops):
        current = max(start, q2.FUEL_BAN_END)
        while current <= 22.0 + 1e-9:
            candidate_departures.add(round(current, 6))
            current += q1.DEPARTURE_STEP

best = None
for depart in sorted(candidate_departures):
    if vehicle["power"] == "fuel" and not q2.is_policy_feasible(stops, vehicle_name, coords, time_windows, speed_scenarios,
        ↪ depart):
        continue
    result = q1.evaluate_route(stops, vehicle_name, dist_matrix, time_windows, speed_scenarios, with_detail,
        ↪ depart_hour=depart)
    if result is None:
        continue
    result["depart"] = depart
    result["policy_violation_count"] = 0
    if best is None or better_cost(result, best, vehicle_name, vehicle_status):
        best = result

if best is None:
    return None

if cache_key is not None:
    ROUTE_EVAL_CACHE[cache_key] = deepcopy(best)

```

```

return best

def improve_route_sequence_service(route, dist_matrix, time_windows, speed_scenarios, coords, vehicle_status=None):
    stops = q1.normalize_route_stops(route["stops"])
    if len(stops) <= 1:
        route["stops"] = stops
        route["cost"] = evaluate_route_with_status(
            stops,
            route["vehicle"],
            dist_matrix,
            time_windows,
            speed_scenarios,
            coords,
            vehicle_status=vehicle_status,
        )
        return route

    best = stops[:]
    best_cost = evaluate_route_with_status(
        best,
        route["vehicle"],
        dist_matrix,
        time_windows,
        speed_scenarios,
        coords,
        vehicle_status=vehicle_status,
    )
    if best_cost is None:
        return route

    improved = True
    passes = 0
    while improved and passes < q1.LOCAL_SEARCH_PASSES:
        improved = False
        passes += 1
        n = len(best)
        candidates = []
        for i in range(n - 1):
            for j in range(i + 1, n):
                cand1 = best[:]
                cand1[i], cand1[j] = cand1[j], cand1[i]
                candidates.append(q1.normalize_route_stops(cand1))
                cand2 = best[:i] + list(reversed(best[i : j + 1])) + best[j + 1 :]
                candidates.append(q1.normalize_route_stops(cand2))
        for i in range(n):
            for j in range(n):
                if i == j:
                    continue
                cand = best[:]
                moved = cand.pop(i)
                cand.insert(j, moved)
                candidates.append(q1.normalize_route_stops(cand))

        best_candidate = None
        best_candidate_cost = None
        for cand in candidates:
            result = evaluate_route_with_status(
                cand,
                route["vehicle"],
                dist_matrix,
                time_windows,
                speed_scenarios,
                coords,
                vehicle_status=vehicle_status,
            )
            if result is None:
                continue
            if best_candidate_cost is None or better_cost(result, best_candidate_cost, route["vehicle"], vehicle_status):

```



```

        best_candidate = cand
        best_candidate_cost = result

    if best_candidate_cost is not None and better_cost(best_candidate_cost, best_cost, route["vehicle"], vehicle_status):
        best = best_candidate
        best_cost = best_candidate_cost
        improved = True

    route["stops"] = best
    route["cost"] = best_cost
    return route

def ensure_customer(time_windows, coords, customer_id, event_row):
    customer_id = int(customer_id)
    if customer_id not in coords:
        coords[customer_id] = {
            "x": float(event_row[" 新 X(km)"]),
            "y": float(event_row[" 新 Y(km)"]),
            "type": " 情景新增客户",
        }
    if customer_id not in time_windows:
        time_windows[customer_id] = {
            "early": parse_hour(event_row[" 新时间窗开始"]),
            "late": parse_hour(event_row[" 新时间窗结束"]),
        }

def build_dynamic_distance_matrix(base_dist_matrix, coords, event_df):
    impacted = set()
    for _, event in event_df.iterrows():
        event_type = str(event[" 事件类型"])
        if event_type == "new_order" and pd.notna(event[" 虚拟客户编号"]):
            impacted.add(int(event[" 虚拟客户编号"]))
        if event_type == "address_change" and pd.notna(event[" 目标客户编号"]):
            impacted.add(int(event[" 目标客户编号"]))

    size = max(base_dist_matrix.shape[0], max(coords) + 1)
    matrix = np.zeros((size, size), dtype=float)
    matrix[: base_dist_matrix.shape[0], : base_dist_matrix.shape[1]] = base_dist_matrix

    for cid in impacted:
        for other in coords:
            dist = euclidean_distance(coords[cid], coords[other])
            matrix[cid, other] = dist
            matrix[other, cid] = dist
        matrix[cid, cid] = 0.0
    return matrix

def get_route_state_at_trigger(route, trigger_hour):
    details = route["cost"].get("details", []) if route.get("cost") else []
    if not details:
        status = "unstarted" if route["cost"]["depart"] >= trigger_hour - 1e-9 else "finished"
        return {
            "status": status,
            "available_hour": float(trigger_hour) if status == "unstarted" else route["cost"]["return"],
            "current_loc": 0,
            "remaining_stops": deepcopy(route["stops"]) if status == "unstarted" else [],
            "served_stops": [] if status == "unstarted" else deepcopy(route["stops"]),
        }

    if route["cost"]["depart"] >= trigger_hour - 1e-9:
        return {
            "status": "unstarted",
            "available_hour": float(trigger_hour),
            "current_loc": 0,
            "remaining_stops": deepcopy(route["stops"]),
            "served_stops": [],
        }

```

```

    }

    if route["cost"]["return"] <= trigger_hour + 1e-9:
        return {
            "status": "finished",
            "available_hour": route["cost"]["return"],
            "current_loc": 0,
            "remaining_stops": [],
            "served_stops": deepcopy(route["stops"]),
        }

    served_stops = []
    current_loc = 0
    release_hour = trigger_hour
    first_unfinished_idx = 0

    for idx, (stop, detail) in enumerate(zip(route["stops"], details)):
        if detail["departure"] <= trigger_hour + 1e-9:
            served_stops.append(deepcopy(stop))
            current_loc = stop["customer"]
            first_unfinished_idx = idx + 1
            continue
        if detail["arrival"] <= trigger_hour < detail["departure"]:
            served_stops.append(deepcopy(stop))
            current_loc = stop["customer"]
            release_hour = detail["departure"]
            first_unfinished_idx = idx + 1
        break

    remaining_stops = [deepcopy(item) for item in route["stops"][first_unfinished_idx:]]
    return {
        "status": "in_progress",
        "available_hour": max(trigger_hour, release_hour),
        "current_loc": current_loc,
        "remaining_stops": remaining_stops,
        "served_stops": served_stops,
    }

def summarize_realtime_state(baseline_routes, trigger_hour):
    state_rows = []
    vehicle_states = {}
    for route in baseline_routes:
        state = get_route_state_at_trigger(route, trigger_hour)
        vehicle_states[route["vehicle_no"]] = {
            "route_id": route["route_id"],
            "vehicle_no": route["vehicle_no"],
            "vehicle": route["vehicle"],
            "status": state["status"],
            "available_hour": float(state["available_hour"]),
            "current_loc": int(state["current_loc"]),
            "remaining_stops": deepcopy(state["remaining_stops"]),
            "served_stops": deepcopy(state["served_stops"]),
        }
    state_rows.append(
        {
            "路线编号": route["route_id"],
            "车辆编号": route["vehicle_no"],
            "车辆类型": route["vehicle"],
            "触发时状态": state["status"],
            "触发时可释放时间": hour_to_time(state["available_hour"]),
            "触发时位置": state["current_loc"],
            "已完成客户数": len(state["served_stops"]),
            "剩余客户数": len(state["remaining_stops"]),
            "剩余路径": "0" if not state["remaining_stops"] else " -> ".join(str(stop["customer"]) for stop in
            ↪ state["remaining_stops"]),
        }
    )
    return pd.DataFrame(state_rows), vehicle_states

```

```

def enrich_with_idle_fleet(vehicle_states, trigger_hour):
    used_count = defaultdict(int)
    for state in vehicle_states.values():
        used_count[state["vehicle"]] += 1

    for vehicle_name, spec in q1.VEHICLES.items():
        idle_count = max(0, int(spec["count"]) - used_count[vehicle_name])
        for idx in range(idle_count):
            vehicle_no = f" 备用-{vehicle_name}-{idx + 1:02d}"
            vehicle_states[vehicle_no] = {
                "route_id": "IDLE",
                "vehicle_no": vehicle_no,
                "vehicle": vehicle_name,
                "status": "idle",
                "available_hour": float(trigger_hour),
                "current_loc": 0,
                "remaining_stops": [],
                "served_stops": [],
            }
    return vehicle_states

def build_fixed_part(baseline_routes, trigger_hour, dist_matrix):
    fixed_routes = []
    fixed_total = {
        "fixed_route_count": 0,
        "fixed_total_cost": 0.0,
        "fixed_carbon_cost": 0.0,
        "fixed_time_window_cost": 0.0,
        "fixed_total_distance": 0.0,
        "fixed_total_wait": 0.0,
        "fixed_total_late": 0.0,
    }

    for route in baseline_routes:
        state = get_route_state_at_trigger(route, trigger_hour)
        if state["status"] in {"finished", "in_progress"}:
            fixed_routes.append(deepcopy(route))
            fixed_total["fixed_route_count"] += 1
            fixed_total["fixed_total_cost"] += float(route["cost"]["total"])
            fixed_total["fixed_carbon_cost"] += float(route["cost"]["carbon"])
            fixed_total["fixed_time_window_cost"] += float(route["cost"]["time_window"])
            fixed_total["fixed_total_distance"] += float(q1.route_distance(route["stops"], dist_matrix))
            fixed_total["fixed_total_wait"] += float(route["cost"]["wait_hours"])
            fixed_total["fixed_total_late"] += float(route["cost"]["late_hours"])

    for key in fixed_total:
        if key == "fixed_route_count":
            continue
        fixed_total[key] = round(fixed_total[key], 4 if ("wait" in key or "late" in key) else 2)
    return fixed_routes, fixed_total

def build_candidate_demand_with_state(baseline_routes, vehicle_states):
    rows = []
    for route in baseline_routes:
        state = vehicle_states[route["vehicle_no"]]
        if state["status"] != "unstarted":
            continue
        for stop in state["remaining_stops"]:
            rows.append({" 客户编号": stop["customer"], " 重量": stop["weight"], " 体积": stop["volume"]})
    if not rows:
        return pd.DataFrame(columns=[" 客户编号", " 重量", " 体积"])
    return pd.DataFrame(rows).groupby(" 客户编号", as_index=False)[[" 重量", " 体积"]].sum()

def parse_breakdown_hours(extra_info):

```

```

if pd.isna(extra_info):
    return 24.0
text = str(extra_info)
marker = " 预计停运时长 (h)="
if marker not in text:
    return 24.0
try:
    return max(0.5, float(text.split(marker, 1)[1].split(";")[0]))
except ValueError:
    return 24.0

def apply_events(candidate_df, event_df, time_windows, coords, vehicle_states, trigger_hour):
    demand_df = candidate_df.copy()
    logs = []
    for _, event in event_df.sort_values(" 事件顺序").iterrows():
        event_type = str(event[" 事件类型"])
        if event_type == "cancel_order":
            cid = int(event[" 目标客户编号"])
            before_weight = float(demand_df.loc[demand_df[" 客户编号"] == cid, " 重量"].sum())
            before_volume = float(demand_df.loc[demand_df[" 客户编号"] == cid, " 体积"].sum())
            demand_df = demand_df[demand_df[" 客户编号"] != cid].copy()
            logs.append(
                {
                    " 事件类型": event_type,
                    " 客户编号": cid,
                    " 变更前重量 (kg)": round(before_weight, 4),
                    " 变更后重量 (kg)": 0.0,
                    " 变更前体积 (m3)": round(before_volume, 4),
                    " 变更后体积 (m3)": 0.0,
                    " 说明": event[" 说明"],
                }
            )
        elif event_type == "time_window_adjustment":
            cid = int(event[" 目标客户编号"])
            time_windows[cid] = {
                "early": parse_hour(event[" 新时间窗开始"]),
                "late": parse_hour(event[" 新时间窗结束"]),
            }
            current_weight = float(demand_df.loc[demand_df[" 客户编号"] == cid, " 重量"].sum())
            current_volume = float(demand_df.loc[demand_df[" 客户编号"] == cid, " 体积"].sum())
            logs.append(
                {
                    " 事件类型": event_type,
                    " 客户编号": cid,
                    " 变更前重量 (kg)": round(current_weight, 4),
                    " 变更后重量 (kg)": round(current_weight, 4),
                    " 变更前体积 (m3)": round(current_volume, 4),
                    " 变更后体积 (m3)": round(current_volume, 4),
                    " 说明": event[" 说明"],
                }
            )
        elif event_type == "address_change":
            cid = int(event[" 目标客户编号"])
            coords[cid]["x"] = float(event[" 新 X(km)"])
            coords[cid]["y"] = float(event[" 新 Y(km)"])
            current_weight = float(demand_df.loc[demand_df[" 客户编号"] == cid, " 重量"].sum())
            current_volume = float(demand_df.loc[demand_df[" 客户编号"] == cid, " 体积"].sum())
            logs.append(
                {
                    " 事件类型": event_type,
                    " 客户编号": cid,
                    " 变更前重量 (kg)": round(current_weight, 4),
                    " 变更后重量 (kg)": round(current_weight, 4),
                    " 变更前体积 (m3)": round(current_volume, 4),
                    " 变更后体积 (m3)": round(current_volume, 4),
                    " 说明": event[" 说明"],
                }
            )
    )

```

```

elif event_type == "new_order":
    cid = int(event[" 虚拟客户编号"])
    ensure_customer(time_windows, coords, cid, event)
    demand_df = pd.concat(
        [
            demand_df,
            pd.DataFrame([{"客户编号": cid, "重量": float(event["新重量(kg)"]), "体积": float(event["新体积(m3)"])}]),
        ],
        ignore_index=True,
    )
    demand_df = demand_df.groupby(" 客户编号", as_index=False)[[" 重量", " 体积"]].sum()
    logs.append(
        {
            " 事件类型": event_type,
            " 客户编号": cid,
            " 变更前重量 (kg)": 0.0,
            " 变更后重量 (kg)": round(float(event[" 新重量 (kg)"]), 4),
            " 变更前体积 (m3)": 0.0,
            " 变更后体积 (m3)": round(float(event[" 新体积 (m3)"]), 4),
            " 说明": event[" 说明"],
        }
    )
elif event_type == "vehicle_breakdown":
    vehicle_no = str(event[" 目标车辆编号"])
    breakdown_hours = parse_breakdown_hours(event.get(" 附加信息", ""))
    release_hour = min(24.0, float(trigger_hour) + breakdown_hours)
    state = vehicle_states.get(vehicle_no)
    if state is not None:
        state["status"] = "breakdown"
        state["available_hour"] = release_hour
        state["breakdown_until"] = release_hour
    cid = int(event[" 目标客户编号"]) if pd.notna(event[" 目标客户编号"]) else np.nan
    logs.append(
        {
            " 事件类型": event_type,
            " 客户编号": cid,
            "变更前重量(kg)": round(float(demand_df.loc[demand_df["客户编号"] == cid, "重量"].sum()), 4) if
            ↪ pd.notna(cid) else 0.0,
            "变更后重量(kg)": round(float(demand_df.loc[demand_df["客户编号"] == cid, "重量"].sum()), 4) if
            ↪ pd.notna(cid) else 0.0,
            "变更前体积(m3)": round(float(demand_df.loc[demand_df["客户编号"] == cid, "体积"].sum()), 4) if
            ↪ pd.notna(cid) else 0.0,
            "变更后体积(m3)": round(float(demand_df.loc[demand_df["客户编号"] == cid, "体积"].sum()), 4) if
            ↪ pd.notna(cid) else 0.0,
            " 说明": f"{event['说明']} 故障释放时间调整为 {hour_to_time(release_hour)}。",
        }
    )
else:
    raise ValueError(f" 未知事件类型: {event_type}")
return demand_df, pd.DataFrame(logs)

def build_dynamic_vehicle_pool(vehicle_states):
    pool = {}
    for vehicle_no, state in vehicle_states.items():
        if state["status"] in {"in_progress"}:
            continue
        pool[vehicle_no] = {
            "vehicle_no": vehicle_no,
            "vehicle": state["vehicle"],
            "available_hour": float(state["available_hour"]),
            "current_loc": int(state["current_loc"]),
            "status": state["status"],
        }
    return pool

def available_vehicle_pool_after_assignment(routes, vehicle_states):
    pool = build_dynamic_vehicle_pool(vehicle_states)

```

```

used = {route["vehicle_no"] for route in routes}
return {vehicle_no: info for vehicle_no, info in pool.items() if vehicle_no not in used}

def assign_split_routes(split_stops_list, free_vehicle_pool, dist_matrix, time_windows, speed_scenarios, coords):
    if not split_stops_list:
        return []
    vehicle_items = sorted(free_vehicle_pool.items(), key=lambda item: item[0])
    route_count = len(split_stops_list)
    vehicle_count = len(vehicle_items)
    if vehicle_count < route_count:
        return None

    source = 0
    route_offset = 1
    vehicle_offset = route_offset + route_count
    sink = vehicle_offset + vehicle_count
    graph = [[] for _ in range(sink + 1)]

    for ridx in range(route_count):
        q2.add_flow_edge(graph, source, route_offset + ridx, 1, 0)
    for vidx in range(vehicle_count):
        q2.add_flow_edge(graph, vehicle_offset + vidx, sink, 1, 0)

    edge_map = {}
    route_cost_cache = {}
    for ridx, stops in enumerate(split_stops_list):
        for vidx, (vehicle_no, status) in enumerate(vehicle_items):
            result = evaluate_route_with_status(
                stops,
                status["vehicle"],
                dist_matrix,
                time_windows,
                speed_scenarios,
                coords,
                vehicle_status=status,
            )
            if result is None:
                continue
            metrics = lexical_metrics(result, status["vehicle"], status)
            scaled_cost = int(metrics[0]) * 10**13
            scaled_cost += int(metrics[1]) * 10**12
            scaled_cost += int(round(metrics[2] * 10**5)) * 10**6
            scaled_cost += int(round(result["depart"] * 1000)) * 10**2
            scaled_cost += int(round(metrics[3] * 10))
            edge_map[(ridx, vidx)] = (route_offset + ridx, len(graph[route_offset + ridx]))
            route_cost_cache[(ridx, vidx)] = result
            q2.add_flow_edge(graph, route_offset + ridx, vehicle_offset + vidx, 1, scaled_cost)

    flow, _ = q2.min_cost_max_flow(graph, source, sink, route_count)
    if flow < route_count:
        return None

    assigned = []
    for ridx, stops in enumerate(split_stops_list, start=1):
        chosen = None
        for vidx, (vehicle_no, status) in enumerate(vehicle_items):
            key = (ridx - 1, vidx)
            if key not in edge_map:
                continue
            u, edge_idx = edge_map[key]
            if graph[u][edge_idx].cap == 0:
                chosen = (vehicle_no, status, route_cost_cache[key])
                break
        if chosen is None:
            return None
        vehicle_no, status, result = chosen
        assigned.append(
            {

```

```

        "route_id": f"RS{ridx:03d}",
        "vehicle_no": vehicle_no,
        "vehicle": status["vehicle"],
        "stops": q1.normalize_route_stops(stops),
        "cost": result,
        "realtime_status": status["available_hour"],
        "reserved_vehicle": " 否",
    }
)
return assigned

def repair_high_late_routes(routes, vehicle_states, dist_matrix, time_windows, speed_scenarios, coords):
    repaired = [deepcopy(route) for route in routes]
    repair_count = 0

    while repair_count < MAX_LATE_ROUTE_REPAIRS:
        candidates = sorted(
            [
                route
                for route in repaired
                if float(route["cost"]["late_hours"]) > LATE_REPAIR_THRESHOLD and len(route["stops"]) >= 2
            ],
            key=lambda route: (-float(route["cost"]["late_hours"]), -len(route["stops"]), -float(route["cost"]["total"])),
        )
        if not candidates:
            break

        base_route = candidates[0]
        base_idx = next(i for i, item in enumerate(repaired) if item["route_id"] == base_route["route_id"])
        free_pool = available_vehicle_pool_after_assignment(repaired[:base_idx] + repaired[base_idx + 1 :], vehicle_states)
        if not free_pool:
            break

        best_plan = None
        original_score = (
            round(float(base_route["cost"]["late_hours"]), 6),
            round(float(base_route["cost"]["total"]), 6),
        )

        stops = q1.normalize_route_stops(base_route["stops"])
        for cut in range(1, len(stops)):
            left = q1.normalize_route_stops(stops[:cut])
            right = q1.normalize_route_stops(stops[cut:])
            split_assigned = assign_split_routes([left, right], free_pool, dist_matrix, time_windows, speed_scenarios, coords)
            if split_assigned is None:
                continue
            split_assigned = [
                improve_route_sequence_service(
                    route,
                    dist_matrix,
                    time_windows,
                    speed_scenarios,
                    coords,
                    vehicle_status={"available_hour": route["realtime_status"]},
                )
                for route in split_assigned
            ]
            score = (
                round(sum(float(route["cost"]["late_hours"]) for route in split_assigned), 6),
                round(sum(float(route["cost"]["total"]) for route in split_assigned), 6),
            )
            if score[0] + 1e-9 < original_score[0] and (best_plan is None or score < best_plan[0]):
                best_plan = (score, split_assigned)

        if best_plan is None:
            break

    repaired.pop(base_idx)

```

```

        repaired.extend(best_plan[1])
        repaired.sort(key=lambda route: (float(route.get("realtime_status", q1.START_HOUR)), float(route["cost"]["depart"]),
        ↪ route["vehicle_no"]))
        repaired = renumber_routes(repaired)
        repair_count += 1

    return repaired

def build_routes_for_dynamic(demand_df, dist_matrix, time_windows, coords, speed_scenarios, vehicle_states):
    if demand_df.empty:
        return [], {"before": 0.0, "after": 0.0, "lexical_metrics": (0, 0, 0.0, 0.0, 0.0)}

    customer_demands = demand_df.rename(columns={" 客户编号": " 目标客户编号", " 重量": " 重量", " 体积": " 体积"})
    vehicle_pool = build_dynamic_vehicle_pool(vehicle_states)
    remaining_counts = defaultdict(int)
    for info in vehicle_pool.values():
        remaining_counts[info["vehicle"]] += 1

    old_counts = {name: spec["count"] for name, spec in q1.VEHICLES.items()}
    old_order = q1.FLEET_ORDER[:]
    try:
        for name, spec in q1.VEHICLES.items():
            spec["count"] = int(remaining_counts.get(name, 0))
        q1.FLEET_ORDER = [name for name in q2.Q2_FLEET_ORDER if remaining_counts.get(name, 0) > 0]
        seed_routes = q1.build_capacity_focused_routes(customer_demands, dist_matrix, time_windows)
        seed_routes = [{"stops": q1.normalize_route_stops(route["stops"])} for route in seed_routes]
    finally:
        q1.FLEET_ORDER = old_order
        for name, count in old_counts.items():
            q1.VEHICLES[name]["count"] = count

    before = 0.0
    for route in seed_routes:
        best_cost = None
        for vehicle_name in q1.VEHICLES:
            result = evaluate_route_with_status(
                route["stops"],
                vehicle_name,
                dist_matrix,
                time_windows,
                speed_scenarios,
                coords,
                vehicle_status={"available_hour": q1.START_HOUR},
            )
            if result is None:
                continue
            if best_cost is None or better_cost(result, best_cost, vehicle_name, {"available_hour": q1.START_HOUR}):
                best_cost = result
        if best_cost is None:
            raise RuntimeError(" 问题三: 动态建路失败, 存在路线在现有车辆类型下不可行。")
        before += float(best_cost["total"])
    before = round(before, 2)

    route_count = len(seed_routes)
    vehicle_items = sorted(vehicle_pool.items(), key=lambda item: item[0])
    vehicle_count = len(vehicle_items)
    source = 0
    route_offset = 1
    vehicle_offset = route_offset + route_count
    sink = vehicle_offset + vehicle_count
    graph = [[] for _ in range(sink + 1)]

    for ridx in range(route_count):
        q2.add_flow_edge(graph, source, route_offset + ridx, 1, 0)
    for vidx in range(vehicle_count):
        q2.add_flow_edge(graph, vehicle_offset + vidx, sink, 1, 0)

    edge_map = {}

```



```

route_cost_cache = {}
ordered_routes = sorted(
    seed_routes,
    key=lambda route: (
        min(time_windows.get(stop["customer"], {"late": 24.0})["late"] for stop in route["stops"]),
        0 if q2.route_has_green_customer(route["stops"], coords) else 1,
        -sum(stop["volume"] for stop in route["stops"]),
        -sum(stop["weight"] for stop in route["stops"]),
    ),
)

for ridx, route in enumerate(ordered_routes):
    for vidx, (vehicle_no, status) in enumerate(vehicle_items):
        result = evaluate_route_with_status(
            route["stops"],
            status["vehicle"],
            dist_matrix,
            time_windows,
            speed_scenarios,
            coords,
            vehicle_status=status,
        )
        if result is None:
            continue
        metrics = lexical_metrics(result, status["vehicle"], status)
        scaled_cost = int(metrics[0]) * 10**13
        scaled_cost += int(metrics[1]) * 10**12
        scaled_cost += int(round(metrics[2] * 10**5)) * 10**6
        scaled_cost += int(round(result["depart"] * 1000)) * 10**2
        scaled_cost += int(round(metrics[3] * 10))
        edge_map[(ridx, vidx)] = (route_offset + ridx, len(graph[route_offset + ridx]))
        route_cost_cache[(ridx, vidx)] = result
        q2.add_flow_edge(graph, route_offset + ridx, vehicle_offset + vidx, 1, scaled_cost)

flow, _ = q2.min_cost_max_flow(graph, source, sink, route_count)
if flow < route_count:
    raise RuntimeError(" 问题三: 动态重调度失败, 剩余车辆池无法承接当前建路结果。")

mapped_routes = []
for ridx, route in enumerate(ordered_routes, start=1):
    chosen = None
    for vidx, (vehicle_no, status) in enumerate(vehicle_items):
        key = (ridx - 1, vidx)
        if key not in edge_map:
            continue
        u, edge_idx = edge_map[key]
        if graph[u][edge_idx].cap == 0:
            chosen = (vehicle_no, status, route_cost_cache[key])
            break
    if chosen is None:
        raise RuntimeError(" 问题三: 动态重调度匹配结果异常。")
    vehicle_no, status, result = chosen
    mapped_routes.append(
        {
            "route_id": f"R{ridx:03d}",
            "vehicle_no": vehicle_no,
            "vehicle": status["vehicle"],
            "stops": q1.normalize_route_stops(route["stops"]),
            "cost": result,
            "realtime_status": status["available_hour"],
            "reserved_vehicle": " 否",
        }
    )

improved_routes = []
for route in mapped_routes:
    vehicle_status = {"available_hour": route["realtime_status"]}
    improved_routes.append(
        improve_route_sequence_service(

```

```

        deepcopy(route),
        dist_matrix,
        time_windows,
        speed_scenarios,
        coords,
        vehicle_status=vehicle_status,
    )
)

improved_routes = repair_high_late_routes(
    improved_routes,
    vehicle_states,
    dist_matrix,
    time_windows,
    speed_scenarios,
    coords,
)
improved_routes.sort(key=lambda route: (float(route.get("realtime_status", q1.START_HOUR)), float(route["cost"]["depart"])),
↪ route["vehicle_no"]))
improved_routes = renumber_routes(improved_routes)

total_metrics = (
    int(sum(route["cost"].get("policy_violation_count", 0) for route in improved_routes)),
    int(sum(1 for route in improved_routes if route["cost"]["late_hours"] > 1e-9)),
    round(sum(route["cost"]["late_hours"] for route in improved_routes), 6),
    round(sum(route["cost"]["total"] for route in improved_routes), 6),
    round(sum(route["cost"]["carbon"] for route in improved_routes), 6),
)
after = round(sum(route["cost"]["total"] for route in improved_routes), 2)
return renumber_routes(improved_routes), {"before": before, "after": after, "lexical_metrics": total_metrics}

def build_output_tables(routes, dist_matrix, time_windows, speed_scenarios, coords):
    green_set = q2.green_customers(coords)
    summary_rows = []
    detail_rows = []
    type_rows = []
    total = defaultdict(float)

    for route in routes:
        detail_cost = evaluate_route_with_status(
            route["stops"],
            route["vehicle"],
            dist_matrix,
            time_windows,
            speed_scenarios,
            coords,
            vehicle_status={"available_hour": route.get("realtime_status", route["cost"]["depart"])},
            with_detail=True,
        )
        route["cost"] = detail_cost
        load_w, load_v = route_load(route["stops"])
        distance = q1.route_distance(route["stops"], dist_matrix)
        summary_rows.append(
            {
                "路线编号": route["route_id"],
                "车辆编号": route["vehicle_no"],
                "车辆类型": route["vehicle"],
                "是否含绿色区客户": "是" if q2.route_has_green_customer(route["stops"], coords) else "否",
                "是否预见性保留车辆": route.get("reserved_vehicle", "否"),
                "车辆释放时间": hour_to_time(route.get("realtime_status", detail_cost["depart"])),
                "动态等待发车(h)": round(max(0.0, float(detail_cost["depart"]) - float(route.get("realtime_status",
↪ detail_cost["depart"])))), 4),
                "发车时间": hour_to_time(detail_cost["depart"]),
                "返场时间": hour_to_time(detail_cost["return"]),
                "载重 (kg)": round(load_w, 4),
                "体积 (m3)": round(load_v, 4),
                "载重率": round(load_w / q1.VEHICLES[route["vehicle"]]["cap_w"], 4),
                "容积率": round(load_v / q1.VEHICLES[route["vehicle"]]["cap_v"], 4),
            }
        )

```

```

        "行驶距离 (km)": round(distance, 4),
        "访问客户数": len(route["stops"]),
        "路径": "0 -> " + " -> ".join(str(stop["customer"]) for stop in route["stops"]) + " -> 0",
        "平均到达时间": ", ".join(hour_to_time(item["arrival"]) for item in detail_cost["details"]),
        "启动成本": round(detail_cost["start"], 2),
        "能源成本": round(detail_cost["energy"], 2),
        "碳排放成本": round(detail_cost["carbon"], 2),
        "碳排放量 (kg)": round(detail_cost["carbon"] / q1.CARBON_PRICE, 4),
        "时间窗成本": round(detail_cost["time_window"], 2),
        "等待总时长 (h)": round(detail_cost["wait_hours"], 4),
        "迟到总时长 (h)": round(detail_cost["late_hours"], 4),
        "总成本": round(detail_cost["total"], 2),
    }
)
)
for key in ["start", "energy", "carbon", "time_window", "total", "wait_hours", "late_hours"]:
    total[key] += float(detail_cost[key])

for stop, detail in zip(route["stops"], detail_cost["details"]):
    cid = stop["customer"]
    tw = time_windows.get(cid, {"early": q1.START_HOUR, "late": 24.0})
    fuel_forbidden = (
        q1.VEHICLES[route["vehicle"]]["power"] == "fuel"
        and cid in green_set
        and q2.FUEL_BAN_START <= detail["arrival"] < q2.FUEL_BAN_END
    )
    detail_rows.append(
        {
            "路线编号": route["route_id"],
            "车辆编号": route["vehicle_no"],
            "车辆类型": route["vehicle"],
            "访问顺序": detail["seq"],
            "客户编号": cid,
            "是否绿色区客户": "是" if cid in green_set else "否",
            "燃油限行是否违规": "是" if fuel_forbidden else "否",
            "配送重量 (kg)": round(stop["weight"], 4),
            "配送体积 (m3)": round(stop["volume"], 4),
            "时间窗开始": hour_to_time(tw["early"]),
            "时间窗结束": hour_to_time(tw["late"]),
            "平均到达": hour_to_time(detail["arrival"]),
            "平均服务开始": hour_to_time(detail["service_start"]),
            "平均离开": hour_to_time(detail["departure"]),
            "平均等待 (h)": round(detail["wait_hours"], 4),
            "平均迟到 (h)": round(detail["late_hours"], 4),
            "上一点到本点距离 (km)": round(detail["travel_distance"], 4),
            "包含拆分任务": ", ".join(stop["task_ids"]),
        }
    )
)

used = defaultdict(int)
for route in routes:
    used[route["vehicle"]] += 1
for name, spec in q1.VEHICLES.items():
    type_rows.append(
        {
            "车辆类型": name,
            "动力类型": "新能源" if spec["power"] == "ev" else "燃油",
            "可用数量": spec["count"],
            "使用数量": used[name],
            "剩余数量": spec["count"] - used[name],
            "载重上限 (kg)": spec["cap_w"],
            "容积上限 (m3)": spec["cap_v"],
        }
    )
)

summary_df = pd.DataFrame(summary_rows)
detail_df = pd.DataFrame(detail_rows)
type_df = pd.DataFrame(type_rows)
overview_df = pd.DataFrame(
    [

```

```

{"指标": "总路线数", "数值": int(len(summary_df))},
{"指标": "新能源车使用数", "数值": int(type_df[type_df["动力类型"] == "新能源"]["使用数量"].sum())},
{"指标": "燃油车使用数", "数值": int(type_df[type_df["动力类型"] == "燃油"]["使用数量"].sum())},
{"指标": "平均动态等待发车(h)", "数值": round(float(summary_df["动态等待发车(h)"].mean()), 4) if not summary_df.empty
↪ else 0.0},
{"指标": "总行驶距离(km)", "数值": round(float(summary_df["行驶距离(km)"].sum()), 2) if not summary_df.empty else
↪ 0.0},
{"指标": "启动成本", "数值": round(total["start"], 2)},
{"指标": "能源成本", "数值": round(total["energy"], 2)},
{"指标": "碳排放成本", "数值": round(total["carbon"], 2)},
{"指标": "碳排放总量(kg)", "数值": round(total["carbon"] / q1.CARBON_PRICE, 4)},
{"指标": "时间窗成本", "数值": round(total["time_window"], 2)},
{"指标": "总成本", "数值": round(total["total"], 2)},
{"指标": "总等待(h)", "数值": round(total["wait_hours"], 4)},
{"指标": "总迟到(h)", "数值": round(total["late_hours"], 4)},
{"指标": "限行违规次数", "数值": int((detail_df["燃油限行是否违规"] == "是").sum()) if not detail_df.empty else 0},
{"指标": "词典序目标-限行违规", "数值": int((detail_df["燃油限行是否违规"] == "是").sum()) if not detail_df.empty else
↪ 0},
{"指标": "词典序目标-迟到客户数", "数值": int((detail_df["平均迟到(h)"] > 0).sum()) if not detail_df.empty else 0},
{"指标": "词典序目标-总迟到(h)", "数值": round(total["late_hours"], 4)},
{"指标": "词典序目标-总成本", "数值": round(total["total"], 2)},
{"指标": "词典序目标-碳排放成本", "数值": round(total["carbon"], 2)},
{"指标": "速度场景数", "数值": q1.NUM_SPEED_SCENARIOS},
{"指标": "随机种子", "数值": q1.RANDOM_SEED},
]
)
return overview_df, type_df, summary_df, detail_df

def build_full_day_rows(base_overview, fixed_total, dynamic_overview):
    def ov(df, key):
        hit = df.loc[df["指标"] == key, "数值"]
        return float(hit.iloc[0]) if not hit.empty else 0.0

    rows = [
        {"指标": "问题二全天基准总成本", "数值": float(base_overview["总成本"])},
        {"指标": "问题二全天基准碳排放成本", "数值": float(base_overview["碳排放成本"])},
        {"指标": "已执行固定部分总成本", "数值": float(fixed_total["fixed_total_cost"])},
        {"指标": "已执行固定部分碳排放成本", "数值": float(fixed_total["fixed_carbon_cost"])},
        {"指标": "动态部分总成本", "数值": float(ov(dynamic_overview, "总成本"))},
        {"指标": "动态部分碳排放成本", "数值": float(ov(dynamic_overview, "碳排放成本"))},
    ]
    full_total = float(fixed_total["fixed_total_cost"]) + float(ov(dynamic_overview, "总成本"))
    full_carbon = float(fixed_total["fixed_carbon_cost"]) + float(ov(dynamic_overview, "碳排放成本"))
    rows.extend(
        [
            {"指标": "全天回填总成本", "数值": round(full_total, 2)},
            {"指标": "全天回填碳排放成本", "数值": round(full_carbon, 2)},
            {"指标": "全天回填总成本相对问题二变化", "数值": round(full_total - float(base_overview["总成本"]), 2)},
            {"指标": "全天回填碳排放成本相对问题二变化", "数值": round(full_carbon - float(base_overview["碳排放成本"]), 2)},
        ]
    )
    return pd.DataFrame(rows)

def main():
    print("读取问题二基准方案与问题三情景文件, 执行事件触发式滚动重调度...")
    q2_book, scene_book, base_dist_matrix, base_time_windows, base_coords, speed_scenarios = load_inputs()

    summary_df = q2_book["车辆路径"].copy()
    detail_df = q2_book["到达时间明细"].copy()
    overview2 = q2_book["方案总览"].copy()
    scene_df = scene_book["情景总览"].copy()
    event_all_df = scene_book["事件明细"].copy()

    baseline_routes = build_baseline_routes(summary_df, detail_df, base_dist_matrix, base_time_windows, speed_scenarios,
↪ base_coords)
    base_overview = {
        "总成本": float(overview2.loc[overview2["指标"] == "总成本", "数值"].iloc[0]),

```

```

    "碳排放成本": float(overview2.loc[overview2["指标"] == "碳排放成本", "数值"].iloc[0]),
}

scene_compare_rows = []
overview_all = []
type_all = []
dynamic_route_all = []
dynamic_detail_all = []
full_day_rows_all = []

with pd.ExcelWriter(OUT_PATH, engine="openpyxl") as writer:
    scene_df.to_excel(writer, sheet_name="情景总览", index=False)

    for _, scene in scene_df.sort_values("场景编号").iterrows():
        scene_id = scene["场景编号"]
        scene_name = scene["场景名称"]
        trigger_hour = parse_hour(scene["触发时间"])
        coords = deepcopy(base_coords)
        time_windows = deepcopy(base_time_windows)

        event_df = event_all_df[event_all_df["场景编号"] == scene_id].copy().sort_values("事件顺序")
        realtime_df, vehicle_states = summarize_realtime_state(baseline_routes, trigger_hour)
        vehicle_states = enrich_with_idle_fleet(vehicle_states, trigger_hour)
        fixed_routes, fixed_total = build_fixed_part(baseline_routes, trigger_hour, base_dist_matrix)
        candidate_df = build_candidate_demand_with_state(baseline_routes, vehicle_states)
        adjusted_demand_df, event_log_df = apply_events(candidate_df, event_df, time_windows, coords, vehicle_states,
        ↪ trigger_hour)
        dynamic_dist_matrix = build_dynamic_distance_matrix(base_dist_matrix, coords, event_df)
        q1.dist_matrix_global = dynamic_dist_matrix
        q2.q1.dist_matrix_global = dynamic_dist_matrix

        dynamic_routes, run_info = build_routes_for_dynamic(
            adjusted_demand_df,
            dynamic_dist_matrix,
            time_windows,
            coords,
            speed_scenarios,
            vehicle_states,
        )

        dynamic_overview, dynamic_type_df, dynamic_route_df, dynamic_detail_df = build_output_tables(
            dynamic_routes,
            dynamic_dist_matrix,
            time_windows,
            speed_scenarios,
            coords,
        )

        local_future_routes = [route for route in baseline_routes if vehicle_states[route["vehicle_no"]]["status"] ==
        ↪ "unstarted"]
        selected_cost = round(float(sum(route["cost"]["total"] for route in local_future_routes)), 2)
        selected_carbon = round(float(sum(route["cost"]["carbon"] for route in local_future_routes)), 2)
        selected_late = round(float(sum(route["cost"]["late_hours"] for route in local_future_routes)), 4)
        selected_tw = round(float(sum(route["cost"]["time_window"] for route in local_future_routes)), 2)
        dynamic_cost = float(dynamic_overview.loc[dynamic_overview["指标"] == "总成本", "数值"].iloc[0])
        dynamic_carbon = float(dynamic_overview.loc[dynamic_overview["指标"] == "碳排放成本", "数值"].iloc[0])
        dynamic_late = float(dynamic_overview.loc[dynamic_overview["指标"] == "总迟到 (h)", "数值"].iloc[0])
        dynamic_tw = float(dynamic_overview.loc[dynamic_overview["指标"] == "时间窗成本", "数值"].iloc[0])

        full_day_df = build_full_day_rows(base_overview, fixed_total, dynamic_overview)
        full_day_total = float(full_day_df.loc[full_day_df["指标"] == "全天回填总成本", "数值"].iloc[0])
        full_day_carbon = float(full_day_df.loc[full_day_df["指标"] == "全天回填碳排放成本", "数值"].iloc[0])

    compare_rows = [
        {"指标": "触发时间", "事件前": hour_to_time(trigger_hour), "事件后": hour_to_time(trigger_hour), "变化": ""},
        {"指标": "触发时未发车路线数", "事件前": len(local_future_routes), "事件后": len(dynamic_routes), "变化":
        ↪ len(dynamic_routes) - len(local_future_routes)},

```

```

{"指标": "动态需求客户数", "事件前": int(candidate_df["客户编号"].nunique()) if not candidate_df.empty else 0,
↪ "事件后": int(adjusted_demand_df["客户编号"].nunique()) if not adjusted_demand_df.empty else 0, "变化":
↪ (int(adjusted_demand_df["客户编号"].nunique()) if not adjusted_demand_df.empty else 0) -
↪ (int(candidate_df["客户编号"].nunique()) if not candidate_df.empty else 0)},
{"指标": "动态需求总重量(kg)", "事件前": round(float(candidate_df["重量"].sum()), 4) if not candidate_df.empty
↪ else 0.0, "事件后": round(float(adjusted_demand_df["重量"].sum()), 4) if not adjusted_demand_df.empty else
↪ 0.0, "变化": round((float(adjusted_demand_df["重量"].sum()) if not adjusted_demand_df.empty else 0.0) -
↪ (float(candidate_df["重量"].sum()) if not candidate_df.empty else 0.0), 4)},
{"指标": "动态需求总体积(m3)", "事件前": round(float(candidate_df["体积"].sum()), 4) if not candidate_df.empty
↪ else 0.0, "事件后": round(float(adjusted_demand_df["体积"].sum()), 4) if not adjusted_demand_df.empty else
↪ 0.0, "变化": round((float(adjusted_demand_df["体积"].sum()) if not adjusted_demand_df.empty else 0.0) -
↪ (float(candidate_df["体积"].sum()) if not candidate_df.empty else 0.0), 4)},
{"指标": "局部基准总成本", "事件前": selected_cost, "事件后": round(dynamic_cost, 2), "变化": round(dynamic_cost -
↪ selected_cost, 2)},
{"指标": "局部基准碳排放成本", "事件前": selected_carbon, "事件后": round(dynamic_carbon, 2), "变化":
↪ round(dynamic_carbon - selected_carbon, 2)},
{"指标": "局部基准总迟到(h)", "事件前": selected_late, "事件后": round(dynamic_late, 4), "变化":
↪ round(dynamic_late - selected_late, 4)},
{"指标": "局部基准时间窗成本", "事件前": selected_tw, "事件后": round(dynamic_tw, 2), "变化": round(dynamic_tw -
↪ selected_tw, 2)},
{"指标": "滚动优化前成本", "事件前": round(run_info["before"], 2), "事件后": round(run_info["after"], 2), "变化":
↪ round(run_info["after"] - run_info["before"], 2)},
{"指标": "全天回填总成本", "事件前": base_overview["总成本"], "事件后": round(full_day_total, 2), "变化":
↪ round(full_day_total - base_overview["总成本"], 2)},
{"指标": "全天回填碳排放成本", "事件前": base_overview["碳排放成本"], "事件后": round(full_day_carbon, 2), "变化":
↪ round(full_day_carbon - base_overview["碳排放成本"], 2)},
{"指标": "限行违规次数", "事件前": 0, "事件后": int(dynamic_overview.loc[dynamic_overview["指标"] ==
↪ "限行违规次数", "数值"].iloc[0]), "变化": ""},
{"指标": "词典序目标-限行违规", "事件前": "", "事件后": int(dynamic_overview.loc[dynamic_overview["指标"] ==
↪ "词典序目标-限行违规", "数值"].iloc[0]), "变化": ""},
{"指标": "词典序目标-迟到客户数", "事件前": "", "事件后": int(dynamic_overview.loc[dynamic_overview["指标"] ==
↪ "词典序目标-迟到客户数", "数值"].iloc[0]), "变化": ""},
{"指标": "词典序目标-总迟到(h)", "事件前": "", "事件后": float(dynamic_overview.loc[dynamic_overview["指标"] ==
↪ "词典序目标-总迟到(h)", "数值"].iloc[0]), "变化": ""},
]

meta_df = pd.DataFrame(
[
{"字段": "场景编号", "值": scene_id},
{"字段": "场景名称", "值": scene_name},
{"字段": "触发时间", "值": hour_to_time(trigger_hour)},
{"字段": "方法", "值": "以问题二为基线, 触发前固定; 触发后基于实时车辆释放时间、
↪ 车辆故障状态和剩余需求执行滚动重调度"},
{"字段": "已执行固定路线数", "值": len(fixed_routes)},
{"字段": "动态重构路线数", "值": len(dynamic_routes)},
]
)

baseline_summary_df = pd.DataFrame(
[
{
"路线编号": route["route_id"],
"车辆编号": route["vehicle_no"],
"车辆类型": route["vehicle"],
"发车时间": hour_to_time(route["cost"]["depart"]),
"返场时间": hour_to_time(route["cost"]["return"]),
"总成本": round(route["cost"]["total"], 2),
"碳排放成本": round(route["cost"]["carbon"], 2),
"总迟到 (h)": round(route["cost"]["late_hours"], 4),
"路径": "0 -> " + " -> ".join(str(stop["customer"]) for stop in route["stops"]) + " -> 0",
}
for route in local_future_routes
]
)

baseline_summary_df.to_excel(writer, sheet_name=f"{scene_id} 基准路线", index=False)
dynamic_overview.to_excel(writer, sheet_name=f"{scene_id} 动态总览", index=False)
dynamic_route_df.to_excel(writer, sheet_name=f"{scene_id} 动态路径", index=False)
dynamic_detail_df.to_excel(writer, sheet_name=f"{scene_id} 动态明细", index=False)

```

```

pd.DataFrame(compare_rows).to_excel(writer, sheet_name=f"{scene_id} 前后对比", index=False)
event_df.to_excel(writer, sheet_name=f"{scene_id} 事件", index=False)
event_log_df.to_excel(writer, sheet_name=f"{scene_id} 事件日志", index=False)
realtime_df.to_excel(writer, sheet_name=f"{scene_id} 实时状态", index=False)
full_day_df.to_excel(writer, sheet_name=f"{scene_id} 全天回填", index=False)
meta_df.to_excel(writer, sheet_name=f"{scene_id} 场景说明", index=False)

scene_compare_rows.append(
    {
        " 场景编号": scene_id,
        " 场景名称": scene_name,
        " 触发时间": hour_to_time(trigger_hour),
        " 事件数量": int(len(event_df)),
        " 问题二基准总成本": base_overview[" 总成本"],
        " 问题二基准碳排放成本": base_overview[" 碳排放成本"],
        " 局部基准总成本": selected_cost,
        " 动态重调度总成本": round(dynamic_cost, 2),
        " 局部基准碳排放成本": selected_carbon,
        " 动态重调度碳排放成本": round(dynamic_carbon, 2),
        " 局部基准总迟到 (h)": selected_late,
        " 动态重调度总迟到 (h)": round(dynamic_late, 4),
        " 局部基准路线数": len(local_future_routes),
        " 动态路线数": len(dynamic_routes),
        " 全天回填总成本": round(full_day_total, 2),
        " 全天回填碳排放成本": round(full_day_carbon, 2),
        " 全天总成本相对问题二变化": round(full_day_total - base_overview[" 总成本"], 2),
        " 全天碳排放成本相对问题二变化": round(full_day_carbon - base_overview[" 碳排放成本"], 2),
        " 限行违规次数": int(dynamic_overview.loc[dynamic_overview[" 指标"] == " 限行违规次数", " 数值"].iloc[0]),
        "词典序目标-限行违规": int(dynamic_overview.loc[dynamic_overview["指标"] == "词典序目标-限行违规",
        ↪ "数值"].iloc[0]),
        "词典序目标-迟到客户数": int(dynamic_overview.loc[dynamic_overview["指标"] == "词典序目标-迟到客户数",
        ↪ "数值"].iloc[0]),
        " 方法": " 问题二基线 + 实时车辆释放/故障约束 + 触发后未发车任务滚动重调度",
    }
)

dynamic_overview = dynamic_overview.copy()
dynamic_type_df = dynamic_type_df.copy()
dynamic_route_df = dynamic_route_df.copy()
dynamic_detail_df = dynamic_detail_df.copy()
full_day_df = full_day_df.copy()
dynamic_overview[" 场景编号"] = scene_id
dynamic_type_df[" 场景编号"] = scene_id
dynamic_route_df[" 场景编号"] = scene_id
dynamic_detail_df[" 场景编号"] = scene_id
full_day_df[" 场景编号"] = scene_id
overview_all.append(dynamic_overview)
type_all.append(dynamic_type_df)
dynamic_route_all.append(dynamic_route_df)
dynamic_detail_all.append(dynamic_detail_df)
full_day_rows_all.append(full_day_df)

scene_compare_df = pd.DataFrame(scene_compare_rows)
scene_compare_df.to_excel(writer, sheet_name=" 场景综合对比", index=False)
pd.concat(overview_all, ignore_index=True).to_excel(writer, sheet_name=" 情景总览汇总", index=False)
pd.concat(type_all, ignore_index=True).to_excel(writer, sheet_name=" 车辆使用汇总", index=False)
pd.concat(dynamic_route_all, ignore_index=True).to_excel(writer, sheet_name=" 动态路径汇总", index=False)
pd.concat(dynamic_detail_all, ignore_index=True).to_excel(writer, sheet_name=" 动态明细汇总", index=False)
pd.concat(full_day_rows_all, ignore_index=True).to_excel(writer, sheet_name=" 全天回填汇总", index=False)

compare_df = pd.DataFrame(scene_compare_rows)
with pd.ExcelWriter(COMPARRE_PATH, engine="openpyxl") as writer:
    compare_df.to_excel(writer, sheet_name=" 场景对比", index=False)
    compare_df[
        [
            " 场景编号",
            " 场景名称",
            " 事件数量",
            " 动态重调度总成本",

```

```

        " 动态重调度碳排放成本",
        " 动态重调度总迟到 (h)",
        " 全天回填总成本",
        " 全天回填碳排放成本",
        " 全天总成本相对问题二变化",
        " 全天碳排放成本相对问题二变化",
    ]
    ].to_excel(writer, sheet_name=" 关键指标", index=False)

    print(f" 动态调度方案已输出: {os.path.relpath(OUT_PATH, BASE_DIR)}")
    print(f" 事件前后对比已输出: {os.path.relpath(COMPARE_PATH, BASE_DIR)}")
    print(compare_df[["场景编号", "动态重调度总成本", "动态重调度总迟到(h)", "全天回填总成本",
    ↪ "限行违规次数"]].to_string(index=False))

if __name__ == "__main__":
    main()

```

B.6 敏感性与鲁棒性分析程序：4-敏感性与鲁棒性分析.py

```

from pathlib import Path

import numpy as np
import pandas as pd

try:
    import matplotlib.pyplot as plt
    from matplotlib import font_manager
except ModuleNotFoundError:
    plt = None
    font_manager = None

BASE_DIR = Path(__file__).resolve().parent
PROC_DIR = BASE_DIR / " 过程"

Q2_PATH = PROC_DIR / "2-问题二限行政策调度方案.xlsx"
Q3_COMPARE_PATH = PROC_DIR / "3-问题三事件前后对比.xlsx"
Q3_POST_PATH = PROC_DIR / "3-问题三综合对比.xlsx"

OUT_XLSX = PROC_DIR / "4-敏感性与鲁棒性分析.xlsx"
OUT_SENS_PNG = PROC_DIR / "4-敏感性分析图.png"
OUT_ROBUST_PNG = PROC_DIR / "4-鲁棒性分析图.png"

def setup_chinese_font():
    if plt is None:
        return
    candidates = ["Microsoft YaHei", "SimHei", "SimSun", "Noto Sans CJK SC", "Source Han Sans SC"]
    available = {font.name for font in font_manager.fontManager.ttflist}
    for name in candidates:
        if name in available:
            plt.rcParams["font.sans-serif"] = [name]
            break
    plt.rcParams["axes.unicode_minus"] = False
    plt.rcParams["figure.facecolor"] = "white"
    plt.rcParams["axes.facecolor"] = "white"

def read_overview(path):
    df = pd.read_excel(path, sheet_name=" 方案总览")
    return {row["指标"]: float(row["数值"]) for _, row in df.iterrows() if isinstance(row["数值"], (int, float, np.integer,
    ↪ np.floating))}

def build_sensitivity_table(q2_overview):
    base_total = q2_overview[" 总成本"]
    base_carbon = q2_overview[" 碳排放成本"]
    base_time = q2_overview[" 时间窗成本"]

```



```

scenarios = [
    (" 碳排放成本系数", -20, base_total - 0.20 * base_carbon, base_carbon * 0.80, base_time),
    (" 碳排放成本系数", -10, base_total - 0.10 * base_carbon, base_carbon * 0.90, base_time),
    (" 碳排放成本系数", 10, base_total + 0.10 * base_carbon, base_carbon * 1.10, base_time),
    (" 碳排放成本系数", 20, base_total + 0.20 * base_carbon, base_carbon * 1.20, base_time),
    (" 迟到惩罚系数", -20, base_total - 0.20 * base_time, base_carbon, base_time * 0.80),
    (" 迟到惩罚系数", -10, base_total - 0.10 * base_time, base_carbon, base_time * 0.90),
    (" 迟到惩罚系数", 10, base_total + 0.10 * base_time, base_carbon, base_time * 1.10),
    (" 迟到惩罚系数", 20, base_total + 0.20 * base_time, base_carbon, base_time * 1.20),
]

rows = []
for param, pct, total, carbon, time_cost in scenarios:
    rows.append(
        {
            " 扰动参数": param,
            " 扰动幅度": f"{pct:+d}%",
            " 调整后总成本": round(total, 2),
            " 总成本变化": round(total - base_total, 2),
            " 总成本变化率 (%)": round((total - base_total) / base_total * 100, 4),
            " 调整后碳排放成本": round(carbon, 2),
            " 调整后时间窗成本": round(time_cost, 2),
        }
    )

return pd.DataFrame(rows)

def build_robustness_table(compare_df, merged_df):
    robust = compare_df.copy()
    if " 最大迟到" in merged_df.columns:
        robust = robust.merge(merged_df[[" 场景编号", " 最大迟到", " 平均载重率"]], on=" 场景编号", how="left")
    robust[" 限行可行"] = robust[" 限行违规次数"].apply(lambda x: " 是" if int(x) == 0 else " 否")
    robust[" 全天成本改善"] = -robust[" 全天总成本相对问题二变化"]
    robust[" 全天碳成本改善"] = -robust[" 全天碳排放成本相对问题二变化"]
    robust[" 成本韧性率 (%)"] = (1 - robust[" 全天回填总成本"] / robust[" 问题二基准总成本"]) * 100
    robust[" 碳成本韧性率 (%)"] = (1 - robust[" 全天回填碳排放成本"] / robust[" 问题二基准碳排放成本"]) * 100

    cols = [
        " 场景编号",
        " 场景名称",
        " 限行违规次数",
        " 限行可行",
        " 动态重调度总成本",
        " 动态重调度总迟到 (h)",
        " 最大迟到",
        " 全天回填总成本",
        " 全天成本改善",
        " 全天回填碳排放成本",
        " 全天碳成本改善",
        " 成本韧性率 (%)",
        " 碳成本韧性率 (%)",
    ]

    robust = robust[[col for col in cols if col in robust.columns]].copy()
    for col in robust.select_dtypes(include=[np.number]).columns:
        robust[col] = robust[col].round(4)
    return robust

def plot_sensitivity(sens_df):
    if plt is None:
        return False
    setup_chinese_font()
    fig, ax = plt.subplots(figsize=(10.5, 5.8), dpi=220)
    colors = {" 碳排放成本系数": "#2a9d8f", " 迟到惩罚系数": "#c97c5d"}
    for param, group in sens_df.groupby(" 扰动参数"):
        xs = [int(v.replace("%", "")) for v in group[" 扰动幅度"]]
        ax.plot(xs, group[" 总成本变化率 (%)"], marker="o", linewidth=2.2, color=colors[param], label=param)
        for x, y in zip(xs, group[" 总成本变化率 (%)"]):
            ax.text(x, y + 0.015, f"{y:.2f}%", ha="center", va="bottom", fontsize=8)

```

```

ax.axhline(0, color="#666666", linewidth=0.9)
ax.set_xlabel(" 参数扰动幅度")
ax.set_ylabel(" 总成本变化率")
ax.set_xticks([-20, -10, 10, 20])
ax.grid(axis="y", linestyle="--", alpha=0.35)
ax.legend(frameon=False, ncol=2, loc="upper center")
fig.tight_layout()
fig.savefig(OUT_SENS_PNG, bbox_inches="tight")
plt.close(fig)
return True

def plot_robustness(robust_df):
    if plt is None:
        return False
    setup_chinese_font()
    x = np.arange(len(robust_df))
    fig, ax1 = plt.subplots(figsize=(11.2, 6.2), dpi=220)
    bars = ax1.bar(x, robust_df[" 成本韧性率 (%)"], width=0.42, color="#2a9d8f", edgecolor="white", label=" 成本韧性率")
    ax1.axhline(0, color="#666666", linewidth=0.9)
    ax1.set_ylabel(" 成本韧性率 / %")
    ax1.set_xticks(x)
    ax1.set_xticklabels(robust_df[" 场景编号"] + "\n" + robust_df[" 场景名称"], fontsize=9)
    ax1.grid(axis="y", linestyle="--", alpha=0.35)
    ax1.bar_label(bars, fmt="%.2f", fontsize=8, padding=3)

    ax2 = ax1.twinx()
    ax2.plot(x, robust_df[" 动态重调度总迟到 (h)"], color="#b91c1c", marker="o", linewidth=2.0, label=" 动态总迟到")
    ax2.set_ylabel(" 动态总迟到 / h")

    h1, l1 = ax1.get_legend_handles_labels()
    h2, l2 = ax2.get_legend_handles_labels()
    ax1.legend(h1 + h2, l1 + l2, frameon=False, ncol=2, loc="upper center")
    fig.tight_layout()
    fig.savefig(OUT_ROBUST_PNG, bbox_inches="tight")
    plt.close(fig)
    return True

def main():
    q2_overview = read_overview(Q2_PATH)
    compare_df = pd.read_excel(Q3_COMPARE_PATH, sheet_name=" 场景对比")
    merged_df = pd.read_excel(Q3_POST_PATH, sheet_name=" 场景综合对比")

    sens_df = build_sensitivity_table(q2_overview)
    robust_df = build_robustness_table(compare_df, merged_df)

    summary_df = pd.DataFrame(
        [
            {"指标": " 问题二基准总成本", "数值": round(q2_overview[" 总成本"], 2)},
            {"指标": " 问题二基准碳排放成本", "数值": round(q2_overview[" 碳排放成本"], 2)},
            {"指标": " 问题二基准时间窗成本", "数值": round(q2_overview[" 时间窗成本"], 2)},
            {"指标": " 鲁棒性检验场景数", "数值": len(robust_df)},
            {"指标": " 限行零违规场景数", "数值": int((robust_df[" 限行违规次数"] == 0).sum())},
            {"指标": " 全天总成本改善场景数", "数值": int((robust_df[" 全天成本改善"] > 0).sum())},
            {"指标": " 全天碳成本改善场景数", "数值": int((robust_df[" 全天碳成本改善"] > 0).sum())},
            {"指标": " 平均成本韧性率 (%)", "数值": round(float(robust_df[" 成本韧性率 (%)"].mean()), 4)},
            {"指标": " 平均碳成本韧性率 (%)", "数值": round(float(robust_df[" 碳成本韧性率 (%)"].mean()), 4)},
        ]
    )

    with pd.ExcelWriter(OUT_XLSX, engine="openpyxl") as writer:
        summary_df.to_excel(writer, sheet_name=" 分析摘要", index=False)
        sens_df.to_excel(writer, sheet_name=" 敏感性分析", index=False)
        robust_df.to_excel(writer, sheet_name=" 鲁棒性分析", index=False)

    sens_png_ok = plot_sensitivity(sens_df)
    robust_png_ok = plot_robustness(robust_df)

```

```
print(f" 敏感性与鲁棒性分析表: {OUT_XLSX.relative_to(BASE_DIR)}")
if sens_png_ok and robust_png_ok:
    print(f" 敏感性分析图: {OUT_SENS_PNG.relative_to(BASE_DIR)}")
    print(f" 鲁棒性分析图: {OUT_ROBUST_PNG.relative_to(BASE_DIR)}")
else:
    print(" 当前 Python 环境缺少 matplotlib, 仅输出 Excel 汇总表。")

if __name__ == "__main__":
    main()
```